



Department of Electronic & Telecommunication Engineering
University of Moratuwa, Sri Lanka

EN2160 - Electronic Design Realization Software Stack for the AMR

220077L - Budvin M.P.L	220197E - Gunathilaka K.L
220299T - Kalubowila K.A.T.S	220439B - Pahasara L.P.P
220449F - Pathirana P.N.A	220468L - Perera M.D.R.N
220533H - Ravishan B.B.N.	220542J - Sadaruwan I.J.M.J
220551K - Samaramanna M.A	220701X - Wickramasinghe S.D

July 10, 2025

Contents

1	Introduction	2
2	Lower Layer Controller (MCU)	2
2.1	Configuration & Hardware Abstraction	2
2.2	Analog Sensor Interface	7
2.3	IMU (BNO055) Low-Level Driver	9
2.4	Encoder & Odometry	15
2.5	Serial-Stream Sensor Parser	21
2.6	Motor Control	21
2.7	Motion Profiling	24
2.8	System Timing	30
2.9	Main Control Loop	32
3	Software Stack Implemented on the Jetson Nano	42
3.1	Configuration & Utilities	42
3.2	Communication Modules	45
3.2.1	Serial Communication Handler	45
3.2.2	UDP Communication Handler	57
3.3	Sensor Data Acquisition	62
3.3.1	LiDAR Data Receiver	62
3.3.2	Serial-Stream Sensor Parser	80
3.4	Localization & Sensor Fusion	81
3.4.1	Extended Kalman Filter (EKF)	81
3.4.2	Localization Orchestration	95
3.5	Path Planning	98
3.5.1	C++ Implementation (A* & PRM)	98
3.5.2	Python Reference Implementation	106
3.6	Mapping (SLAM)	110
3.6.1	Online SLAM (Real-Time Occupancy Grid)	110
3.6.2	Offline SLAM (Batch Scan-to-Map Alignment)	118
3.6.3	Comparison: Online vs. Offline SLAM	138
3.7	Main Control Loop	138
3.7.1	Application Entry	138
4	Remote Monitoring and Control Interface	143
4.0.1	Graphical User Interface	143
4.0.2	Command-Line Interface	149
5	Conclusion	151

1 Introduction

This document presents the end-to-end software architecture of our Autonomous Mobile Robot (AMR), spanning three core domains:

- **Lower Layer Controller (MCU):** Real-time firmware on the ATmega32U4 handles millisecond-scale loops for sensor acquisition, odometry, motion profiling, and motor actuation.
- **Jetson Nano Software Stack:** High-level modules perform communication (serial and UDP), sensor fusion (EKF), mapping (online and offline SLAM), and path planning (A*, PRM, and hybrid optimizations).
- **Remote Monitoring & Control Interface:** Both a PyQt-based GUI and a lightweight CLI tool enable live telemetry visualization, PID parameter tuning, and keyboard teleoperation over UDP.

Together, these components ensure deterministic control via a 10 ms main loop that unifies low-level sensing and actuation; flexible mapping where online SLAM delivers reactive occupancy grids and offline SLAM provides globally consistent maps with loop closures; optimal navigation through sampling-based and grid-based planners that craft smooth, efficient trajectories tailored to the environment; and seamless teleoperation that allows users to monitor and control the robot from either a full-featured dashboard or a simple terminal interface.

2 Lower Layer Controller (MCU)

The ATmega32U4 microcontroller serves as the hard-real-time core of our AMR, executing precise, millisecond-scale loops to read sensors, update odometry, manage motion profiles, drive stepper motors, and maintain a reliable USB link to the Jetson Nano. Its firmware is organized into well-defined modules, each consisting of a header (.h) for interface declarations and a source file (.c) for implementation. All hardware pin mappings, timing constants, and robot-specific parameters are centralized in a configuration header, ensuring that changes to wiring or mechanical design require edits in only one place.

2.1 Configuration & Hardware Abstraction

A robust embedded system begins with a clear mapping between the microcontroller’s peripherals and the robot’s physical components. The `config.h` header gathers all such mappings and constants:

- **Clock & Core Settings:** Sets system clock to 16 MHz (`F_CPU`) for accurate timing, includes essential AVR libraries.
- **Pin Assignments:** mappings for stepper drivers, encoders, ADC channels (battery, cliff), emergency stop, and I²C lines.
- **Timing & Sampling:** Timer and ADC prescalers, sample counts for noise reduction.
- **Physical & Teleop Defaults:** wheel diameter, track width, gear ratio, encoder CPR, derived mm/tick and deg differential, default teleop setpoints.

```

1  /*
2  *  config.h
3  *
4  *  Created: 5/6/2025 4:35:44 AM
5  *  Author: Endeavor360
6  */
7
8  #ifndef CONFIG_H_
9  #define CONFIG_H_
10
11 // MCU clock
12 #define F_CPU 16000000UL
13
14 #include <avr/io.h>
15 #include <util/delay.h>
16 #include <avr/interrupt.h>
17 #include <stdbool.h>
18 #include <stdint.h>
19
20
21 //-----MOTORS
22
23 // Steps per revolution (full-step mode)
24 #define STEPS_PER_REV 200U
25
26 /* ----- LEFT motor (M1) Û now on Timer-3 / OC3A (PC6 / Arduino D5
27    ↳ ) ----- */
28 // PUL- -> D5 = PC6 / OC3A
29 // DIR- -> D12 = PD6
30 // ENA- -> D10 = PB6 (active-HIGH in inverted logic)
31 #define LEFT_PUL_DDR DDRC
32 #define LEFT_PUL_PORT PORTC
33 #define LEFT_PUL_BIT PC6 /* OC3A */
34
35 #define LEFT_DIR_DDR DDRD
36 #define LEFT_DIR_PORT PORTD
37 #define LEFT_DIR_BIT PD6
38
39 #define LEFT_ENA_DDR DDRB
40 #define LEFT_ENA_PORT PORTB
41 #define LEFT_ENA_BIT PB6
42
43 // Right motor pins (Arduino Leonardo) M2
44 // PUL- -> D9 = PB5 / OC1A
45 // DIR- -> D8 = PB4
46 // ENA- -> D6 = PD7 (active High; explained above)
47 #define RIGHT_PUL_DDR DDRB
48 #define RIGHT_PUL_PORT PORTB
49 #define RIGHT_PUL_BIT PB5
50
51 #define RIGHT_DIR_DDR DDRD
52 #define RIGHT_DIR_PORT PORTD
53 #define RIGHT_DIR_BIT PD4
54
55 #define RIGHT_ENA_DDR DDRD
56 #define RIGHT_ENA_PORT PORTD

```

```

56 #define RIGHT_ENA_BIT    PD7
57
58 #ifndef _BV //this is just to silence the shitty linter in microchip
59     ↪ studio
60 #define _BV(bit) (1 << (bit))
61 #endif
62
63 /* ----- timer prescaler / clock divisors
64     ↪ ----- */
65 #define CLOCK_DIVISOR_TIMER1    1024U
66 #define PRE_SCALE_TIMER1        (_BV(CS12) | _BV(CS10))
67
68 #define CLOCK_DIVISOR_TIMER3    1024U
69 #define PRE_SCALE_TIMER3        (_BV(CS32) | _BV(CS30))
70
71 //-----USB-CDC
72     ↪ -----
73 // constants corresponding to the various serial parameters
74 #define USB_SERIAL_DTR 0x01
75 #define USB_SERIAL_RTS 0x02
76 #define USB_SERIAL_1_STOP 0
77 #define USB_SERIAL_1_5_STOP 1
78 #define USB_SERIAL_2_STOP 2
79 #define USB_SERIAL_PARITY_NONE 0
80 #define USB_SERIAL_PARITY_ODD 1
81 #define USB_SERIAL_PARITY_EVEN 2
82 #define USB_SERIAL_PARITY_MARK 3
83 #define USB_SERIAL_PARITY_SPACE 4
84 #define USB_SERIAL_DCD 0x01
85 #define USB_SERIAL_DSR 0x02
86 #define USB_SERIAL_BREAK 0x04
87 #define USB_SERIAL_RI 0x08
88 #define USB_SERIAL_FRAME_ERR 0x10
89 #define USB_SERIAL_PARITY_ERR 0x20
90 #define USB_SERIAL_OVERRUN_ERR 0x40
91
92 #define EP_TYPE_CONTROL 0x00
93 #define EP_TYPE_BULK_IN 0x81
94 #define EP_TYPE_BULK_OUT 0x80
95 #define EP_TYPE_INTERRUPT_IN 0xC1
96 #define EP_TYPE_INTERRUPT_OUT 0xC0
97 #define EP_TYPE_ISOCHRONOUS_IN 0x41
98 #define EP_TYPE_ISOCHRONOUS_OUT 0x40
99 #define EP_SINGLE_BUFFER 0x02
100 #define EP_DOUBLE_BUFFER 0x06
101 #define EP_SIZE(s) ((s) == 64 ? 0x30 : ((s) == 32 ? 0x20 : ((s) == 16 ?
102     ↪ 0x10 : 0x00)))
103
104 #define MAX_ENDPOINT 4
105
106 #define LSB(n) (n & 255)
107 #define MSB(n) ((n >> 8) & 255)
108
109 #define HW_CONFIG() (UHWCON = 0x01)
110
111 #ifndef M1

```

```

110 #define PLL_CONFIG() (PLLCSR = 0x02) // fixed to 8MHz clock
111 #else
112 #define PLL_CONFIG() (PLLCSR = 0x12) // 0001 0010 For a 16MHz clock
113 #endif
114
115 #define USB_CONFIG() (USBCON = ((1 << USBE) | (1 << OTGPADE)))
116 #define USB_FREEZE() (USBCON = ((1 << USBE) | (1 << FRZCLK)))
117
118 // standard control endpoint request types
119 #define GET_STATUS 0
120 #define CLEAR_FEATURE 1
121 #define SET_FEATURE 3
122 #define SET_ADDRESS 5
123 #define GET_DESCRIPTOR 6
124 #define GET_CONFIGURATION 8
125 #define SET_CONFIGURATION 9
126 #define GET_INTERFACE 10
127 #define SET_INTERFACE 11
128 // HID (human interface device)
129 #define HID_GET_REPORT 1
130 #define HID_GET_PROTOCOL 3
131 #define HID_SET_REPORT 9
132 #define HID_SET_IDLE 10
133 #define HID_SET_PROTOCOL 11
134 // CDC (communication class device)
135 #define CDC_SET_LINE_CODING 0x20
136 #define CDC_GET_LINE_CODING 0x21
137 #define CDC_SET_CONTROL_LINE_STATE 0x22
138
139
140
141 //-----IMU
142 ↪ -----
143 /* BNO055 I2C addresses (BOOT pin low = ADDR = 0x28, high = 0x29) */
144 #define BNO055_ADDR_A 0x28u
145 #define BNO055_ADDR_B 0x29u
146 #define BNO055_I2C_ADDR BNO055_ADDR_A /**< change if needed */
147
148 /* Desired I2C speed (Hz) */
149 #define TWI_SCL_HZ 100000UL
150
151 /* ----- hard reset via MCU GPIO ----- */
152 #define BNO_RST_DDR DDRF
153 #define BNO_RST_PORT PORTF
154 #define BNO_RST_PIN PF7
155
156 //-----ANALOG
157 ↪ -----
158 /* 0-7 = PF0-7, 8-13 = PB4-7 */
159 #define ADC_CH_BAT_MAIN 0 // PF0 ũ main battery divider
160 #define ADC_CH_BAT_AUX 1 // PF1 ũ aux battery divider
161 #define ADC_CH_CLIFF_LEFT 4 // PF4 ũ Sharp IR left
162 #define ADC_CH_CLIFF_FRONT 5 // PF5 ũ Sharp IR centre
163 #define ADC_CH_CLIFF_RIGHT 6 // PF6 ũ Sharp IR right
164
165 /* ----- Conversion parameters ----- */
166 #define ADC_NUM_SAMPLES 4 // simple software average

```

```

166 #define ADC_PRESCALER_BITS ((1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0)) /* a128
    ↪ */
167
168 /* ----- Battery-scaling maths ----- */
169 * VBAT = ADCraw * (AVcc / 1023) * (R1+R2) / R2
170 * Put the resistor ratios here so the driver can give
171 * you millivolts directly. Example 100 k? / 33 k?. */
172 #define BAT_DIV_FACTOR_NUM (1) // (R1+R2)*100 / R2
173 #define BAT_DIV_FACTOR_DEN (1)
174
175
176
177 /* ===== ENCODERS (LEFT & RIGHT) ===== */
178
179 /* ---- LEFT encoder ---- */
180 #define ENC_L_A_DDR DDRD
181 #define ENC_L_A_PORT PORTD
182 #define ENC_L_A_PINREG PIND
183 #define ENC_L_A_BIT 2 // PD2 -> INTO (D1)
184
185 #define ENC_L_B_DDR DDRD
186 #define ENC_L_B_PORT PORTD
187 #define ENC_L_B_PINREG PIND
188 #define ENC_L_B_BIT 3 // PD3 -> INT1 (D0)
189
190 /* ---- RIGHT encoder ---- */
191 #define ENC_R_B_DDR DDRE
192 #define ENC_R_B_PORT PORTE
193 #define ENC_R_B_PINREG PINE
194 #define ENC_R_B_BIT 6 // PE6 -> INT6 (D7)
195
196 #define ENC_R_A_DDR DDRB
197 #define ENC_R_A_PORT PORTB
198 #define ENC_R_A_PINREG PINB
199 #define ENC_R_A_BIT 4 // PB4 -> PCINT4 (D8)
200
201 /* ---- Emergency button (shared PCINT) ---- */
202 #define EMG_BTN_DDR DDRB
203 #define EMG_BTN_PORT PORTB
204 #define EMG_BTN_PINREG PINB
205 #define EMG_BTN_BIT 7 // PB7 -> PCINT7 (D11)
206
207 /* ===== PROFILER PARAMETERS ===== */
208
209 #define ENCODER_PPR 1000U // quadrature pulses per
    ↪ channel
210 #define WHEEL_DIAMETER_MM 200.0f // wheel diameter [mm]
211 #define WHEEL_BASE_MM 500.0f // track width: distance
    ↪ between wheels [mm]
212 #define GEAR_RATIO 10
213
214 #define MM_PER_ROTATION (M_PI * WHEEL_DIAMETER_MM)
215 #define DEG_PER_MM_DIFF (180.0f / (M_PI * WHEEL_BASE_MM))
216
217 #define LOOP_TIME 10
218
219 // Teleoperator Mode - Distance and Angles
220 #define FORWARD_DIST 400.0f // mm - Forward movement per

```

```

221 ↪ command
#define BACKWARD_DIST      400.0f    // mm - Backward movement per
222 ↪ command
#define LEFT_TURN_ANGLE    45.0f     // degrees - CCW rotation per
223 ↪ command
#define RIGHT_TURN_ANGLE   45.0f     // degrees - CW rotation per
224 ↪ command
225 // Teleoperator Mode - Speeds
226 #define TELEOP_SPEED      300.0f    // mm/s - Linear speed
227 #define TELEOP_OMEGA      50.0f     // deg/s - Angular speed
228
229 // Teleoperator Mode - Accelerations
230 #define TELEOP_ACC        200.0f     // mm/s2 - Linear acceleration
231 #define TELEOP_ALPHA      30.0f     // deg/s2 - Angular acceleration
232
233 #endif // CONFIG_H

```

Listing 1: Global configuration and pin assignments (config.h)

2.2 Analog Sensor Interface

Accurate battery monitoring and cliff-detection rely on precise ADC readings. The driver in `analog.h/analog.c` initializes the ADC, samples channels multiple times, and converts raw counts to physical units:

- **Initialization:** `analog_init()` sets reference to `AVcc` and disables digital inputs on analog pins.
- **Sampling:** `analog_read_raw(channel)` performs hardware averaging over `ADC_NUM_SAMPLES`.
- **Battery Level Measurement:** Converts raw ADC values via resistor-divider to millivolts for two battery packs.
- **Cliff Sensor Readings:** Reads three IR cliff detectors, normalizing voltages to a consistent scale for drop-off detection.

```

1  /*
2  * analog.h Ħ ADC sampler API (header?only stub)
3  * -----
4  * Placeholder prototypes for IR distance and battery monitor helpers.
5  * Implement the definitions in adc.c later.
6  */
7
8  #ifndef ANALOG_H_
9  #define ANALOG_H_
10
11 #include "config.h"
12 #include <stdint.h>
13
14 /* initialise hardware (call once at start-up) */
15 void      analog_init(void);
16
17 /* blocking single-shot read of an arbitrary ADC channel */
18 uint16_t  analog_read_raw(uint8_t channel);
19
20 /* helpers that deliver already-scaled values -----*/

```



```

21 uint16_t analog_get_battery_1_mV(void);
22 uint16_t analog_get_battery_2_mV(void);
23
24 uint16_t analog_get_cliff_left(void);
25 uint16_t analog_get_cliff_front(void);
26 uint16_t analog_get_cliff_right(void);
27
28 #endif /* ANALOG_H_ */

```

Listing 2: ADC interface for battery and cliff sensors (analog.h)

```

1  /*
2   * analog.c
3   *
4   * Created: 5/6/2025 4:37:23 AM
5   * Author: Endeavor360
6   */
7
8  #include "analog.h"
9  #include <avr/io.h>
10
11 static inline void adc_select_channel(uint8_t ch)
12 {
13     /* For channels 0-7, MUX[3:0]=ch ; for 8-13 set MUX5 in ADCSRB */
14     if (ch <= 7) {
15         ADMUX = (ADMUX & 0xF0) | ch;           // keep REFS & ADLAR
16         ADCSRB &= ~(1<<MUX5);
17     } else {
18         ADMUX = (ADMUX & 0xF0) | (ch - 8);
19         ADCSRB |= (1<<MUX5);
20     }
21 }
22
23 /* ----- */
24 void analog_init(void)
25 {
26     /* AVcc reference, right-adjust, start on channel 0 */
27     ADMUX = (1<<REFS0);           /* AVcc with ext. cap on AREF */
28     adc_select_channel(0);
29
30     /* prescaler, enable, no free-running */
31     ADCSRA = (1<<ADEN) | ADC_PRESCALER_BITS;
32
33     /* Disable digital input buffers on the used analog pins to save
34      * → power/noise */
35     DIDR0 = (1<<ADC0D) | (1<<ADC1D) | (1<<ADC4D) | (1<<ADC5D) | (1<<
36      * → ADC6D);
37
38 }
39
40 /* blocking, software-averaged read ----- */
41 uint16_t analog_read_raw(uint8_t channel)
42 {
43     uint32_t acc = 0;
44
45     adc_select_channel(channel);
46
47     for (uint8_t i = 0; i < ADC_NUM_SAMPLES; ++i) {
48         ADCSRA |= (1<<ADSC);           /* start conversion */
49     }
50 }

```

```

46     while (ADCSRA & (1<<ADSC)) {;}           /* wait until ADSC clears */
47     acc += ADC;                               /* read ADCL then ADCH */
48 }
49 return (uint16_t)(acc / ADC_NUM_SAMPLES);
50 }
51
52 /* ----- convenience wrappers ----- */
53 static inline uint16_t to_millivolt(uint16_t adc)
54 {
55     /* (adc * 1100 mV / 1023) * scale-factor */
56     uint32_t mv = (uint32_t)adc * 1100UL / 1023UL; /* ? AVcc=3.3 V ?
57     ↪ update */
58     mv = mv * BAT_DIV_FACTOR_NUM / BAT_DIV_FACTOR_DEN;
59     return (uint16_t)mv;
60 }
61
62 uint16_t analog_get_battery_1_mV(void)
63 {
64     return to_millivolt(analog_read_raw(ADC_CH_BAT_MAIN));
65 }
66
67 uint16_t analog_get_battery_2_mV(void)
68 {
69     return to_millivolt(analog_read_raw(ADC_CH_BAT_AUX));
70 }
71
72 uint16_t analog_get_cliff_left (void){ return analog_read_raw(
73     ↪ ADC_CH_CLIFF_LEFT); }
74
75 uint16_t analog_get_cliff_front(void){ return analog_read_raw(
76     ↪ ADC_CH_CLIFF_FRONT); }
77
78 uint16_t analog_get_cliff_right(void){ return analog_read_raw(
79     ↪ ADC_CH_CLIFF_RIGHT); }

```

Listing 3: ADC implementation with averaging (analog.c)

2.3 IMU (BNO055) Low-Level Driver

The BNO055 IMU is accessed via low-level TWI routines, offering orientation and acceleration data with minimal overhead:

- **TWI Primitives:** Initialization and read/write functions with status checks.
- **Sensor Configuration:** GPIO reset and entering NDOF fusion mode.
- **Data Acquisition:** Euler angles, angular rates, and linear accelerations.
- **Calibration:** Status polling and offset read/write.

```

1 /**
2  * @file bno055_ll.h
3  * @brief Low-level BNO055 IMU driver for ATmega32U4 (register-only, no
4  *     ↪ Wire).
5  *
6  * Author : Kiran Gunathilaka (May 2025)
7  *
8  * TWI overview (ATmega32U4, DS40002085A, 25):
9  *     SCL freq = F_CPU / (16 + 2*TWBR*4^TWPS)

```

```

10  *      Write sequence : START ? SLA+W ? reg ? data ? STOP
11  *      Read sequence  : START ? SLA+W ? reg
12  *                        REPEATED START ? SLA+R ? data ? NACK ? STOP
13
14  */
15  #ifndef BNO055_LL_H_
16  #define BNO055_LL_H_
17
18  #include "config.h"
19  #include <avr/io.h>
20  #include <util/delay.h>
21  #include <stdint.h>
22  #include <stdbool.h>
23
24
25
26  /*
27  ↪ -----
28  ↪      */
29  /*                                public driver prototypes
30  ↪                                */
31  /*
32  ↪ -----
33  ↪      */
34  void twi_init(void); /* 25.6
35  ↪ */
36  bool twi_write(uint8_t sla, const uint8_t *buf, uint8_t len); /* low
37  ↪ lvl */
38  bool twi_read(uint8_t sla, uint8_t *buf, uint8_t len); /* low
39  ↪ lvl */
40
41  bool bno055_init(void); /* reset + NDOF (blocking) */
42  bool bno055_write8(uint8_t reg, uint8_t val);
43  bool bno055_read8(uint8_t reg, uint8_t *val);
44  bool bno055_read(uint8_t reg, uint8_t *buf, uint8_t len);
45  void bno055_gpio_reset(void); /* toggle external RST pin */
46
47  void bno055_get_euler(int16_t *h, int16_t *r, int16_t *p); /* deg/16 */
48  void bno055_get_omega (int16_t *gx, int16_t *gy, int16_t *gz); /* ř/
49  ↪ sũ16 */
50  void bno055_get_accel (int16_t *ax, int16_t *ay, int16_t *az); /* m/
51  ↪ sšũ100 */
52  bool bno055_is_fully_calibrated(void);
53
54  bool bno055_apply_offsets(const uint8_t calib[22]); /* write 22-byte
55  ↪ blob */
56  bool bno055_read_offsets(uint8_t calib[22]); /* read 22-byte
57  ↪ blob */
58
59  #endif /* BNO055_LL_H_ */

```

Listing 4: BNO055 low-level I2C interface (bno055_ll.h)

```

1  /**
2   * @file bno055_ll.c
3   * @brief Implementation of low-level BNO055 IMU driver for ATmega32U4
4   *        using only the TWI registers (see datasheet pp. 225-255).
5   */
6
7  #include "bno055_ll.h"
8
9  static const uint8_t offset_reg_first = 0x55; /* ACCEL_OFFSET_X_LSB */
10 static const uint8_t offset_reg_last  = 0x6A; /* MAG_RADIUS_MSB */
11
12 /* LOCAL MACROS */
13 #define TW_START (1u << TWSTA)
14 #define TW_STOP (1u << TWSTO)
15 #define TW_INT_FLAG (1u << TWINT)
16 #define TW_ENABLE (1u << TWEN)
17 #define TW_ACK (1u << TWEA)
18
19 /* TW_STATUS = TWSR & 0xF8 (if 25.7.4) */
20 #define TW_STATUS (TWSR & 0xF8u)
21
22 /* INTERNAL UTILITIES */
23 static inline void twi_wait(void)
24 {
25     while (!(TWSR & TW_INT_FLAG))
26     { /* wait for TWINT set */
27     }
28 }
29
30 static bool twi_start(uint8_t addr_rw)
31 {
32     TWSR = TW_INT_FLAG | TW_START | TW_ENABLE;
33     twi_wait();
34     uint8_t st = TW_STATUS;
35     if (st != 0x08 && st != 0x10)
36         return false; /* START / REP START */
37
38     /* load SLA+R/W */
39     TWDR = addr_rw;
40     TWSR = TW_INT_FLAG | TW_ENABLE;
41     twi_wait();
42     st = TW_STATUS;
43     return (st == 0x18 /* SLA+W ACK */) || (st == 0x40 /* SLA+R ACK */);
44     ↪ ;
45 }
46
47 static void twi_stop(void)
48 {
49     TWSR = TW_INT_FLAG | TW_STOP | TW_ENABLE;
50 }
51
52 /* PUBLIC TWI ROUTINES */
53 void twi_init(void)
54 {
55     /* prescaler bits (TWPS1:0) = 0 ? prescaler = 1 */
56     TWSR &= ~((1u << TWPS0) | (1u << TWPS1));

```

```

57  /* choose TWBR such that SCL = TWI_SCL_HZ */
58  * TWBR = (F_CPU / SCL - 16) / (2 * TWPS)
59  * At 8MHz with 400kHz I2C: TWBR = (8,000,000/400,000 - 16) / 2 = 4
    ↪ */
60  TWBR = (uint8_t)((F_CPU / TWI_SCL_HZ - 16UL) / 2UL);
61
62  /* For debug to verify correct value: */
63  if (F_CPU == 8000000UL && TWI_SCL_HZ == 400000UL)
64  {
65      /* For 8MHz clock, with 400kHz I2C, TWBR should be 4 */
66      TWBR = 4;
67  }
68
69  TWCR = TW_ENABLE; /* enable module, no interrupt yet */
70 }
71
72 bool twi_write(uint8_t sla, const uint8_t *buf, uint8_t len)
73 {
74     if (!twi_start((sla << 1) | 0))
75         return false;
76     for (uint8_t i = 0; i < len; ++i)
77     {
78         TWDR = buf[i];
79         TWCR = TW_INT_FLAG | TW_ENABLE;
80         twi_wait();
81         if (TW_STATUS != 0x28)
82         {
83             twi_stop();
84             return false;
85         } /* data ACK */
86     }
87     twi_stop();
88     return true;
89 }
90
91 bool twi_read(uint8_t sla, uint8_t *buf, uint8_t len)
92 {
93     if (!twi_start((sla << 1) | 1))
94         return false;
95     for (uint8_t i = 0; i < len; ++i)
96     {
97         /* ACK all bytes except last */
98         TWCR = TW_INT_FLAG | TW_ENABLE | (i == len - 1 ? 0 : TW_ACK);
99         twi_wait();
100         if ((i < len - 1 && TW_STATUS != 0x50) || /* data with ACK */
101             (i == len - 1 && TW_STATUS != 0x58)) /* data with NACK */
102         {
103             twi_stop();
104             return false;
105         }
106         buf[i] = TWDR;
107     }
108     twi_stop();
109     return true;
110 }
111
112 /* BNO055 BASIC ACCESS */
113 bool bno055_write8(uint8_t reg, uint8_t val)

```

```

114 {
115     uint8_t pkt[2] = {reg, val};
116     return twi_write(BNO055_I2C_ADDR, pkt, 2);
117 }
118
119 bool bno055_read8(uint8_t reg, uint8_t *val)
120 {
121     if (!twi_write(BNO055_I2C_ADDR, &reg, 1))
122         return false;
123     return twi_read(BNO055_I2C_ADDR, val, 1);
124 }
125
126 bool bno055_read(uint8_t reg, uint8_t *buf, uint8_t len)
127 {
128     if (!twi_write(BNO055_I2C_ADDR, &reg, 1))
129         return false;
130     return twi_read(BNO055_I2C_ADDR, buf, len);
131 }
132
133 /* HIGH-LEVEL HELPERS */
134 static bool bno055_set_mode(uint8_t mode)
135 {
136     return bno055_write8(0x3D, mode); /* BNO055_OPR_MODE_ADDR */
137 }
138
139 void bno055_gpio_reset(void)
140 {
141     BNO_RST_DDR |= (1<<BNO_RST_PIN); /* drive pin */
142     BNO_RST_PORT &= ~(1<<BNO_RST_PIN); /* pull low */
143     _delay_ms(10);
144     BNO_RST_PORT |= (1<<BNO_RST_PIN); /* release */
145     _delay_ms(650); /* wait for reboot */
146 }
147
148 bool bno055_init(void)
149 {
150     twi_init();
151
152     /* Ensure sensor is present */
153     uint8_t id = 0;
154     if (!bno055_read8(0x00, &id) || id != 0xA0)
155         return false; /* CHIP_ID */
156
157     /* Switch to CONFIG, reset, then NDOF */
158     bno055_set_mode(0x00); /* CONFIG */
159     _delay_ms(25);
160
161     bno055_write8(0x3F, 0x20); /* SYS_TRIGGER, reset */
162     _delay_ms(650); /* ~650 ms boot time */
163
164     bno055_set_mode(0x0C); /* NDOF fusion */
165     _delay_ms(20);
166     return true;
167 }
168
169 void bno055_get_euler(int16_t *h, int16_t *r, int16_t *p)
170 {
171     uint8_t buf[6];

```

```

172     if (bno055_read(0x1A, buf, 6))
173     { /* EULER_H_LSB */
174         *h = (int16_t)(buf[0] | ((uint16_t)buf[1] << 8));
175         *r = (int16_t)(buf[2] | ((uint16_t)buf[3] << 8));
176         *p = (int16_t)(buf[4] | ((uint16_t)buf[5] << 8));
177     }
178 }
179
180 //decide not needed later
181 void bno055_get_omega(int16_t *gx, int16_t *gy, int16_t *gz)
182 {
183     uint8_t buf[6];
184     if (bno055_read(0x14, buf, 6)) /* GYRO_DATA_X_LSB */
185     {
186         *gx = (int16_t)(buf[0] | ((uint16_t)buf[1] << 8));
187         *gy = (int16_t)(buf[2] | ((uint16_t)buf[3] << 8));
188         *gz = (int16_t)(buf[4] | ((uint16_t)buf[5] << 8));
189     }
190 }
191
192 /* 1 LSB = 1/100 m s-2 */
193 void bno055_get_accel(int16_t *ax, int16_t *ay, int16_t *az)
194 {
195     uint8_t buf[6];
196     if (bno055_read(0x28, buf, 6)) /*
197         ↪ LINEAR_ACCEL_DATA_X_LSB */
198     {
199         *ax = (int16_t)(buf[0] | ((uint16_t)buf[1] << 8));
200         *ay = (int16_t)(buf[2] | ((uint16_t)buf[3] << 8));
201         *az = (int16_t)(buf[4] | ((uint16_t)buf[5] << 8));
202     }
203 }
204
205 bool bno055_is_fully_calibrated(void)
206 {
207     uint8_t cal;
208     if (!bno055_read8(0x35, &cal))
209         return false; /* CALIB_STAT */
210     return ((cal >> 6) & 0x03) == 3; /* SYS == 3 */
211 }
212
213 bool bno055_apply_offsets(const uint8_t calib[22])
214 {
215     /* 1) remember current operating mode */
216     uint8_t mode;
217     if (!bno055_read8(0x3D, &mode)) return false; /* OPR_MODE */
218
219     /* 2) switch to CONFIG mode */
220     if (!bno055_write8(0x3D, 0x00)) return false;
221     _delay_ms(25);
222
223     /* 3) write the entire 22-byte block */
224     for (uint8_t i = 0; i < 22; ++i) {
225         if (!bno055_write8(offset_reg_first + i, calib[i]))
226             return false;
227     }
228
229     /* 4) restore previous mode */

```

```

229     if (!bno055_write8(0x3D, mode)) return false;
230     _delay_ms(20);
231
232     return true;
233 }
234
235 bool bno055_read_offsets(uint8_t calib[22])
236 {
237     uint8_t mode;
238     if (!bno055_read8(0x3D, &mode)) return false;
239
240     if (!bno055_write8(0x3D, 0x00)) return false;    /* CONFIG */
241     _delay_ms(25);
242
243     for (uint8_t i = 0; i < 22; ++i) {
244         if (!bno055_read8(offset_reg_first + i, &calib[i]))
245             return false;
246     }
247
248     bno055_write8(0x3D, mode);                        /* back to old mode */
249     _delay_ms(20);
250     return true;
251 }

```

Listing 5: BNO055 implementation (bno055_ll.c)

2.4 Encoder & Odometry

Wheel odometry uses quadrature encoders and interrupts to maintain position and heading estimates:

- **Interrupts:** Multiple external and pin-change interrupts handle encoder channels and emergency button.
- **Count Handling:** Shared ISR routine updates tick counters and timestamps.
- **Kinematics:** Delta ticks - mm and deg using configured geometry.
- **Speed & Timing:** Real-time velocity and loop timing for use in motion profiles.
- **Emergency Stop:** Immediate flag on button press.

```

1  /*
2  *  encoder.h
3  *
4  *  Created: 5/6/2025 4:44:05 AM
5  *  Author: Endeavor360
6  */
7
8
9  #ifndef ENCODERS_H_
10 #define ENCODERS_H_
11
12
13 #include <stdint.h>
14 #include <stdbool.h>
15 #include "sys/time.h"
16

```



```

17  /* ----- API ----- */
18  void      encoder_init(void);
19
20  int32_t    encoder_get_left(void);
21  int32_t    encoder_get_right(void);
22
23  void      encoder_reset_left(void);
24  void      encoder_reset_right(void);
25  void      encoder_reset_both(void);
26
27  bool      encoder_emergency_hit(void);
28
29  /* ----- high-level odometry helpers ----- */
30  void      encoder_odometry_reset(void);
31  void      encoder_odometry_update(void);      /* call at ~1 kHz
    ↪          */
32
33  float      encoder_left_speed_mm_s(void);
34  float      encoder_right_speed_mm_s(void);
35
36  float      encoder_robot_speed_mm_s(void);    /* forward speed
    ↪          */
37  float      encoder_robot_omega_dps(void);    /* yaw rate (ř/s)
    ↪          */
38
39  float      encoder_robot_distance_mm(void);   /* travelled distance
    ↪          */
40  float      encoder_robot_angle_deg(void);    /* accumulated heading
    ↪          */
41
42  uint32_t    encoder_loop_time_us(void);      /* ?t used in last update
    ↪          */
43
44  #endif /* ENCODERS_H_ */

```

Listing 6: Quadrature decoder and odometry API (encoder.h)

```

1  /* =====
2  * encoder.c  Robust quadrature-encoder reader (ATmega32U4)
3  * 4 quadrature decoder with mixed INT/PCINT
4  *
5  * Author : Endeavor360
6  * Date   : 25-May-2025
7  * ===== */
8  #include <avr/io.h>
9  #include <avr/interrupt.h>
10 #include <stdbool.h>
11 #include <util/atomic.h>
12 #include "config.h"
13 #include "encoder.h"
14
15 /* private state */
16 static volatile int32_t left_cnt, right_cnt;
17 static volatile uint8_t left_last_state, right_last_state;
18
19 static int32_t prev_left_cnt = 0;
20 static int32_t prev_right_cnt = 0;
21

```

```

22 static int32_t left_delta = 0;
23 static int32_t right_delta = 0;
24
25 static float fwd_change_mm = 0.0f;
26 static float rot_change_deg = 0.0f;
27
28 static float robot_distance_mm = 0.0f;
29 static float robot_angle_deg = 0.0f;
30
31 static uint64_t prev_ts_us = 0;
32 static uint32_t loop_dt_us = 1;
33
34 /* ----- helpers ----- */
35 static inline float inv_dt(void) { return 1e6f / (float)loop_dt_us; }
36
37 static volatile bool emg_flag = false;
38
39 /* forward */
40 static inline void enc_handle(volatile int32_t *cnt, volatile uint8_t *
    ↪ last, uint8_t a, uint8_t b)
41 {
42     uint8_t encoded = (a << 1) | b;
43     uint8_t trans = (*last << 2) | encoded;
44     switch (trans)
45     {
46     case 0b1101:
47     case 0b0100:
48     case 0b0010:
49     case 0b1011:
50         (*cnt)--;
51         break;
52     case 0b1110:
53     case 0b0111:
54     case 0b0001:
55     case 0b1000:
56         (*cnt)++;
57         break;
58     default: /* bounce/invalid */
59         break;
60     }
61     *last = encoded;
62 }
63
64 /* Initialise both encoders + emergency pin */
65 void encoder_init(void)
66 {
67     /*----- LEFT: PD2/PD3 as INT2/INT3 -----*/
68     ENC_L_A_DDR &= ~_BV(ENC_L_A_BIT);
69     ENC_L_B_DDR &= ~_BV(ENC_L_B_BIT);
70     ENC_L_A_PORT |= _BV(ENC_L_A_BIT);
71     ENC_L_B_PORT |= _BV(ENC_L_B_BIT);
72
73     /* INT2 on PD2, any edge */
74     EICRA |= _BV(ISC20);
75     EICRA &= ~_BV(ISC21);
76     EIMSK |= _BV(INT2);
77
78     /* INT3 on PD3, any edge */

```

```

79  EICRA |= _BV(ISC30);
80  EICRA &= ~_BV(ISC31);
81  EIMSK |= _BV(INT3);
82
83  /* Seed last_state */
84  left_last_state = ((ENC_L_A_PINREG & _BV(ENC_L_A_BIT)) ? 2 : 0) | ((
    ↪ ENC_L_B_PINREG & _BV(ENC_L_B_BIT)) ? 1 : 0);
85
86  /*----- RIGHT: PE6 as INT6, PB4 as PCINT4 and EMG BTN as
    ↪ PCINT7 -----*/
87  /* pins input + pull-up */
88  ENC_R_B_DDR &= ~_BV(ENC_R_B_BIT);
89  ENC_R_B_PORT |= _BV(ENC_R_B_BIT);
90
91  ENC_R_A_DDR &= ~_BV(ENC_R_A_BIT);
92  EMG_BTN_DDR &= ~_BV(EMG_BTN_BIT);
93  ENC_R_A_PORT |= _BV(ENC_R_A_BIT);
94  EMG_BTN_PORT |= _BV(EMG_BTN_BIT);
95
96  /* INT6 on PE6 any edge */
97  EICRB |= _BV(ISC60); // sense on either edge
98  EICRB &= ~_BV(ISC61); // (leave ISC61 = 0)
99  EIMSK |= _BV(INT6); // enable INT6
100
101  /* enable PCINT[7:0] */
102  PCICR |= _BV(PCIE0);
103  /* mask PB4, PB7 */
104  PCMSKO |= _BV(ENC_R_A_BIT) | _BV(EMG_BTN_BIT);
105
106  right_last_state = ((ENC_R_A_PINREG & _BV(ENC_R_A_BIT)) ? 2 : 0) | ((
    ↪ ENC_R_B_PINREG & _BV(ENC_R_B_BIT)) ? 1 : 0);
107
108  /* Zero counters */
109  left_cnt = right_cnt = 0;
110
111  encoder_odometry_reset();
112 }
113
114 /* ----- LEFT ISRs (INT2 & INT3) ----- */
115 ISR(INT2_vect)
116 {
117     uint8_t a = (ENC_L_A_PINREG & _BV(ENC_L_A_BIT)) ? 1 : 0;
118     uint8_t b = (ENC_L_B_PINREG & _BV(ENC_L_B_BIT)) ? 1 : 0;
119     enc_handle(&left_cnt, &left_last_state, a, b);
120 }
121
122 ISR(INT3_vect)
123 {
124     uint8_t a = (ENC_L_A_PINREG & _BV(ENC_L_A_BIT)) ? 1 : 0;
125     uint8_t b = (ENC_L_B_PINREG & _BV(ENC_L_B_BIT)) ? 1 : 0;
126     enc_handle(&left_cnt, &left_last_state, a, b);
127 }
128
129 ISR(INT6_vect)
130 {
131     uint8_t a = (ENC_R_A_PINREG & _BV(ENC_R_A_BIT)) ? 1 : 0;
132     uint8_t b = (ENC_R_B_PINREG & _BV(ENC_R_B_BIT)) ? 1 : 0;
133     enc_handle(&right_cnt, &right_last_state, a, b);

```

```

134 }
135
136 /* ----- RIGHT + EMERGENCY (PCINT0) ----- */
137 ISR(PCINT0_vect)
138 {
139     /* 1) emergency check */
140     if (!(EMG_BTN_PINREG & _BV(EMG_BTN_BIT)))
141     {
142         emg_flag = true;
143     }
144
145     /* 2) right encoder decode */
146     uint8_t a = (ENC_R_A_PINREG & _BV(ENC_R_A_BIT)) ? 1 : 0;
147     uint8_t b = (ENC_R_B_PINREG & _BV(ENC_R_B_BIT)) ? 1 : 0;
148     enc_handle(&right_cnt, &right_last_state, a, b);
149 }
150
151 /* ===== public API (unchanged) ===== */
152 int32_t encoder_get_left(void)
153 {
154     cli();
155     int32_t c = left_cnt;
156     sei();
157     return c;
158 }
159
160 int32_t encoder_get_right(void)
161 {
162     cli();
163     int32_t c = right_cnt;
164     sei();
165     return c;
166 }
167
168 void encoder_reset_left(void)
169 {
170     cli();
171     left_cnt = 0;
172     sei();
173 }
174
175 void encoder_reset_right(void)
176 {
177     cli();
178     right_cnt = 0;
179     sei();
180 }
181
182 void encoder_reset_both(void)
183 {
184     encoder_reset_left();
185     encoder_reset_right();
186 }
187
188 bool encoder_emergency_hit(void)
189 {
190     bool hit;
191     cli(); /* atomic: read-then-clear */

```

```

192 hit = emg_flag;
193 // emg_flag = false; // needs to stop all operations in a way that
    ↪ restart can fix it
194 sei();
195 return hit;
196 }
197
198 void encoder_odometry_reset(void)
199 {
200     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
201     {
202         prev_left_cnt = left_cnt = 0;
203         prev_right_cnt = right_cnt = 0;
204
205         fwd_change_mm = rot_change_deg = 0.0f;
206         robot_distance_mm = robot_angle_deg = 0.0f;
207
208         prev_ts_us = micros64(); /* â€œ use global timer0 based time-base
    ↪ */
209         loop_dt_us = 1;
210     }
211 }
212
213 void encoder_odometry_update(void)
214 {
215     uint64_t now_us = micros64();
216     loop_dt_us = (uint32_t)(now_us - prev_ts_us);
217     if (loop_dt_us == 0)
218         loop_dt_us = 1;
219     prev_ts_us = now_us;
220
221     /* snapshot counts atomically */
222     int32_t l, r;
223     ATOMIC_BLOCK(ATOMIC_RESTORESTATE)
224     {
225         l = left_cnt;
226         r = right_cnt;
227     }
228     left_delta = l - prev_left_cnt;
229     right_delta = r - prev_right_cnt;
230     prev_left_cnt = l;
231     prev_right_cnt = r;
232
233     const float mm_per_pulse = MM_PER_ROTATION / (4.0f * ENCODER_PPR *
    ↪ GEAR_RATIO);
234     float left_mm = left_delta * mm_per_pulse;
235     float right_mm = right_delta * mm_per_pulse;
236
237     fwd_change_mm = 0.5f * (left_mm + right_mm);
238     rot_change_deg = (right_mm - left_mm) * DEG_PER_MM_DIFF;
239
240     robot_distance_mm += fwd_change_mm;
241     robot_angle_deg += rot_change_deg;
242 }
243
244 float encoder_left_speed_mm_s(void) { return left_delta * (
    ↪ MM_PER_ROTATION / (4.0f * ENCODER_PPR * GEAR_RATIO)) * inv_dt();
    ↪ }

```

```

245 float encoder_right_speed_mm_s(void) { return right_delta * (
    ↪ MM_PER_ROTATION / (4.0f * ENCODER_PPR * GEAR_RATIO)) * inv_dt();
    ↪ }
246 float encoder_robot_speed_mm_s(void) { return fwd_change_mm * inv_dt();
    ↪ }
247 float encoder_robot_omega_dps(void) { return rot_change_deg * inv_dt();
    ↪ }
248 float encoder_robot_distance_mm(void) { return robot_distance_mm; }
249 float encoder_robot_angle_deg(void) { return robot_angle_deg; }
250 uint32_t encoder_loop_time_us(void) { return loop_dt_us; }

```

Listing 7: Interrupt-driven encoder implementation (encoder.c)

2.5 Serial-Stream Sensor Parser

UART packets deliver sensor data to the Jetson Nano. Though the parser resides on the host, the MCU structures the stream as follows:

- Encoder, IMU, battery, and cliff sensor fields.
- CRC checksum and packet framing.
- Error counters for stale or corrupted packets.

2.6 Motor Control

Stepper motors are driven with precise timers and GPIO signals:

- **Timer Configuration:** Two 16-bit timers in CTC mode generate step pulses.
- **Control API:** Enable/disable, direction, and speed setting functions.
- **Chassis Kinematics:** Splitting linear/angular commands into wheel frequencies.
- **Emergency Stop:** Disables all drivers instantly.

```

1  /*
2   * motors.h
3   *
4   * Created: 5/6/2025 4:25:54 AM
5   * Author: Endeavor360
6   */
7
8  #ifndef MOTORS_H
9  #define MOTORS_H
10
11 #include <stdint.h>
12 #include <stdbool.h>
13
14 void motors_init(void);
15 void motors_update(float velocity, float omega);
16 void motors_enable_left(bool en);
17 void motors_enable_right(bool en);
18 void motors_enable_all(bool en);
19 void motors_set_dir_left(bool fwd);
20 void motors_set_dir_right(bool fwd);
21 void motors_set_speed_left(uint16_t rpm);
22 void motors_set_speed_right(uint16_t rpm);

```

```

23 void motors_set_speed_both(uint16_t rpm_left, uint16_t rpm_right);
24 void motors_stop_all();
25
26 #endif // MOTORS_H

```

Listing 8: Motor control interface (motors.h)

```

1  /*
2   * motors.c
3   *
4   * Created: 5/6/2025 4:26:05 AM
5   * Author: Endeavor360
6   */
7
8  #include <avr/io.h>
9  #include <avr/interrupt.h>
10 #include <stdbool.h>
11 #include <stdint.h>
12 #include "motors.h"
13 #include "config.h"
14
15 /* private state
16    ↳ ----- */
17 static uint16_t left_top;
18 static uint16_t right_top;
19
20 /* helpers
21    ↳ -----
22    ↳ */
23 static inline uint32_t velocity_to_freq(uint16_t velocity)
24 {
25     return ((uint32_t)velocity * STEPS_PER_REV * GEAR_RATIO) / (
26         ↳ WHEEL_DIAMETER_MM * M_PI);
27 }
28
29 /* public functions
30    ↳ ----- */
31 void motors_init(void)
32 {
33     /* I/O direction
34        ↳ ----- */
35     LEFT_PUL_DDR |= _BV(LEFT_PUL_BIT);
36     LEFT_DIR_DDR |= _BV(LEFT_DIR_BIT);
37     LEFT_ENA_DDR |= _BV(LEFT_ENA_BIT);
38
39     RIGHT_PUL_DDR |= _BV(RIGHT_PUL_BIT);
40     RIGHT_DIR_DDR |= _BV(RIGHT_DIR_BIT);
41     RIGHT_ENA_DDR |= _BV(RIGHT_ENA_BIT);
42
43     /* default levels if; pulses high, forward, drivers disabled
44        ↳ ----- */
45     LEFT_PUL_PORT |= _BV(LEFT_PUL_BIT);
46     LEFT_DIR_PORT |= _BV(LEFT_DIR_BIT);
47     LEFT_ENA_PORT &= ~_BV(LEFT_ENA_BIT);
48
49     RIGHT_PUL_PORT |= _BV(RIGHT_PUL_BIT);
50     RIGHT_DIR_PORT |= _BV(RIGHT_DIR_BIT);
51     RIGHT_ENA_PORT &= ~_BV(RIGHT_ENA_BIT);

```

```

45
46  /* âĀĤ Timer-3 (16-bit) drives LEFT motor PUL on OC3A (PC6/D5) âĀĤ */
47  TCCR3A = _BV(COM3A0); /* toggle OC3A on compare match */
48  TCCR3B = _BV(WGM32); /* CTC mode (TOP = OCR3A), clk stopped */
49
50  /* âĀĤ Timer-1 (16-bit) drives RIGHT motor PUL on OC1A (PB5/D9) âĀĤ
51  → */
52  TCCR1A = _BV(COM1A0); /* toggle OC1A on compare match */
53  TCCR1B = _BV(WGM12); /* CTC mode, clk stopped */
54
55  motors_enable_all(true);
56
57  void motors_enable_left(bool en)
58  {
59      (en ? (LEFT_ENA_PORT |= _BV(LEFT_ENA_BIT))
60          : (LEFT_ENA_PORT &= ~_BV(LEFT_ENA_BIT)));
61  }
62
63  void motors_enable_right(bool en)
64  {
65      (en ? (RIGHT_ENA_PORT |= _BV(RIGHT_ENA_BIT))
66          : (RIGHT_ENA_PORT &= ~_BV(RIGHT_ENA_BIT)));
67  }
68
69  void motors_enable_all(bool en)
70  {
71      motors_enable_left(en);
72      motors_enable_right(en);
73  }
74
75  void motors_set_dir_left(bool fwd)
76  {
77      (fwd ? (LEFT_DIR_PORT |= _BV(LEFT_DIR_BIT))
78          : (LEFT_DIR_PORT &= ~_BV(LEFT_DIR_BIT)));
79  }
80
81  void motors_set_dir_right(bool fwd)
82  {
83      (fwd ? (RIGHT_DIR_PORT |= _BV(RIGHT_DIR_BIT))
84          : (RIGHT_DIR_PORT &= ~_BV(RIGHT_DIR_BIT)));
85  }
86
87  void motors_set_speed_left(uint16_t vel)
88  {
89      uint32_t freq = velocity_to_freq(vel);
90      uint32_t top = (F_CPU / (2UL * freq * CLOCK_DIVISOR_TIMER3)) - 1UL;
91      if (top > 0xFFFF)
92          top = 0xFFFF; /* clamp to 16-bit */
93
94      left_top = (uint16_t)top;
95      OCR3A = left_top;
96
97      /* start Timer-3 with /1024 prescale */
98      TCCR3B &= ~(_BV(CS32) | _BV(CS31) | _BV(CS30)); /* clear first */
99      TCNT3 = 0;
100     TCCR3B |= PRE_SCALE_TIMER3;
101 }

```



```

102
103 void motors_set_speed_right(uint16_t vel)
104 {
105     uint32_t freq = velocity_to_freq(vel);
106     uint32_t top = (F_CPU / (2UL * freq * CLOCK_DIVISOR_TIMER1)) - 1UL;
107
108     right_top = (uint16_t)top;
109     OCR1A = right_top;
110
111     /* start Timer-1 with /1024 pre-scale */
112     TCCR1B &= ~(_BV(CS12) | _BV(CS11) | _BV(CS10)); /* clear first */
113     TCNT1 = 0;
114     TCCR1B |= PRE_SCALE_TIMER1;
115 }
116
117 void motors_set_speed_both(uint16_t vel_left, uint16_t vel_right)
118 {
119     motors_set_speed_left(vel_left);
120     motors_set_speed_right(vel_right);
121 }
122
123 void motors_update( float velocity, float omega)
124 {
125     /* Feed-forward terms based on desired wheel tangential speed */
126     float tangent_speed = omega * WHEEL_BASE_MM * M_PI/ 360.0 ;
127     float left_speed    = velocity - tangent_speed;
128     float right_speed   = velocity + tangent_speed;
129
130     // Get absolute speeds for motor control
131     float left_abs = fabsf(left_speed);
132     float right_abs = fabsf(right_speed);
133
134     // Set directions based on speed signs
135     motors_set_dir_left(left_speed >= 0);
136     motors_set_dir_right(right_speed < 0); //invert due opposite
137     ↪ orientation
138
139     motors_set_speed_both((uint16_t)left_abs, (uint16_t)right_abs);
140 }
141
142 void motors_stop_all()
143 {
144     motors_enable_all(false);
145
146     TCCR1B &= ~(_BV(CS12) | _BV(CS11) | _BV(CS10)); /* stop Timer-1 */
147     TCCR3B &= ~(_BV(CS32) | _BV(CS31) | _BV(CS30)); /* stop Timer-3 */
148 }

```

Listing 9: Stepper driver implementation (motors.c)

2.7 Motion Profiling

Trapezoidal profiles ensure smooth starts/stops for moves and turns:

- Profile structure and state machine.
- APIs to start, update, and query profiles.

```

1  /*
2  * profiler.h
3  *
4  * Created: 5/21/2025 9:20:39 PM
5  * Author: Endeavor360
6  */
7
8  #ifndef PROFILER_H_
9  #define PROFILER_H_
10
11 #include <stdbool.h>
12 #include <stdint.h>
13
14 /* ===== Profile types ===== */
15
16 typedef enum {
17     PS_IDLE = 0,
18     PS_ACCELERATING,
19     PS BRAKING,
20     PS_FINISHED,
21 } ProfileState;
22
23 typedef enum {
24     PK_FORWARD = 0,
25     PK_ROTATION = 1,
26 } ProfileKind;
27
28 typedef struct {
29     volatile ProfileState state;
30     volatile float speed; /* mm/s or deg/s */
31     volatile float position; /* mm or deg */
32
33     int8_t sign; /* direction */
34     ProfileKind kind; /* forward / rotation */
35
36     float acceleration;
37     float one_over_acc;
38
39     float target_speed;
40     float final_speed;
41     float final_position;
42 } Profile;
43
44 /* Forward declaration of Motion aggregate */
45 typedef struct {
46     Profile forward;
47     Profile rotation;
48 } MotionType;
49
50 /* ===== Profile API ===== */
51 void profile_reset(Profile *p);
52 void profile_start(Profile *p, float distance, float top_s, float
    ↪ final_s, float acc);
53 void profile_stop(Profile *p);
54 void profile_update(Profile *p);
55
56 void profile_soft_reset(Profile *p); //only resets distance

```

```

57
58 /* ===== Motion facade ===== */
59 void motion_reset_drive_system(void);
60 void motion_stop(void);
61 float motion_position(void);
62 float motion_velocity(void);
63 float motion_acceleration(void);
64 void motion_set_target_velocity(float v);
65 float motion_angle(void);
66 float motion_omega(void);
67 float motion_alpha(void);
68 void motion_start_move(float dist, float top_v, float final_v, float
    ↪ acc);
69 bool motion_move_finished(void);
70 void motion_start_turn(float dist, float top_w, float final_w, float
    ↪ acc);
71 bool motion_turn_finished(void);
72 void motion_update(void);
73 void motion_wait_until_position(float pos_mm);
74 void motion_wait_until_distance(float dist_mm);
75
76 void motion_SOFT_reset_drive_system(void);
77
78 #endif // PROFILER_H_

```

Listing 10: Motion profiling definitions (profiler.h)

```

1 /*
    ↪ -----
    ↪
2  * profiler.c Motion and trapezoidal profile manager
3  *
4  *
5  * Author    : Endeavor360
6  * Created   : 3rd July 2025
7  *
    ↪ -----
    ↪ */
8
9 #include "config.h"
10 #include <avr/io.h>
11 #include <util/delay.h>
12 #include <stdbool.h>
13 #include <math.h>
14 #include "motors.h"
15 #include "encoder.h"
16 #include "profiler.h"
17
18 /* ===== helpers ===== */
19 static inline float enc_loop_time_s(void)
20 {
21     return (float)encoder_loop_time_us() * 1.0e-6f;
22 }
23
24 static inline float feedback_speed(ProfileKind k)
25 {
26     return (k == PK_FORWARD) ? encoder_robot_speed_mm_s() :
        ↪ encoder_robot_omega_dps();

```

```

27 }
28
29 static float profile_braking_distance(const Profile *p)
30 {
31     return fabsf(p->speed * p->speed - p->final_speed * p->final_speed) *
        ↪ 0.5f * p->one_over_acc;
32 }
33
34 /* ===== Profile API ===== */
35 void profile_reset(Profile *p)
36 {
37     cli();
38     p->position = 0.0f;
39     p->speed = 0.0f;
40     p->target_speed = 0.0f;
41     p->state = PS_IDLE;
42     sei();
43 }
44
45 void profile_start(Profile *p, float distance, float top_speed, float
        ↪ final_speed, float acceleration)
46 {
47     p->sign = (distance < 0.0f) ? -1 : +1;
48     if (distance < 0.0f)
49         distance = -distance;
50
51     if (distance < 1.0f)
52     {
53         p->state = PS_FINISHED;
54         return;
55     }
56
57     if (final_speed > top_speed)
58         final_speed = top_speed;
59
60     p->position = 0.0f;
61     p->final_position = distance;
62
63     p->target_speed = p->sign * fabsf(top_speed);
64     p->final_speed = p->sign * fabsf(final_speed);
65
66     p->acceleration = fabsf(acceleration);
67     p->one_over_acc = (p->acceleration >= 1.0f) ? (1.0f / p->acceleration
        ↪ ) : 1.0f;
68
69     p->state = PS_ACCELERATING;
70 }
71
72 void profile_stop(Profile *p)
73 {
74     cli();
75     p->target_speed = 0.0f;
76     p->speed = 0.0f;
77     p->state = PS_FINISHED;
78     sei();
79 }
80
81 void profile_update(Profile *p)

```

```

82 {
83     if (p->state == PS_IDLE)
84         return;
85
86     float dt = enc_loop_time_s();
87     float delta_v = p->acceleration * dt;
88     float remaining = fabsf(p->final_position) - fabsf(p->position);
89
90     if (p->state == PS_ACCELERATING)
91     {
92         if (remaining < profile_braking_distance(p))
93         {
94             p->state = PS BRAKING;
95             p->target_speed = (p->final_speed == 0.0f)? 0.0f: p->final_speed;
96         }
97     }
98
99     /* reach target speed */
100     if (p->speed < p->target_speed)
101     {
102         p->speed += delta_v;
103         if (p->speed > p->target_speed)
104             p->speed = p->target_speed;
105     }
106     else if (p->speed > p->target_speed)
107     {
108         p->speed -= delta_v;
109         if (p->speed < p->target_speed)
110             p->speed = p->target_speed;
111     }
112
113     /* integrate position with feedback */
114     float fb_speed = feedback_speed(p->kind);
115     p->position += fb_speed * dt;
116
117     if (p->state != PS_FINISHED && remaining < 0.125f)
118     {
119         p->state = PS_FINISHED;
120         p->target_speed = p->final_speed;
121     }
122 }
123
124
125
126 /* ===== Motion aggregate ===== */
127 MotionType motionType; /* single instance */
128
129
130 void motion_reset_drive_system(void)
131 {
132     motors_stop_all();
133
134     encoder_odometry_reset();
135     profile_reset(&motionType.forward);
136     profile_reset(&motionType.rotation);
137
138     motors_enable_all(true);
139 }

```

```

140
141 void profile_soft_reset(Profile *p)
142 {
143     cli();
144     p->position = 0.0f;
145     sei();
146 }
147
148 void motion_SOFT_reset_drive_system(void)
149 {
150     encoder_odometry_reset();
151     profile_soft_reset(&motionType.forward);
152     profile_soft_reset(&motionType.rotation);
153 }
154
155 void motion_stop(void) { motors_stop_all(); }
156
157 float motion_position(void) { return motionType.forward.position; }
158 float motion_velocity(void) { return motionType.forward.speed; }
159 float motion_acceleration(void) { return motionType.forward.
    ↪ acceleration; }
160
161 void motion_set_target_velocity(float v) { motionType.forward.
    ↪ target_speed = v; }
162
163 float motion_angle(void) { return motionType.rotation.position; }
164 float motion_omega(void) { return motionType.rotation.speed; }
165 float motion_alpha(void) { return motionType.rotation.acceleration; }
166
167 bool motion_move_finished(void) { return motionType.forward.state
    ↪ == PS_FINISHED; }
168 bool motion_turn_finished(void) { return motionType.rotation.state ==
    ↪ PS_FINISHED; }
169
170 void motion_start_move(float distance, float top_v, float final_v,
    ↪ float acc)
171 {
172     motionType.forward.kind = PK_FORWARD; // Add this line
173     profile_start(&motionType.forward, distance, top_v, final_v, acc);
174 }
175
176 void motion_start_turn(float distance, float top_w, float final_w,
    ↪ float acc)
177 {
178     motionType.rotation.kind = PK_ROTATION; // Add this line
179     profile_start(&motionType.rotation, distance, top_w, final_w, acc);
180 }
181
182 void motion_update(void)
183 {
184     profile_update(&motionType.forward);
185     profile_update(&motionType.rotation);
186 }
187
188
189 void motion_wait_until_position(float position_mm)
190 {
191     while (motion_position() < position_mm)

```

```

192     {
193         _delay_ms(LOOP_TIME);
194     }
195 }
196
197 void motion_wait_until_distance(float distance_mm)
198 {
199     motion_wait_until_position(motion_position() + distance_mm);
200 }

```

Listing 11: Profile update logic (profiler.c)

2.8 System Timing

A high-resolution 64-bit timer supports deterministic scheduling:

- Timer0 free-run with overflow ISR.
- Atomic microsecond timestamp API.

```

1  /*
2  * systime.h
3  *
4  * Created: 7/2/2025 12:05:12 PM
5  * Author: Endeavor360
6  */
7
8
9  #ifndef SYSTIME_H_
10 #define SYSTIME_H_
11
12 #include <stdint.h>
13
14 /* Call once at start-up */
15 void systime_init(void);
16
17 /* 1-µs resolution, monotonic.
18  * Rolls over after ~292 000 years at 16 MHz (2?? half-µs)
19  * → */
20 uint64_t micros64(void);
21 #endif /* SYSTIME_H_ */

```

Listing 12: Microsecond timer API (systime.h)

```

1  /*
2  * systime.c  Û  64-bit, 1-µs time base using Timer/Counter 0
3  * ATmega32U4, F_CPU = 16 000 000 Hz
4  *
5  * Created: 7/2/2025 12:04:55 PM
6  * Author: Endeavor360
7  *
8  *
9  * Timer-0 setup
10 *   Û normal (free-running) mode
11 *   Û prescaler æ8 ? clk/8 = 2 MHz ? 0.5 µs per tick
12 *   Û overflow every 256 ticks ? 128 µs ? ISR ? 7 812 Hz
13 */

```

```

14
15 #include "systime.h"
16 #include <avr/io.h>
17 #include <avr/interrupt.h>
18
19 /* 0.5  $\mu$ s @ 2?? ? 9.2E10 $\mu$ s ? 2.9E10? years */
20 static volatile uint64_t _ovf64 = 0ULL;
21
22 /* ----- initialisation ----- */
23 void systime_init(void)
24 {
25     cli(); /* global IRQs off
26     ↪ */
27     TCCR0A = 0; /* normal counting mode
28     ↪ */
29     TCCR0B = _BV(CS01); /* CS01:0 = 010 ? prescaler æ8
30     ↪ */
31     TIMSK0 = _BV(TOIE0); /* enable overflow interrupt
32     ↪ */
33     sei(); /* back on
34     ↪ */
35 }
36
37 /* ----- overflow ISR (every 128  $\mu$ s) ----- */
38 ISR(TIMERO_OVF_vect)
39 {
40     _ovf64++; /* software high-word
41     ↪ */
42 }
43
44 /* ----- atomic 64-bit read helper ----- */
45 static inline uint64_t atomic64(volatile uint64_t *p)
46 {
47     uint64_t v;
48     uint8_t s = SREG; cli();
49     v = *p;
50     SREG = s;
51     return v;
52 }
53
54 /* ----- 1- $\mu$ s timestamp ----- */
55 uint64_t micros64(void)
56 {
57     uint64_t ovf;
58     uint8_t tcnt;
59
60     uint8_t s = SREG; cli(); /* critical section
61     ↪ */
62     ovf = _ovf64;
63     tcnt = TCNT0;
64
65     /* if overflow happened after reading TCNT0 but before cli() */
66     if ((TIFR0 & _BV(TOV0)) && tcnt < 255)
67         ovf++;
68
69     SREG = s; /* restore
70     ↪ */
71 }
72

```



```

64      /* total half-ts ticks = (ovf << 8) / tcnt
65      divide by 2 ? shift right once to get whole ts                                */
66      uint64_t half_us = (ovf << 8) | tcnt;
67      return half_us >> 1;
68  }

```

Listing 13: Timer overflow handling (systime.c)

2.9 Main Control Loop

The core loop in `main.c` initializes subsystems and executes a 10 ms cycle for control tasks that ties together sensing, estimation, and actuation. On startup, peripherals are initialized in sequence: USB, system timer, motors, encoders, ADC, I²C, and IMU (with reset/retry logic). A hardware timer (Timer4) fires an interrupt every 10 ms, setting a flag that drives the control loop:

1. Emergency Check: Immediately halt motion if the panic button was pressed
2. Read CSV-formatted instructions from USB (mode flags, movement targets, teleop commands).
3. Sensor & Odometry Update: Refresh encoder-based odometry and trapezoidal profiles; sample IMU, battery, and cliff sensors.
4. Motion Control: In Autonomous mode launches or monitors planned moves/turns; send velocity setpoints to motors. In Teleop Mode maps joystick inputs to speed profiles for manual driving.
5. Telemetry & Debug Streaming: Transmit pose, sensor readings, and diagnostic counters back to the Jetson for visualization and logging.
6. Loop Exit & Shutdown: On fault or external stop, disable all motors, join any background routines, and flush communication buffers.

```

1  /*
2   * avr_controller.c
3   * Modified: 21-May-2025 (telemetry via Timer-3 @ 50 Hz)
4   * Author   : Endeavor360
5   */
6
7  #include "config.h"
8  #include <avr/io.h>
9  #include <avr/interrupt.h>
10 #include <stdio.h>
11 #include <util/delay.h>
12 #include <stdbool.h>
13 #include <math.h>
14
15 #include "motors.h"
16 #include "m_usb.h"
17 #include "bno055_ll.h"
18 #include "analog.h"
19 #include "encoder.h"
20 #include "profiler.h"
21 #include "systime.h"
22
23 #define RX_BUF_SIZE 64

```

```

24
25 // Receive buffer
26 static char rx_buf[RX_BUF_SIZE];
27 static uint8_t rx_index = 0;
28
29 // Parsed parameters
30 static float rx_distance; // mm
31 static float rx_angle; // degrees
32 static uint16_t rx_max_vel; // mm/s
33 static uint16_t rx_max_omega; // deg/s
34 static uint16_t rx_last_vel;
35 static uint16_t rx_last_omega;
36 static float rx_lin_acc; // mm/s
37 static float rx_ang_acc; // deg/s
38
39 uint8_t id;
40
41 typedef enum
42 {
43     AUTONOMOUS = 0,
44     TELEOPERATOR = 1,
45 } controlMode;
46 static uint8_t control_mode = AUTONOMOUS;
47
48 typedef enum
49 {
50     DEBUG_OFF = 0,
51     MOTION_DEBUG = 1,
52     RX_ECHO = 2,
53     MD_AND_ECHO = 3,
54 } debugMode;
55 static uint8_t debug_mode = MOTION_DEBUG;
56
57 typedef enum
58 {
59     CONSTANT = 0,
60     DECELERATING = 1,
61     ACCELERATING = 2,
62     NONETELEOP = 3,
63 } teleOpStates;
64 static uint8_t teleStates = NONETELEOP;
65
66 static uint8_t last_command = 0;
67 static uint8_t current_command = 0;
68
69 static bool f, b, l, r;
70
71 /* ----- PROTOTYPES
72    ↳ -----*/
73 static void timer4_init(void); /* sets up
74    ↳ periodic IRQ */
75 static void send_telemetry(bool emerg, bool profileDone); /* heavy USB
76    ↳ / sensor work */
77 static void usb_send_ram(const char *s);
78 static uint8_t parse_jetson(const char *line);
79 static void receive_from_jetson(void);
80 static void send_debug(void);

```

```

79 static void send_cmd_echo(void);
80
81 /* ----- GLOBAL FLAG -----
   ↪ */
82 static volatile uint8_t loop_execute = 0; /* set in ISR, cleared in
   ↪ main */
83
84 bool emerg = false;
85 bool profile_done = true;
86
87 typedef enum
88 {
89     NONE = 0,
90     FORWARD = 1,
91     ROTATION = 2,
92     BOTH = 3,
93 } MoveType;
94
95 MoveType determineFinishnes = NONE;
96
97 /* ===== */
98 int main(void)
99 {
100     /* ---- initialize everything ---- */
101     _delay_ms(100);
102
103     m_usb_init();
104     systime_init();
105     motors_init();
106     encoder_init();
107     analog_init();
108     twi_init();
109
110     _delay_ms(100);
111     bno055_gpio_reset();
112     _delay_ms(100);
113
114     if (!bno055_init())
115     {
116         m_usb_tx_string("IMU Failed\r\n");
117     }
118
119     /* ---- start 125 Hz timer & enable global IRQs ---- */
120     timer4_init(); /* Timer-3 compare-match every 8 ms */
121     sei();         /* global interrupt enable */
122
123     /* ----- MAIN LOOP ----- */
124     motion_reset_drive_system(); // initialize the encoder counts,
   ↪ velocities, omegas, distances, angles to 0
125     while (1)
126     {
127         /* ----- Emergency Button press status ----- */
128         emerg = encoder_emergency_hit();
129
130         if (loop_execute)
131         {
132             loop_execute = 0;
133

```

```

134 receive_from_jetson();
135 encoder_odometry_update();
136 motion_update();
137
138 // bool imu_ok = bno055_read8(0x00, &id) && (id == 0xA0);
139 // if (imu_ok) bno055_gpio_reset();
140
141 if (!emerg)
142 {
143     if (control_mode == AUTONOMOUS)
144     {
145         if (determineFinishnes == BOTH &&
146             ↪ motion_turn_finished() &&
147             ↪ motion_move_finished())
148         {
149             profile_done = true;
150             motion_reset_drive_system();
151             determineFinishnes = NONE;
152         }
153         else if (determineFinishnes == FORWARD &&
154             ↪ motion_move_finished())
155         {
156             profile_done = true;
157             motion_reset_drive_system();
158             determineFinishnes = NONE;
159         }
160         else if (determineFinishnes == ROTATION &&
161             ↪ motion_turn_finished())
162         {
163             profile_done = true;
164             motion_reset_drive_system();
165             determineFinishnes = NONE;
166         }
167         else
168         {
169             motors_update(motion_velocity(), motion_omega()
170                 ↪ );
171         }
172     }
173     else if (control_mode == TELEOPERATOR)
174     {
175         if ((determineFinishnes == FORWARD &&
176             ↪ motion_move_finished()) || (
177             ↪ determineFinishnes == ROTATION &&
178             ↪ motion_turn_finished()))
179         {
180             profile_done = true;
181
182             if (teleStates == ACCELERATING)
183             {
184                 teleStates = CONSTANT;
185             }
186             else if (teleStates == DECELERATING)
187             {
188                 teleStates = NONETELEOP;
189             }
190             motion_reset_drive_system();
191         }
192     }
193 }

```

```

184
185         else
186         {
187             motors_update(motion_velocity(), motion_omega());
188         }
189     }
190 }
191 else
192 {
193     motors_stop_all();
194 }
195
196 if (debug_mode == MOTION_DEBUG || debug_mode == MD_AND_ECHO
    ↪ )
197 {
198     send_debug();
199 }
200
201 send_telemetry(emerg, profile_done);
202 }
203 }
204 }
205
206 /* ----- TIMER-4 INITIALISATION (CTC, 50 Hz)
    ↪ -----
207 16 MHz / 1024 = 15625 Hz
208 15625 Hz * 0.01 s = 156 * 100Hz = 15625 Hz */
209 static void timer4_init(void)
210 {
211     /* reset all Timer-4 control registers (mandatory for this timer)
        ↪ */
212     TCCR4A = 0;
213     TCCR4B = 0;
214     TCCR4C = 0;
215     TCCR4D = 0; // set WGM40 and WGM41 to normal mode
216
217     TCNT4 = 0; /* start from zero */
218     OCR4A = 156; /* compare after 156 counts (~10 ms) */
219
220     TIMSK4 |= _BV(OCIE4A); /* enable Compare-A interrupt */
221
222     /* start clock: prescaler = 1024 * 15625 = 1562500 Hz
        ↪ */
223     TCCR4B |= _BV(CS43) | _BV(CS41) | _BV(CS40);
224 }
225
226 /* ----- TIMER-4 COMPARE ISR
    ↪ ----- */
227 ISR(TIMER4_COMPA_vect)
228 {
229     TCNT4 = 0; /* emulate CTC */
230     loop_execute = 1; /* signal main loop */
231 }
232
233 /* ----- TELEMETRY SENDER (called from main)
    ↪ ----- */
234 static void send_telemetry(bool emerg, bool profileDone)
235 {

```

```

236 char line[180];
237
238 /* --- orientation --- */
239 int16_t h16, r16, p16;
240 bno055_get_euler(&h16, &r16, &p16);
241
242 /* --- angular rate --- */
243 int16_t gx16, gy16, gz16;
244 bno055_get_omega(&gx16, &gy16, &gz16);
245
246 /* --- linear acceleration --- */
247 int16_t ax16, ay16, az16;
248 bno055_get_accel(&ax16, &ay16, &az16);
249
250 float h = h16 / 16.0f;
251 float r = r16 / 16.0f;
252 float p = p16 / 16.0f;
253
254 const float wx = gx16 / 16.0f; /*  $\dot{A}\tilde{r}/s$  */
255 const float wy = gy16 / 16.0f;
256 const float wz = gz16 / 16.0f;
257
258 const float ax = ax16 / 100.0f; /*  $m/s\tilde{A}s$  */
259 const float ay = ay16 / 100.0f;
260 const float az = az16 / 100.0f;
261
262 // uint8_t cal = bno055_is_fully_calibrated() ? 1u : 0u;
263
264 /* ----- ADC ----- */
265 uint16_t vbat_1 = analog_get_battery_1_mV();
266 uint16_t vbat_2 = analog_get_battery_2_mV();
267 uint16_t cliffL = analog_get_cliff_left();
268 uint16_t cliffC = analog_get_cliff_front();
269 uint16_t cliffR = analog_get_cliff_right();
270
271 /* ----- Encoders ----- */
272 int32_t encL = encoder_get_left();
273 int32_t encR = encoder_get_right();
274
275 /* ----- Format & ship ----- */
276 /* Packet Structure: { Yaw Roll Pitch IMUOmega accrX accrY
    ↳ encoderLeft encoderRight bat1Voltage bat2Voltage LeftCliff
    ↳ CenterCliff RightCliff emergencyFlag } */
277 snprintf(line, sizeof(line),
278          "%3.2f %3.2f %3.2f %+3.2f %+3.2f %+3.2f %+3.2f %+3.2f
    ↳ %+3.2f %10ld %10ld %u %u %u %u %u %u %u\r\n", h, r,
    ↳ p, wx, wy, wz, ax, ay, az, (long)encL, (long)encR,
    ↳ vbat_1, vbat_2, cliffL, cliffC, cliffR, emerg,
    ↳ profileDone);
279
280 usb_send_ram(line);
281 m_usb_tx_push();
282 }
283
284 // SpdL: %+6.1f SpdR: %+6.1f Vel: %+6.1f Omg: %+5.1f dist: %+8.1f ang:
    ↳ %+7.1f dt: %7.1f
285 static void send_debug(void)
286 {

```

```

287     char line[128];
288
289     /* grab data */
290     const float sl = encoder_left_speed_mm_s();
291     const float sr = encoder_right_speed_mm_s();
292     const float v = encoder_robot_speed_mm_s();
293     const float w = encoder_robot_omega_dps();
294     const float d = encoder_robot_distance_mm();
295     const float a = encoder_robot_angle_deg();
296     const float dt = encoder_loop_time_us();
297
298     snprintf(line, sizeof(line),
299              "%+6.1f %+6.1f %+6.1f %+5.1f %+8.1f %+7.1f %7.1f  ",
300              sl, sr, v, w, d, a, dt);
301
302     usb_send_ram(line);
303     m_usb_tx_push();
304 }
305
306 // just for debugging
307 static void send_cmd_echo(void)
308 {
309     /* use the existing globals filled by parse_jetson_auto() */
310     extern float rx_distance, rx_angle, rx_lin_acc, rx_ang_acc;
311     extern uint16_t rx_max_vel, rx_max_omega,
312             rx_last_vel, rx_last_omega;
313
314     char buf[150];
315     /* Format:  âĀĀCMD d=500.0 a=90.0 vmax=300 wmax=120 vend=0 wend=0
316         ↪ acc=500.0 aacc=360.0âĀĀĪ */
317     snprintf(buf, sizeof(buf),
318              "CMD d=%g a=%g vmax=%u wmax=%u vend=%u wend=%u acc=%g aacc
319              ↪ =%g\r\n",
320              rx_distance, rx_angle,
321              rx_max_vel, rx_max_omega,
322              rx_last_vel, rx_last_omega,
323              rx_lin_acc, rx_ang_acc);
324
325     usb_send_ram(buf);
326     m_usb_tx_push();
327 }
328
329 /* ----- Tiny helper ----- */
330 static void usb_send_ram(const char *s)
331 {
332     while (*s)
333         m_usb_tx_char(*s++);
334 }
335
336 static uint8_t parse_jetson(const char *line)
337 {
338     extern float rx_distance, rx_angle, rx_lin_acc, rx_ang_acc;
339     extern uint16_t rx_max_vel, rx_max_omega, rx_last_vel,
340             ↪ rx_last_omega;
341     extern bool f, b, l, r;
342     extern uint8_t control_mode, debug_mode;

```

```

342     uint16_t temp1, temp2, temp3, temp4;
343
344     // Note: "%f" for floats, "%u" for uint16_t on AVR
345     int cnt = sscanf(line,
346                     "%hhu,%hhu,%f,%f,%u,%u,%u,%u,%f,%f",
347                     &control_mode,
348                     &debug_mode,
349                     &rx_distance,
350                     &rx_angle,
351                     &temp1,
352                     &temp2,
353                     &temp3,
354                     &temp4,
355                     &rx_lin_acc,
356                     &rx_ang_acc);
357
358     if (control_mode == AUTONOMOUS)
359     {
360         rx_max_vel = temp1;
361         rx_max_omega = temp2;
362         rx_last_vel = temp3;
363         rx_last_omega = temp4;
364     }
365     else if (control_mode == TELEOPERATOR)
366     {
367         f = temp1;
368         b = temp2;
369         l = temp3;
370         r = temp4;
371     }
372
373     return cnt;
374 }
375
376 static void receive_from_jetson(void)
377 {
378
379     while (m_usb_rx_available())
380     {
381         char c = m_usb_rx_char();
382         // start parsing if new line detected
383         if (c == '\n' || c == '\r')
384         {
385             if (rx_index > 0)
386             {
387                 rx_buf[rx_index] = '\0';
388                 if (parse_jetson(rx_buf))
389                 {
390
391                     if (debug_mode == RX_ECHO || debug_mode ==
392                         ↪ MD_AND_ECHO)
393                     {
394                         send_cmd_echo();
395                     }
396
397                     if (control_mode == AUTONOMOUS)
398                     {
399                         if (rx_distance != 0 && rx_angle != 0)

```



```

399     {
400         determineFinishnes = BOTH;
401         motion_reset_drive_system();
402         motion_start_move(rx_distance, rx_max_vel,
403             ↪ rx_last_vel, rx_lin_acc);
404         motion_start_turn(rx_angle, rx_max_omega,
405             ↪ rx_last_omega, rx_ang_acc);
406     }
407     else if (rx_distance != 0)
408     {
409         determineFinishnes = FORWARD;
410         motion_reset_drive_system();
411         motion_start_move(rx_distance, rx_max_vel,
412             ↪ rx_last_vel, rx_lin_acc);
413     }
414     else if (rx_angle != 0)
415     {
416         determineFinishnes = ROTATION;
417         motion_reset_drive_system();
418         motion_start_turn(rx_angle, rx_max_omega,
419             ↪ rx_last_omega, rx_ang_acc);
420     }
421     else
422     {
423         determineFinishnes = NONE;
424     }
425     profile_done = false;
426 }
427 else if (control_mode == TELEOPERATOR)
428 {
429     current_command = (f << 3) | (b << 2) | (1 <<
430         ↪ 1) | r;
431
432     bool need_new_profile =
433     (current_command && teleStates == NONETELEOP);
434
435     if (need_new_profile)
436     {
437         teleStates = ACCELERATING;
438         motion_reset_drive_system();
439
440         if (f)
441         {
442             determineFinishnes = FORWARD;
443             motion_start_move(FORWARD_DIST,
444                 ↪ TELEOP_SPEED, TELEOP_SPEED,
445                 ↪ TELEOP_ACC);
446         }
447         else if (b)
448         {
449             determineFinishnes = FORWARD;
450             motion_start_move(-BACKWARD_DIST,
451                 ↪ TELEOP_SPEED, TELEOP_SPEED,
452                 ↪ TELEOP_ACC);
453         }
454         else if (1)
455         {

```

```

448         determineFinishnes = ROTATION;
449         motion_start_turn(LEFT_TURN_ANGLE,
            ↪ TELEOP_OMEGA, TELEOP_OMEGA,
            ↪ TELEOP_ALPHA);
450     }
451     else if (r)
452     {
453         determineFinishnes = ROTATION;
454         motion_start_turn(-RIGHT_TURN_ANGLE,
            ↪ TELEOP_OMEGA, TELEOP_OMEGA,
            ↪ TELEOP_ALPHA);
455     }
456
457     profile_done = false;
458 }
459 else if ((current_command != last_command) &&
    ↪ ((teleStates != DECELERATING) && (
    ↪ teleStates != NONETELEOP)))
460 {
461     teleStates = DECELERATING;
462     motion_SOFT_reset_drive_system();
463
464     if (last_command & 0b1000)
465     {
466         determineFinishnes = FORWARD;
467         motion_start_move(FORWARD_DIST / 2,
            ↪ TELEOP_SPEED, 0, TELEOP_ACC);
468     }
469     else if (last_command & 0b0100)
470     {
471         determineFinishnes = FORWARD;
472         motion_start_move(-BACKWARD_DIST / 2,
            ↪ TELEOP_SPEED, 0, TELEOP_ACC);
473     }
474     else if (last_command & 0b0010)
475     {
476         determineFinishnes = ROTATION;
477         motion_start_turn(LEFT_TURN_ANGLE / 2,
            ↪ TELEOP_OMEGA, 0, TELEOP_ALPHA);
478     }
479     else if (last_command & 0b0001)
480     {
481         determineFinishnes = ROTATION;
482         motion_start_turn(-RIGHT_TURN_ANGLE /
            ↪ 2, TELEOP_OMEGA, 0, TELEOP_ALPHA)
            ↪ ;
483     }
484
485     profile_done = false;
486 }
487 else if ((current_command == last_command) && (
    ↪ teleStates == CONSTANT) && profile_done)
488 {
489     // Maintain constant speed smoothly
490     motors_update(motion_velocity(),
        ↪ motion_omega());
491 }
492

```

```

493         last_command = current_command;
494     }
495 }
496
497     rx_index = 0;
498 }
499 }
500 else if (rx_index < (RX_BUF_SIZE - 1))
501 {
502     rx_buf[rx_index++] = c;
503 }
504 }
505 }

```

Listing 14: Main application loop (main.c)

3 Software Stack Implemented on the Jetson Nano

The Jetson Nano serves as the brain of our AMR, running high-level control loops, sensor fusion, and path planning algorithms. Its software is organized into focused modules- configuration, communication, sensing, localization, planning, and the main execution loop so that individual pieces can be tested, tuned, or replaced without rippling changes across the entire system.

3.1 Configuration & Utilities

Purpose & Overview

A stable, predictable robot requires consistent parameters and reusable helper routines. Our `config.hpp` centralizes all compile-time tuning values-port names, baud rates, EKF covariances, map resolutions so that adapting to a new environment or robot variant is as simple as editing a single header. Meanwhile, `robot_utils.hpp` provides coordinate transforms, timestamping, and logging macros to keep the core modules focused on algorithmic logic rather than boilerplate.

Global Configuration:

- Serial port settings (baud rate, parity, timeouts) tuned to match the ATmega32U4’s capabilities.
- UDP parameters (local/remote IP, telemetry/control ports) to isolate networking details.
- LiDAR driver constants (frame size, packet timeouts) ensuring robust reassembly of scans.
- EKF noise terms (process and measurement variances) exposed for rapid filter tuning.
- Map grid resolution and inflation radius, letting planners stay agnostic to the robot’s footprint.

```

1  #ifndef CONFIG_HPP
2  #define CONFIG_HPP
3
4  #include <cstdint>
5  #include <string>
6  #include <chrono>
7
8  // Timing configuration
9  constexpr int LOOP_TIME = 10; // Main loop time in milliseconds

```

```

10
11 // Serial communication configuration
12 constexpr int DEFAULT_BAUD_RATE = 9600;
13 constexpr int RX_BUFFER_SIZE = 512;
14
15 // UDP communication
16 constexpr uint16_t LOCAL_UDP_PORT = 9000;
17 constexpr uint16_t REMOTE_UDP_PORT = 9001;
18 const std::string REMOTE_IP = "192.168.1.68";
19
20 //LIDAR configuration
21 constexpr int canvas = 1000;
22 constexpr float rangeMax = 15.f;
23 constexpr auto dt = std::chrono::milliseconds(LOOP_TIME);
24
25 // MCU-side enums brought to the host
26 enum ControlMode : uint8_t
27 {
28     AUTONOMOUS = 0,
29     TELEOPERATOR = 1
30 };
31
32 enum DebugMode : uint8_t
33 {
34     DEBUG_OFF = 0,
35     MOTION_DEBUG = 1,
36     RX_ECHO = 2,
37     MD_AND_ECHO = 3
38 };
39
40 enum CommandStatus : uint8_t
41 {
42     CMD_TOBE_WRITTEN = 0,
43     CMD_JUST_WROTE = 1,
44     CMD_EMPTY = 2
45 };
46
47 // System state management
48 struct SystemState
49 {
50     ControlMode controlMode{AUTONOMOUS};
51     DebugMode debugMode{DEBUG_OFF};
52     bool expectDebugData{false};
53     bool expectCommandEcho{false};
54     bool profileExecuted{true}; //Flip this flag to false immediately
55     //    ↳ upon received from UDP com or higher layer path planner class
56     //    ↳ , make it true if it is given to mcu for execution
57     bool profileOngoing{false}; //Flip this flag to true when the
58     //    ↳ profile is being executed, make it false when the profile is
59     //    ↳ finished as reported by the mcu's last received profile
60     //    ↳ execution status bit
61     bool emergencyStop{false}; // Set to true when emergency stop is
62     //    ↳ triggered from MCU throguh the bit recieved from the
63     //    ↳ telemetry data, set to false when the emergency stop is
64     //    ↳ released from teleoperator
65
66     void updateExpectations()
67     {

```

```

60     expectDebugData = (debugMode == MOTION_DEBUG || debugMode ==
        ↪ MD_AND_ECHO);
61     expectCommandEcho = (debugMode == RX_ECHO || debugMode ==
        ↪ MD_AND_ECHO);
62 }
63 };
64
65 #endif // CONFIG_HPP

```

Listing 15: Global Configuration (config.hpp)

Utility Routines:

- *Coordinate conversions*: World $\leftrightarrow$ grid indices for seamless interplay between SLAM maps and image-based planners.
- *Angle math*: Normalize headings to $[-\pi, \pi]$, convert radians $\leftrightarrow$ degrees, and compute angular differences robustly.
- *Timestamps*: High-resolution clocks and formatted strings facilitate consistent logging and CSV output.
- *Logging macros*: Uniform debug/info/warning channels that prepend file, line, and function names for easier traceability.

```

1  #ifndef ROBOT_UTILS_HPP
2  #define ROBOT_UTILS_HPP
3
4  #include "serial_com.hpp"
5  #include <cmath>
6
7  class RobotUtils {
8  public:
9      // Robot physical parameters
10     static constexpr double WHEEL_DIAMETER = 0.2;           // 200mm = 0.2m
11     static constexpr double WHEEL_RADIUS = WHEEL_DIAMETER / 2.0; //
        ↪ 0.1m
12     static constexpr double WHEEL_BASE = 0.5;               // 500mm = 0.5
        ↪ m
13     static constexpr double TICKS_PER_REV = 40000.0;         // 4 Å 1000
        ↪ Å 10 = 40,000 ticks
14     static constexpr double WHEEL_CIRCUMFERENCE = M_PI * WHEEL_DIAMETER
        ↪ ; // Å Å 0.2m = 0.628m
15     static constexpr double METERS_PER_TICK = WHEEL_CIRCUMFERENCE /
        ↪ TICKS_PER_REV;
16
17     // Convert encoder ticks to distance (meters)
18     static double ticksToMeters(long ticks) {
19         return ticks * METERS_PER_TICK;
20     }
21
22     // Convert encoder tick difference to velocity (m/s)
23     static double tickDiffToVelocity(float tickDiff, double dt) {
24         return (tickDiff * METERS_PER_TICK) / dt;
25     }
26
27     // Convert degrees to radians
28     static double degToRad(double degrees) {

```

```

29         return degrees * M_PI / 180.0;
30     }
31
32     // Convert radians to degrees
33     static double radToDeg(double radians) {
34         return radians * 180.0 / M_PI;
35     }
36
37     // Extract wheel velocities from SensorPacket
38     static std::pair<double, double> getWheelVelocities(const
39         ↪ SensorPacket& sensor, double dt) {
40         double leftVel = tickDiffToVelocity(sensor.dEncL, dt);
41         double rightVel = tickDiffToVelocity(sensor.dEncR, dt);
42         return {leftVel, rightVel};
43     }
44
45     // Convert angular velocity from tick-difference to rad/s
46     static double angularVelocityFromEncoders(const SensorPacket&
47         ↪ sensor, double dt) {
48         auto [leftVel, rightVel] = getWheelVelocities(sensor, dt);
49         return (rightVel - leftVel) / WHEEL_BASE;
50     }
51
52     // Normalize angle to [-π, π]
53     static double normalizeAngle(double angle) {
54         while (angle > M_PI) angle -= 2.0 * M_PI;
55         while (angle < -M_PI) angle += 2.0 * M_PI;
56         return angle;
57     }
58 };
59 #endif // ROBOT_UTILS_HPP

```

Listing 16: Utility Routines (robot_utils.hpp)

3.2 Communication Modules

3.2.1 Serial Communication Handler

Role & Design

Robust direct control of motors and low-level sensors lives on the ATmega32U4, so we need a rock-solid UART channel. Our C++ `serial_com.cpp/.hpp` pair encapsulates packet framing, CRC checks, and thread-safe buffering to guarantee that no telemetry is lost, even if bursts of debug messages occur.

Packet Definitions:

- `SensorPacket`, `MotionDebugPacket`, `CommandEchoPacket` structs packed to MCU byte layout.
- Fixed preamble and CRC16 footer on each packet to detect corruption.

I/O Architecture:

- A background reader thread fills a circular byte buffer at up to 1 Mbps, ensuring no data overrun.

- Parser state machine extracts complete packets, validates CRC, and pushes them into a synchronized queue.
- Sender API formats `CommandPacket` structs, computes CRC, and writes atomically to the UART port.

Fault Detection & Recovery:

- If no valid packet arrives within a configurable timeout, the port is reset automatically.
- CRC error rates and dropped-packet counters are exposed for health monitoring.

```

1 // serial_com.hpp
2 #ifndef SERIAL_COM_HPP
3 #define SERIAL_COM_HPP
4
5 #include <atomic>
6 #include <cstdint>
7 #include <mutex>
8 #include <string>
9 #include <termios.h>
10 #include <vector>
11 #include "../core/config.hpp"
12
13 /* Inbound packets */
14 struct SensorPacket
15 {
16     // Orientation (degrees)
17     float yaw{}, roll{}, pitch{};
18
19     // Gyroscope readings (deg/s) on X, Y, Z axes
20     float gyroX{}, gyroY{}, gyroZ{};
21
22     // Accelerometer readings (m/s²) on X, Y, Z axes
23     float accelX{}, accelY{}, accelZ{};
24
25     // Encoder counts
26     long encL{}, encR{};
27
28     // Battery voltages (mV) and cliff sensor readings
29     uint16_t vbat1{}, vbat2{}, cliffL{}, cliffC{}, cliffR{};
30
31     // Flags
32     uint8_t emergency{}, profileDone{};
33
34     // Deltas & derived velocities
35     float dYaw{}, dEncL{}, dEncR{}, linVel{}, angVel{};
36
37     // Validity flag
38     bool valid{false};
39 };
40
41 struct MotionDebugPacket
42 {
43     float spdL{}, spdR{}, vel{}, omg{}, dist{}, ang{}, loopDt{};
44     bool valid{false};
45 };
46

```

```

47 struct CommandEchoPacket
48 {
49     float distance{}, angle{};
50     uint16_t maxVel{}, maxOmega{}, lastVel{}, lastOmega{};
51     float linAcc{}, angAcc{};
52     bool valid{false};
53 };
54
55 /* âĀĀâĀĀâĀĀ outbound commands (host âĒĒ MCU) âĀĀâĀĀâĀĀ */
56 struct CommandPacket
57 {
58     ControlMode mode{AUTONOMOUS};
59     DebugMode dbg{DEBUG_OFF};
60     CommandStatus cmdStatus{CMD_EMPTY};
61
62     // AUTONOMOUS fields
63     float distance{}, angle{};
64     uint16_t maxVel{}, maxOmega{}, lastVel{}, lastOmega{};
65     float linAcc{}, angAcc{};
66
67     // TELEOPERATOR fields (0/1)
68     uint8_t f{}, b{}, l{}, r{};
69 };
70
71 class Serial_Com
72 {
73 public:
74     explicit Serial_Com(const std::string &port, int baud =
75         ↪ DEFAULT_BAUD_RATE);
76     ~Serial_Com();
77
78     // Non-blocking: read, parse, update internal packets
79     void spinOnce(CommandPacket &cmd);
80
81     // Send a new command (marks cmdStatus)
82     void sendCommand(CommandPacket &cmd);
83
84     // Thread-safe getters for the latest data
85     SensorPacket getSensor() const;
86     MotionDebugPacket getDebug() const;
87     CommandEchoPacket getCommandEcho() const;
88
89     // System state (toggles which parsers run)
90     void setSystemState(const SystemState &state);
91     SystemState getSystemState() const;
92
93     // Debug/test
94     void testCommunication();
95     void printSensorData(const SensorPacket &sensor) const;
96     void printDebugData(const MotionDebugPacket &debug) const;
97     void printCommandEcho(const CommandEchoPacket &echo) const;
98
99     bool isConnected() const { return fd >= 0; }
100
101     // Find available /dev/tty* ports
102     static std::vector<std::string> getAvailablePorts();
103 private:

```



```

104     int fd{-1};
105     std::string rxBuf;
106     mutable std::mutex mtx;
107
108     SensorPacket sensor{};
109     MotionDebugPacket debug{};
110     CommandEchoPacket cmdEcho{};
111     SystemState sysState{};
112
113     // Parsers
114     bool parseCombinedLine(const std::string &line);
115     bool parseTelemetryOnly(const std::string &line);
116     bool parseDebugOnly(const std::string &line);
117     bool parseCommandEcho(const std::string &line);
118
119     // Helpers
120     static speed_t baudToTermios(int baud);
121     void writeLine(const std::string &line);
122     void updateSensorDeltas(SensorPacket &s);
123
124     static bool deltaInit;
125     static float lastYaw;
126     static long lastEncL, lastEncR;
127 };
128
129 #endif // SERIAL_COM_HPP

```

Listing 17: serial_com.hpp

```

1  #include "serial_com.hpp"
2  #include <cmath>
3  #include <cstring>
4  #include <fcntl.h>
5  #include <iomanip>
6  #include <iostream>
7  #include <sstream>
8  #include <unistd.h>
9  #include <termios.h>
10 #include <errno.h>
11 #include <thread>
12 #include <chrono>
13 #include <dirent.h>
14
15 bool Serial_Com::deltaInit = false;
16 float Serial_Com::lastYaw = 0.0f;
17 long Serial_Com::lastEncL = 0;
18 long Serial_Com::lastEncR = 0;
19
20 /* ~~~~~ ctor / dtor ~~~~~
   ↳ ~~~~~ */
21 Serial_Com::Serial_Com(const std::string &port, int baud)
22 {
23     fd = ::open(port.c_str(), O_RDWR | O_NOCTTY | O_SYNC);
24     if (fd < 0)
25     {
26         perror("open");
27         throw std::runtime_error("serial open failed");
28     }

```



```

84         {
85             std::cout << "[RAW] " << rxBuf << std::endl;
86             bool handled = false;
87
88             // 1) command-echo right after send
89             if (sysState.expectCommandEcho && cmd.cmdStatus ==
90                 → CMD_JUST_WROTE)
91             {
92                 if (parseCommandEcho(rxBuf))
93                 {
94                     cmd.cmdStatus = CMD_EMPTY;
95                     handled = true;
96                 }
97             }
98             // 2) debug+telemetry combined
99             if (!handled && sysState.expectDebugData)
100             {
101                 handled = parseCombinedLine(rxBuf);
102             }
103             // 3) echo fallback
104             if (!handled && sysState.expectCommandEcho)
105             {
106                 handled = parseCommandEcho(rxBuf);
107                 if (handled)
108                     cmd.cmdStatus = CMD_EMPTY;
109             }
110             // 4) pure telemetry
111             if (!handled)
112                 parseTelemetryOnly(rxBuf);
113
114             rxBuf.clear();
115         }
116         else if (ch >= 32 && ch <= 126)
117         {
118             rxBuf += ch;
119         }
120     }
121 }
122 else if (bytes < 0 && errno != EAGAIN && errno != EWOULDBLOCK)
123 {
124     perror("read");
125 }
126 }
127
128 void Serial_Com::sendCommand(CommandPacket &cmd)
129 {
130     if (fd < 0)
131         return;
132     std::ostringstream oss;
133     if (cmd.mode == AUTONOMOUS)
134     {
135         oss << int(cmd.mode) << ',' << int(cmd.dbg) << ',' <<
136             << cmd.distance << ',' << cmd.angle << ',' <<
137             << cmd.maxVel << ',' << cmd.maxOmega << ',' <<
138             << cmd.lastVel << ',' << cmd.lastOmega << ',' <<
139             << cmd.linAcc << ',' << cmd.angAcc << "\r\n";
140     }

```



```

197         "%f %f %f %f %f %f %f %f %f %f "
198         "%f %f %f %f %f %f %f %f %f %f "
199         "%ld %ld %u %u %u %u %u %u %u %u",
200         &spdL, &spdR, &vel, &omg, &dist, &ang, &dt,
201         &yaw, &roll, &pitch, &gX, &gY, &gZ, &aX, &aY, &aZ,
202         &eL, &eR, &v1, &v2, &cL, &cC, &cR, &em, &pr);
203     if (p != 25)
204         return false;
205
206     MotionDebugPacket d{spdL, spdR, vel, omg, dist, ang, dt, true};
207     SensorPacket s;
208     s.yaw = yaw;
209     s.roll = roll;
210     s.pitch = pitch;
211     s.gyroX = gX;
212     s.gyroY = gY;
213     s.gyroZ = gZ;
214     s.accelX = aX;
215     s.accelY = aY;
216     s.accelZ = aZ;
217     s.encL = eL;
218     s.encR = eR;
219     s.vbat1 = v1;
220     s.vbat2 = v2;
221     s.cliffL = cL;
222     s.cliffC = cC;
223     s.cliffR = cR;
224     s.emergency = uint8_t(em);
225     s.profileDone = uint8_t(pr);
226     s.valid = true;
227
228     updateSensorDeltas(s);
229     {
230         std::scoped_lock lk(mtx);
231         debug = d;
232         sensor = s;
233     }
234     std::cout << "[PARSED] Combined debug+telemetry data\n";
235     return true;
236 }
237
238 bool Serial_Com::parseTelemetryOnly(const std::string &line)
239 {
240     // 18 fields: 9 floats, 2 longs, 7 ints
241     float yaw, roll, pitch, gX, gY, gZ, aX, aY, aZ;
242     long eL, eR;
243     unsigned v1, v2, cL, cC, cR, em, pr;
244     int p = std::sscanf(
245         line.c_str(),
246         "%f %f %f %f %f %f %f %f %f %f %ld %ld %u %u %u %u %u %u",
247         &yaw, &roll, &pitch, &gX, &gY, &gZ, &aX, &aY, &aZ,
248         &eL, &eR, &v1, &v2, &cL, &cC, &cR, &em, &pr);
249     if (p != 18)
250     {
251         std::cout << "[PARSE] Failed telemetry: " << line << "\n";
252         return false;
253     }
254

```

```

255     SensorPacket s;
256     s.yaw = yaw;
257     s.roll = roll;
258     s.pitch = pitch;
259     s.gyroX = gX;
260     s.gyroY = gY;
261     s.gyroZ = gZ;
262     s.accelX = aX;
263     s.accelY = aY;
264     s.accelZ = aZ;
265     s.encL = eL;
266     s.encR = eR;
267     s.vbat1 = v1;
268     s.vbat2 = v2;
269     s.cliffL = cL;
270     s.cliffC = cC;
271     s.cliffR = cR;
272     s.emergency = uint8_t(em);
273     s.profileDone = uint8_t(pr);
274     s.valid = true;
275
276     updateSensorDeltas(s);
277     {
278         std::scoped_lock lk(mtx);
279         sensor = s;
280     }
281     std::cout << "[PARSED] Telemetry only\n";
282     return true;
283 }
284
285 bool Serial_Com::parseDebugOnly(const std::string &line)
286 {
287     float spdL, spdR, vel, omg, dist, ang, dt;
288     if (std::sscanf(line.c_str(), "%f %f %f %f %f %f %f",
289                     &spdL, &spdR, &vel, &omg, &dist, &ang, &dt) != 7)
290         return false;
291     MotionDebugPacket d{spdL, spdR, vel, omg, dist, ang, dt, true};
292     {
293         std::scoped_lock lk(mtx);
294         debug = d;
295     }
296     std::cout << "[PARSED] Debug only\n";
297     return true;
298 }
299
300 bool Serial_Com::parseCommandEcho(const std::string &line)
301 {
302     if (line.rfind("CMD", 0) != 0)
303         return false;
304     CommandEchoPacket e{};
305     if (std::sscanf(
306         line.c_str(),
307         "CMD d=%f a=%f vmax=%hu wmax=%hu vend=%hu wend=%hu acc=%f
308         ↪ aacc=%f",
309         &e.distance, &e.angle,
310         &e.maxVel, &e.maxOmega,
311         &e.lastVel, &e.lastOmega,
312         &e.linAcc, &e.angAcc) != 8)

```



```

425     ↪ endl;
426     sendCommand(cmd);
427     std::this_thread::sleep_for(std::chrono::milliseconds(10000));
428     // Test 3: Send a rotation command
429     cmd.distance = 0.0f;
430     cmd.angle = 180.0f; // 180 deg turn
431     cmd.maxVel = 0; // no linear
432     cmd.maxOmega = 120; // 120 deg/s
433     cmd.linAcc = 0.0f;
434     cmd.angAcc = 360.0f; // 360 deg/s↪
435
436     std::cout << "Sending rotation command (90 degrees)..." << std::
437     ↪ endl;
438     sendCommand(cmd);
439     std::this_thread::sleep_for(std::chrono::milliseconds(10000));
440     // Test 4: Test teleoperator mode
441     cmd.mode = TELEOPERATOR;
442     cmd.dbg = RX_ECHO;
443     cmd.f = 1; // forward
444     cmd.b = 0;
445     cmd.l = 0;
446     cmd.r = 0;
447
448     std::cout << "Sending teleoperator command (forward)..." << std::
449     ↪ endl;
450     sendCommand(cmd);
451     std::this_thread::sleep_for(std::chrono::milliseconds(200));
452
453     std::cout << "Communication test complete." << std::endl;
454 }
455 void Serial_Com::printSensorData(const SensorPacket &sensor) const
456 {
457     if (!sensor.valid)
458         return;
459
460     std::cout << std::fixed << std::setprecision(2);
461     std::cout << "IMU: Y:" << sensor.yaw << "↪ R:" << sensor.roll
462     << "↪ P:" << sensor.pitch << "↪ ";
463     std::cout << "Gyro: X:" << sensor.gyroX << " Y:" << sensor.gyroY
464     << " Z:" << sensor.gyroZ << " ";
465     std::cout << "Accel: X:" << sensor.accelX << " Y:" << sensor.accelY
466     << " Z:" << sensor.accelZ << " ";
467     std::cout << "Enc: L:" << sensor.encL << " R:" << sensor.encR << "
468     ↪ ";
469     std::cout << "Bat: " << sensor.vbat1 << "mV/" << sensor.vbat2 << "
470     ↪ mV ";
471     std::cout << "Cliff: L:" << sensor.cliffL << " C:" << sensor.cliffC
472     << " R:" << sensor.cliffR << " ";
473     std::cout << "Emerg:" << int(sensor.emergency)
474     << " Done:" << int(sensor.profileDone);
475 }
476 void Serial_Com::printDebugData(const MotionDebugPacket &debug) const
477 {
478     if (!debug.valid)

```

```

478         return;
479
480     std::cout << std::fixed << std::setprecision(1);
481     std::cout << "Motion: SpdL:" << debug.spdL << " SpdR:" << debug.
        ↳ spdR
482         << " Vel:" << debug.vel << " Omg:" << debug.omg
483         << " Dist:" << debug.dist << " Ang:" << debug.ang
484         << " dt:" << debug.loopDt << "Âts";
485 }
486
487 void Serial_Com::printCommandEcho(const CommandEchoPacket &echo) const
488 {
489     if (!echo.valid)
490         return;
491
492     std::cout << std::fixed << std::setprecision(1);
493     std::cout << "Echo: d=" << echo.distance << " a=" << echo.angle
494         << " vmax=" << echo.maxVel << " wmax=" << echo.maxOmega
495         << " vend=" << echo.lastVel << " wend=" << echo.lastOmega
496         << " acc=" << echo.linAcc << " aacc=" << echo.angAcc;
497 }

```

Listing 18: serial_com.cpp

3.2.2 UDP Communication Handler

Role & Design

For real-time telemetry and remote command injection, UDP offers minimal latency and lower overhead than TCP. Using Boost.Asio, `udp_com.cpp/.hpp` provides non-blocking sockets, loss detection, and thread-safe send queues.

Command Reception:

- Listens on `UDP_LOCAL_PORT` for incoming control messages (velocity setpoints, goal poses).
- Deserializes and immediately updates atomic command variables.

Telemetry Broadcasting:

- Packages IMU readings, encoder counts, battery voltage, and optional map or LiDAR chunks into sequenced UDP frames.
- Sequence numbers help the remote UI detect missing datagrams.

Threading Model:

- Separate threads for receive and transmit to avoid blocking the main loop.
- Send queue with backpressure: if the queue depth exceeds a threshold, low-priority debug messages are dropped first.

```

1 // udp_com.hpp
2 #ifndef UDP_COM_HPP
3 #define UDP_COM_HPP
4
5 #include "config.hpp"

```

```

6  #include "communication/serial_com.hpp" // for CommandPacket,
   ↪ SensorPacket, MotionDebugPacket
7  #include <stdint>
8  #include <string>
9  #include <thread>
10 #include <atomic>
11 #include <vector>
12 #include <netinet/in.h> // sockaddr_in
13
14 class UDPCom {
15 public:
16     UDPCom(uint16_t localPort,
17             const std::string &remoteIP,
18             uint16_t remotePort);
19     ~UDPCom();
20
21     // Start/stop background receiver
22     void start();
23     void stop();
24
25     // Outgoing telemetry
26     void sendCommandEcho(const CommandPacket &cmdEcho);
27     void sendSensor      (const SensorPacket &s);
28     void sendDebug       (const MotionDebugPacket &d);
29
30     // Bulk data
31     void sendMap          (const std::vector<uint8_t> &mapData);
32     void sendPointCloud   (const std::vector<uint8_t> &pcData);
33
34 private:
35     void recvLoop();
36     void handleIncoming(const std::string &msg);
37
38     int sockfd_;
39     sockaddr_in localAddr_, remoteAddr_;
40     std::thread recvThread_;
41     std::atomic<bool> running_{false};
42     static constexpr size_t MAX_UDP_PAYLOAD = 1200;
43 };
44
45 #endif // UDP_COM_HPP

```

Listing 19: udp_com.hpp

```

1  #include "communication/udp_com.hpp"
2  #include <arpa/inet.h>
3  #include <sys/socket.h>
4  #include <unistd.h>
5  #include <iostream>
6  #include <sstream>
7  #include <cstring>
8
9  // These globals are defined in main.cpp
10 extern SensorPacket g_sensor;
11 extern MotionDebugPacket g_debug;
12 extern CommandPacket g_cmd;
13 extern std::mutex g_cmd_mtx;
14

```

```

15 UDPCom::UDPCom(uint16_t localPort,
16                 const std::string &remoteIP,
17                 uint16_t remotePort)
18 {
19     sockfd_ = socket(AF_INET, SOCK_DGRAM, 0);
20     if (sockfd_ < 0)
21         throw std::runtime_error("UDP socket creation failed");
22
23     std::memset(&localAddr_, 0, sizeof(localAddr_));
24     localAddr_.sin_family = AF_INET;
25     localAddr_.sin_addr.s_addr = INADDR_ANY;
26     localAddr_.sin_port = htons(localPort);
27     if (bind(sockfd_, (sockaddr *)&localAddr_, sizeof(localAddr_)) < 0)
28         perror("UDP bind");
29
30     std::memset(&remoteAddr_, 0, sizeof(remoteAddr_));
31     remoteAddr_.sin_family = AF_INET;
32     remoteAddr_.sin_port = htons(remotePort);
33     inet_pton(AF_INET, remoteIP.c_str(), &remoteAddr_.sin_addr);
34 }
35
36 UDPCom::~UDPCom()
37 {
38     stop();
39     close(sockfd_);
40 }
41
42 void UDPCom::start()
43 {
44     running_ = true;
45     recvThread_ = std::thread(&UDPCom::recvLoop, this);
46 }
47
48 void UDPCom::stop()
49 {
50     running_ = false;
51     if (recvThread_.joinable())
52         recvThread_.join();
53 }
54
55 void UDPCom::recvLoop()
56 {
57     char buf[2048];
58     sockaddr_in src{};
59     socklen_t srclen = sizeof(src);
60     while (running_)
61     {
62         int n = recvfrom(sockfd_, buf, sizeof(buf) - 1, 0,
63                         (sockaddr *)&src, &srclen);
64         if (n > 0)
65         {
66             std::string msg(buf, n);
67             handleIncoming(msg);
68         }
69     }
70 }
71
72 void UDPCom::handleIncoming(const std::string &msg)

```

```

73 {
74     // Parse mode/debug changes and full CommandPacket
75     if (msg.rfind("CMD,", 0) == 0)
76     {
77         // Format: CMD,<mode>,<dbg>,<distance>,<angle>,<maxVel>,<
78             ↪ maxOmega>,<
79             // <lastVel>,<lastOmega>,<linAcc>,<angAcc>
80         std::istringstream ss(msg.substr(4));
81         CommandPacket c{};
82         char comma;
83         int m, d;
84         if (ss >>
85             m >> comma >> d >> comma >> c.distance >> comma >> c.angle
86             ↪ >> comma >> c.maxVel >> comma >> c.maxOmega >> comma
87             ↪ >> c.lastVel >> comma >> c.lastOmega >> comma >> c.
88             ↪ linAcc >> comma >> c.angAcc)
89         {
90             std::lock_guard<std::mutex> lk(g_cmd_mtx);
91             if (g_cmd.cmdStatus != CMD_TOBE_WRITTEN)
92             {
93                 c.mode = static_cast<ControlMode>(m);
94                 c.dbg = static_cast<DebugMode>(d);
95                 c.cmdStatus = CMD_TOBE_WRITTEN;
96                 g_cmd = c; // update global command
97                 std::cout << "[UDP] CMD received: mode=" << m << " dbg="
98                 ↪ " << d << "\n";
99             }
100         }
101         return;
102     }
103     else if (msg.rfind("TELEOP,", 0) == 0)
104     {
105         std::istringstream ss(msg.substr(7));
106         int f, b, l, r;
107         char comma;
108         if (ss >> f >> comma >> b >> comma >> l >> comma >> r)
109         {
110             std::lock_guard<std::mutex> lk(g_cmd_mtx);
111             // fill the global command packet
112             g_cmd.mode = TELEOPERATOR;
113             g_cmd.dbg = RX_ECHO; // or whatever debug mode you prefer
114             g_cmd.f = static_cast<uint8_t>(f);
115             g_cmd.b = static_cast<uint8_t>(b);
116             g_cmd.l = static_cast<uint8_t>(l);
117             g_cmd.r = static_cast<uint8_t>(r);
118             g_cmd.cmdStatus = CMD_TOBE_WRITTEN;
119             std::cout << "[UDP] TELEOP RC: f=" << f
120                 << " b=" << b
121                 << " l=" << l
122                 << " r=" << r << "\n";
123         }
124         return;
125     }
126 }
127
128 void UDPCom::sendCommandEcho(const CommandPacket &cmdEcho)
129 {
130     std::ostringstream ss;

```

```

126     ss << "ECHO,"
127     << cmdEcho.distance << "," << cmdEcho.angle << ","
128     << cmdEcho.maxVel << "," << cmdEcho.maxOmega << ","
129     << cmdEcho.lastVel << "," << cmdEcho.lastOmega << ","
130     << cmdEcho.linAcc << "," << cmdEcho.angAcc;
131     auto str = ss.str();
132     sendto(sockfd_, str.data(), str.size(), 0,
133           (sockaddr *)&remoteAddr_, sizeof(remoteAddr_));
134 }
135
136 void UDPCom::sendSensor(const SensorPacket &s)
137 {
138     std::ostringstream ss;
139     ss << "SENSOR,"
140     << s.yaw << "," << s.roll << "," << s.pitch << ","
141     << s.gyroX << "," << s.gyroY << "," << s.gyroZ << ","
142     << s.accelX << "," << s.accelY << "," << s.accelZ << ","
143     << s.encL << "," << s.encR << ","
144     << s.vbat1 << "," << s.vbat2 << ","
145     << s.cliffL << "," << s.cliffC << "," << s.cliffR << ","
146     << int(s.emergency) << "," << int(s.profileDone);
147     auto str = ss.str();
148     sendto(sockfd_, str.data(), str.size(), 0,
149           (sockaddr *)&remoteAddr_, sizeof(remoteAddr_));
150 }
151
152 void UDPCom::sendDebug(const MotionDebugPacket &d)
153 {
154     std::ostringstream ss;
155     ss << "DEBUG,"
156     << d.spdL << "," << d.spdR << ","
157     << d.vel << "," << d.omg << ","
158     << d.dist << "," << d.ang << ","
159     << d.loopDt;
160     auto str = ss.str();
161     sendto(sockfd_, str.data(), str.size(), 0,
162           (sockaddr *)&remoteAddr_, sizeof(remoteAddr_));
163 }
164
165 void UDPCom::sendMap(const std::vector<uint8_t> &mapData)
166 {
167     size_t total = mapData.size(), offset = 0, chunkID = 0;
168     size_t totalChunks = (total + MAX_UDP_PAYLOAD - 1) /
169     ↪ MAX_UDP_PAYLOAD;
170     while (offset < total)
171     {
172         size_t chunkSize = std::min(MAX_UDP_PAYLOAD, total - offset);
173         std::ostringstream hdr;
174         hdr << "MAP," << chunkID << "," << totalChunks << ",";
175         auto h = hdr.str();
176         sendto(sockfd_, h.data(), h.size(), 0,
177               (sockaddr *)&remoteAddr_, sizeof(remoteAddr_));
178         sendto(sockfd_,
179               reinterpret_cast<const char *>(mapData.data() + offset),
180               chunkSize, 0,
181               (sockaddr *)&remoteAddr_, sizeof(remoteAddr_));
182         offset += chunkSize;
183         ++chunkID;

```

```

183     }
184 }
185
186 void UDPCom::sendPointCloud(const std::vector<uint8_t> &pcData)
187 {
188     size_t total = pcData.size(), offset = 0, chunkID = 0;
189     size_t totalChunks = (total + MAX_UDP_PAYLOAD - 1) /
190         ↪ MAX_UDP_PAYLOAD;
191     while (offset < total)
192     {
193         size_t chunkSize = std::min(MAX_UDP_PAYLOAD, total - offset);
194         std::ostringstream hdr;
195         hdr << "PC," << chunkID << "," << totalChunks << ",";
196         auto h = hdr.str();
197         sendto(sockfd_, h.data(), h.size(), 0,
198             (sockaddr *)&remoteAddr_, sizeof(remoteAddr_));
199         sendto(sockfd_,
200             reinterpret_cast<const char *>(pcData.data() + offset),
201             chunkSize, 0,
202             (sockaddr *)&remoteAddr_, sizeof(remoteAddr_));
203         offset += chunkSize;
204         ++chunkID;
205     }
206 }

```

Listing 20: udp_com.cpp

3.3 Sensor Data Acquisition

3.3.1 LiDAR Data Receiver

Role & Design

The LakiBeam LiDAR streams over UDP; we must reassemble, filter, and convert these into usable scans at up to 20 Hz. This pipeline spans low-level drivers to high-level handlers.

Low-level Driver:

- Defines raw packet formats (`udp_pack_t`, `depth_t`) and endianness-fix macros.
- Two threads: one pulls raw UDP packets, the other stitches complete 360° scans by tracking packet counters.
- Double-buffer pattern with mutexes ensures that the processing thread never blocks the receiver.

```

1  #ifndef LAKIBeamUDP_H
2  #define LAKIBeamUDP_H
3
4  #include <boost/thread.hpp>
5  #include <boost/asio.hpp>
6  #include <boost/scoped_ptr.hpp>
7  #include <boost/smart_ptr.hpp>
8  #include <boost/date_time/posix_time/posix_time.hpp>
9  #include <boost/bimap.hpp>
10
11 #include <sstream>
12 #include <cstdlib>

```

```

13 #include <iostream>
14 #include <stdio.h>
15 #include <limits>
16
17 using namespace std;
18
19 ///
20     ➔ âũšä;ŁçŦĺăčřæŸŮïŽçŦĺæĭěæŭĹéŽđæŁă;ŁçŦĺăŘŸéĞŘæĹěèŋęïÑæšqæĹăĹŖèĈ;ăŸĹéĬççŽĎæĎŘ
21     ➔
22 #ifndef USED
23 #define USED(x) ((void)x)
24 #endif
25
26 /// èřĈèřŤæŮěăŘčïŽăĹńéĂŖæĹŖăĎřèřĈèřŤăĹqæĹř
27 #ifndef DEBUG
28 #define DEBUG(x) (cout << x << endl)
29 #endif
30
31 ///
32     ➔ âŘŮæŭĹăŋŮèĹĈăřžé;ŘïŽăŸŦĹéĬăŏŽăžĹçŽĎæŤřæĎŏçžŖæđĎă;ŖăŸŮéŽŭèççŽĎăĎŘèŏŏçžŖæđĎçŽ
33     ➔
34 #define __autoalign__ __attribute__((packed))
35
36 /**
37  *
38  * ➔ äŸžăžĚççžŖăŸĂăžčçăĹéĈŮăïŦăřžéĈĺăĹĚæŤřæĎŏççžăđŦèĹŽèqŦăžĚéĞĎæŮřăŏŽăžĹ
39  * ➔
40 */
41 #ifndef i32_t
42 typedef signed int i32_t;
43 #endif
44 #ifndef u32_t
45 typedef unsigned int u32_t;
46 #endif
47 #ifndef u16_t
48 typedef unsigned short u16_t;
49 #endif
50 #ifndef u16_t
51 typedef unsigned short i16_t;
52 #endif
53 #ifndef u8_t
54 typedef unsigned char u8_t;
55 #endif
56 #ifndef c8_t
57 typedef char i8_t;
58 #endif
59
60 #ifndef udp_socket_t
61 typedef boost::asio::ip::udp::socket udp_socket_t;
62 #endif
63 #ifndef address_t
64 typedef boost::asio::ip::address address_t;
65 #endif
66 #ifndef service_t
67 typedef boost::asio::io_service service_t;
68 #endif
69 typedef boost::scoped_ptr<boost::thread> thread_t;
70 #ifndef service_work_t

```



```

118     u8_t rssi0;          ///< 0x00 to 0xFF
119     u16_t distance1;
120     u8_t rssi1;
121 };
122
123 /** @struct little_pack_t UDP packet structure
124  * @brief äÿÄäÿÿéZüë;æûsâžæTŗæDôçZDæTŗæDôçzSædDçZDæÄäRâDÿäËÇ
125  */
126 struct little_pack_t
127 {
128     u16_t head;           ///< æTŗæDôäðt
129     u16_t azimuth;        ///< ætNëüIæTŗæDôçZDëÿSâžæäÄ
130     depth_t depth[CONFIG_BLOCK_DEPTHS]; ///< æûsâžææTŗæDôçÇzä;DâNË
131 };
132
133 /** @struct udp_pack_t UDP packet structure
134  * @brief äÿÄäÿÿUDPæTŗæDôäNËçZDæTŗæDôçzSædD
135  */
136 struct udp_pack_t
137 {
138     little_pack_t depths[CONFIG_UDP_BLOCKS]; ///< æûsâžææTŗæDôäDÿäËÇ
139     u32_t timestamp;      ///< æÛüéÛtæLç
140     u16_t factory;        ///< äüëäÛÇäLqæÄŗ
141 };
142
143 /** @struct cicle_pack_t UDP packet structure
144  * @brief éGDNæÛŗäŗÄëçËçZDæûsâžææTŗæDôäNËæÄäRâDÿäËÇ
145  */
146 struct cicle_pack_t
147 {
148     u32_t timestamp;      ///< æÛüéÛtæLç
149     u16_t angle;          ///< èÿSâžæäÄ
150     u16_t distance;       ///< æûsâžææTŗæDô
151     u8_t rssi;            ///< 0x00 to 0xFF
152 };
153
154 /** @struct repark_t
155  * → æûsâžææTŗæDôäIÛiNäÿÄäIÛäNËäRñéZüë;äÿÄäSÍçZDæûsâžææTŗæDô
156  * @brief äÿÄäÿÿgäzçèqIäÿÄäIÛæûsâžææTŗæDô
157  */
158 struct repark_t
159 {
160     u32_t interval;       ///< éÛtëZTæÛüéÛt
161     u16_t maxdots;        ///< çÇzä;DäÿtæTŗ
162     cicle_pack_t dotcloud[CONFIG_CIRCLE_DOTS]; ///< æûsâžææTŗæDôçÇzäžS
163 };
164 #pragma pack(pop)
165
166 class LakiBeamUDP
167 {
168 public:
169     /*!
170         ædDëÄäãG;æTŗ
171         \param local_ip æñäŗip
172         \param local_port æñäŗçñŗäRçäRü
173         \param laser_ip éZüë;ip
174         \param laser_port éZüë;çñŗäRçäRü

```

```

175      */
176      LakiBeamUDP(string local_ip, string local_port, string laser_ip,
177      ↪ string laser_port);
178      /*!
179      æĎŘæĎĎăĜ;æŤř
180      */
181      ~LakiBeamUDP();
182      /*!
183      âĤŇæŋěěĜĎæŮřçžĎăŇĚăĽŖěĈ;
184      \param pack çŤĹăĹăăŇĚěçžăĜžăŘč
185      */
186      void sync_get_repackedpack(repack_t &pack);
187      /*!
188      âĤŮăĜžæŤřæĎŏăŇĚěçž èĽŤăŽăĥtrueéĈčăžĹçŋŇăĎŖèĽŤăŽăæŤřæĎŏ
189      \param pack çŤĹăĹăăŇĚěçžăĜžăŘč
190      */
191      bool get_repackedpack(repack_t &pack);
192
193 private:
194     string local_ip;
195     string local_port;
196     string laser_ip;
197     string laser_port;
198
199     thread_t receive_thd;
200     thread_t udprepack_thd;
201
202     mutex_t thd_mutex;
203     bool everythingOK;
204     volatile u32_t dbmain;
205     volatile u32_t dbmain_used;
206     udp_pack_t doublebuffer[CONFIG_FRAME_MAX];
207
208     volatile u32_t urpmain;
209     volatile u32_t urpmain_used;
210     repack_t udprepack[CONFIG_MAX_REPACK];
211
212     char bufffff[CONFIG_FRAME];
213     udp_socket_t* socket;
214     boost::asio::io_service io_servicess;
215     udp_endpoint_t *local_ep;
216     udp_endpoint_t *laser_ep;
217
218 private:
219     /*!
220     æŮěăĤŮUDPæŤřæĎŏăŇĚçžĽçĹŇ
221     */
222     void receive_thread(void);
223     /*!
224     UDPæŇĚăŇĚçžĎăŇĚçžĽçĹŇ
225     */
226     void udprepack_thread(void);
227     /*!
228     âqŋăĚĚæšqæĽèťăăĝŇăŇĚçžĎăŇĚěçžîĹăŷĂæŋqéĂèçĹăŇĚîĽ
229     \param pack âŇĚěçžăĚěăĤčæŤřçžĎ
230     */
231     void fill_nostart(udp_pack_t pack[2]);
232     /*!

```



```

38         catch (boost::system::error_code& err)
39         {
40             break;
41         }
42     }
43 }
44
45 LakiBeamUDP::~LakiBeamUDP()
46 {
47     io_servicess.stop();
48
49     receive_thd->interrupt();
50     receive_thd->join();
51
52     delete socket;
53     delete local_ep;
54     delete laser_ep;
55
56     udprepack_thd->interrupt();
57     udprepack_thd->join();
58 }
59
60 void LakiBeamUDP::sync_get_repackedpack(repack_t &pack)
61 {
62     while (urpmain < 2)
63     {
64     }
65     while (urpmain <= (urpmain_used + 1))
66     {
67     }
68     urpmain_used = urpmain - 1;
69
70     pack = udprepack[(urpmain_used) % CONFIG_MAX_REPACK];
71 }
72
73 bool LakiBeamUDP::get_repackedpack(repack_t &pack)
74 {
75     bool result = true;
76
77     if ((urpmain < 2) || (urpmain <= (urpmain_used + 1)))
78     {
79         result = false;
80     }
81     else
82     {
83         urpmain_used = urpmain - 1;
84
85         if (result == true)
86         {
87             pack = udprepack[(urpmain_used) % CONFIG_MAX_REPACK];
88         }
89
90         return result;
91     }
92 }
93
94 }
95

```

```

96 void LakiBeamUDP::on_read(const boost::system::error_code& err, std::
    ↳ size_t read_bytes)
97 {
98     if (!err)
99     {
100         char* temp;
101         if (read_bytes == CONFIG_FRAME)
102         {
103             temp = reinterpret_cast<char*>(&(doublebuffer[dbmain %
    ↳ CONFIG_FRAME_MAX]));
104             memcpy(temp, buffff, CONFIG_FRAME);
105             dbmain++;
106         }
107     }
108     socket->async_receive_from(boost::asio::buffer(buffff, CONFIG_FRAME
    ↳ ), *laser_ep, boost::bind(&LakiBeamUDP::on_read, this, boost
    ↳ ::asio::placeholders::error, boost::asio::placeholders::
    ↳ bytes_transferred));
109 }
110
111 void LakiBeamUDP::udprepack_thread()
112 {
113     while (1)
114     {
115         try
116         {
117             bool goon = true;
118             udp_pack_t doublebuffertemp[2];
119             {
120                 if ((dbmain - dbmain_used) >= 2)
121                 {
122                     char *source1 = reinterpret_cast<char *>(&
    ↳ doublebuffertemp[0]);
123                     char *source2 = reinterpret_cast<char *>(&
    ↳ doublebuffertemp[1]);
124                     char *from1 = reinterpret_cast<char *>(&(
    ↳ doublebuffer[dbmain_used % CONFIG_FRAME_MAX]
    ↳ ));
125                     char *from2 = reinterpret_cast<char *>(&(
    ↳ doublebuffer[(dbmain_used + 1) %
    ↳ CONFIG_FRAME_MAX]));
126                     u32_t realindex = dbmain_used % CONFIG_FRAME_MAX;
127                     if (realindex == CONFIG_FRAME_INDEX_MAX)
128                     {
129                         memcpy(source1, from1, CONFIG_FRAME);
130                         memcpy(source2, from2, CONFIG_FRAME);
131                     }
132                     else
133                     {
134                         memcpy(source1, from1, CONFIG_FRAME * 2);
135                     }
136                     dbmain_used++;
137                 }
138                 else
139                 {
140                     goon = false;
141                 }
142             }

```

```

143
144     if (goon == true)
145     {
146         if ((doublebuffertemp[0].factory == 0x4037) && (
147             ↪ doublebuffertemp[1].factory == 0x4037))
148         {
149             // âřžæŁĖ0ă;DăÑĚèčž
150             u32_t start_index = 0xFFFFFFFF;
151             for (int i = 0; i < CONFIG_UDP_BLOCKS; i++)
152             {
153                 if (doublebuffertemp[0].depths[i].azimuth == 0)
154                 {
155                     start_index = i;
156                 }
157             }
158             // 0ă;DæŤřæDóăÑĚ
159             if (start_index != 0xFFFFFFFF)
160             {
161                 fill_havestart(doublebuffertemp, start_index);
162             }
163             // éĭđ0ă;DæŤřæDóăÑĚ
164             else
165             {
166                 fill_nostart(doublebuffertemp);
167             }
168         }
169         else
170         {
171             //                                DEBUG(doublebuffertemp[1].
172             ↪ factory);
173             //                                DEBUG(doublebuffertemp[0].
174             ↪ factory);
175         }
176     }
177     boost::this_thread::interruption_point();
178     if ((dbmain - dbmain_used) < 2)
179     {
180         boost::this_thread::sleep(boost::posix_time::milliseconds(2));
181     }
182     catch (...)
183     {
184         break;
185     }
186 }
187
188 void LakiBeamUDP::fill_nostart(udp_pack_t pack[2])
189 {
190     (void)pack;
191     u32_t start_time = pack[0].timestamp;
192     u32_t end_time = pack[1].timestamp;
193
194     u16_t tempdots = udprepack[urpmain % CONFIG_MAX_REPACK].maxdots;
195     u32_t div_time = (end_time - start_time) / CONFIG_BLOCK Depths /
196         ↪ CONFIG_UDP_BLOCKS;
197
198     for (int i = 0; i < CONFIG_UDP_BLOCKS; i++)

```

```

197 {
198     if (pack[0].depths[i].head == 0xEEFF)
199     {
200         if (i < (CONFIG_UDP_BLOCKS - 1))
201         {
202             u16_t start_angle = pack[0].depths[i].azimuth;
203             u16_t end_angle = pack[0].depths[i + 1].azimuth;
204             u16_t div_angle;
205             if (end_angle >= start_angle)
206             {
207                 div_angle = (end_angle - start_angle) /
208                     ↪ CONFIG_BLOCK_DEPTHS;
209             }
210             else
211             {
212                 div_angle = (CONFIG_DEGREE_MAX - start_angle +
213                     ↪ end_angle) / CONFIG_BLOCK_DEPTHS;
214             }
215             for (int j = 0; j < CONFIG_BLOCK_DEPTHS; j++)
216             {
217                 udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
218                     ↪ tempdots % CONFIG_CIRCLE_DOTS].timestamp =
219                     ↪ start_time;
220                 udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
221                     ↪ tempdots % CONFIG_CIRCLE_DOTS].angle =
222                     ↪ start_angle;
223                 udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
224                     ↪ tempdots % CONFIG_CIRCLE_DOTS].distance =
225                     ↪ pack->depths[i].depth[j].distance0;
226                 udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
227                     ↪ tempdots % CONFIG_CIRCLE_DOTS].rssi = pack->
228                     ↪ depths[i].depth[j].rssi0;
229                 udprepack[urpmain % CONFIG_MAX_REPACK].interval +=
230                     ↪ div_time;
231                 tempdots++;
232                 if ((start_angle + div_angle) >= CONFIG_DEGREE_MAX)
233                 {
234                     start_angle = start_angle + div_angle -
235                         ↪ CONFIG_DEGREE_MAX;
236                 }
237                 else
238                 {
239                     start_angle += div_angle;
240                 }
241                 start_time += div_time;
242             }
243         }
244     }
245     else
246     {
247         u16_t start_angle = pack[0].depths[i].azimuth;
248         u16_t end_angle = pack[1].depths[0].azimuth;
249         u16_t div_angle;
250         if (end_angle >= start_angle)
251         {
252             div_angle = (end_angle - start_angle) /
253                 ↪ CONFIG_BLOCK_DEPTHS;
254         }
255     }

```



```

242         else
243         {
244             div_angle = (CONFIG_DEGREE_MAX - start_angle +
245                 ↪ end_angle) / CONFIG_BLOCK Depths;
246         }
247         for (int j = 0; j < CONFIG_BLOCK Depths; j++)
248         {
249             udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
250                 ↪ tempdots % CONFIG_CIRCLE_DOTS].timestamp =
251                 ↪ start_time;
252             udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
253                 ↪ tempdots % CONFIG_CIRCLE_DOTS].angle =
254                 ↪ start_angle;
255             udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
256                 ↪ tempdots % CONFIG_CIRCLE_DOTS].distance =
257                 ↪ pack->depths[i].depth[j].distance0;
258             udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
259                 ↪ tempdots % CONFIG_CIRCLE_DOTS].rssi = pack->
260                 ↪ depths[i].depth[j].rssi0;
261             udprepack[urpmain % CONFIG_MAX_REPACK].interval +=
262                 ↪ div_time;
263             tempdots++;
264             if ((start_angle + div_angle) >= CONFIG_DEGREE_MAX)
265             {
266                 start_angle = start_angle + div_angle -
267                     ↪ CONFIG_DEGREE_MAX;
268             }
269             else
270             {
271                 start_angle += div_angle;
272             }
273             start_time += div_time;
274         }
275     }
276 }
277 udprepack[urpmain % CONFIG_MAX_REPACK].maxdots = tempdots;
278 }
279
280 void LakiBeamUDP::fill_havestart(udp_pack_t pack[], u32_t start)
281 {
282     // çñăÿĂăÑĚ
283     if (urpmain == 0)
284     {
285         urpmain++;
286         urpmain_used++;
287
288         udprepack[urpmain % CONFIG_MAX_REPACK].maxdots = 0;
289         udprepack[urpmain % CONFIG_MAX_REPACK].interval = 0;
290
291         (void)pack;
292
293         u32_t start_time = pack[0].timestamp;
294         u32_t end_time = pack[1].timestamp;
295
296         u16_t tempdots = udprepack[urpmain % CONFIG_MAX_REPACK].maxdots
297             ↪ ;
298         u32_t div_time = (end_time - start_time) / CONFIG_BLOCK Depths

```

```

288         ↪ / CONFIG_UDP_BLOCKS;
289
290     for (int i = start; i < CONFIG_UDP_BLOCKS; i++)
291     {
292         if (i < (CONFIG_UDP_BLOCKS - 1))
293         {
294             u16_t start_angle = pack[0].depths[i].azimuth;
295             u16_t end_angle = pack[0].depths[i + 1].azimuth;
296             u16_t div_angle;
297             if (end_angle >= start_angle)
298             {
299                 div_angle = (end_angle - start_angle) /
300                 ↪ CONFIG_BLOCK_DEPTHS;
301             }
302             else
303             {
304                 div_angle = (CONFIG_DEGREE_MAX - start_angle +
305                 ↪ end_angle) / CONFIG_BLOCK_DEPTHS;
306             }
307
308             for (int j = 0; j < CONFIG_BLOCK_DEPTHS; j++)
309             {
310                 udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
311                 ↪ tempdots % CONFIG_CIRCLE_DOTS].timestamp =
312                 ↪ start_time;
313                 udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
314                 ↪ tempdots % CONFIG_CIRCLE_DOTS].angle =
315                 ↪ start_angle;
316                 udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
317                 ↪ tempdots % CONFIG_CIRCLE_DOTS].distance =
318                 ↪ pack->depths[i].depth[j].distance0;
319                 udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
320                 ↪ tempdots % CONFIG_CIRCLE_DOTS].rssi = pack->
321                 ↪ depths[i].depth[j].rssi0;
322                 udprepack[urpmain % CONFIG_MAX_REPACK].interval +=
323                 ↪ div_time;
324                 tempdots++;
325                 if ((start_angle + div_angle) >= CONFIG_DEGREE_MAX)
326                 {
327                     start_angle = start_angle + div_angle -
328                     ↪ CONFIG_DEGREE_MAX;
329                 }
330                 else
331                 {
332                     start_angle += div_angle;
333                 }
334                 start_time += div_time;
335             }
336         }
337     }
338     else
339     {
340         u16_t start_angle = pack[0].depths[i].azimuth;
341         u16_t end_angle = pack[1].depths[0].azimuth;
342         u16_t div_angle;
343         if (end_angle >= start_angle)
344         {
345             div_angle = (end_angle - start_angle) /

```

```

333         ↪ CONFIG_BLOCK_DEPTHS;
334     }
335     else
336     {
337         div_angle = (CONFIG_DEGREE_MAX - start_angle +
338             ↪ end_angle) / CONFIG_BLOCK_DEPTHS;
339     }
340     start_angle = start_angle + div_angle * (start + 1) *
341     ↪ CONFIG_BLOCK_DEPTHS;
342     for (int j = 0; j < CONFIG_BLOCK_DEPTHS; j++)
343     {
344         udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
345             ↪ tempdots % CONFIG_CIRCLE_DOTS].timestamp =
346             ↪ start_time;
347         udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
348             ↪ tempdots % CONFIG_CIRCLE_DOTS].angle =
349             ↪ start_angle;
350         udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
351             ↪ tempdots % CONFIG_CIRCLE_DOTS].distance =
352             ↪ pack->depths[i].depth[j].distance0;
353         udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
354             ↪ tempdots % CONFIG_CIRCLE_DOTS].rssi = pack->
355             ↪ depths[i].depth[j].rssi0;
356         udprepack[urpmain % CONFIG_MAX_REPACK].interval +=
357             ↪ div_time;
358         tempdots++;
359         if ((start_angle + div_angle) >= CONFIG_DEGREE_MAX)
360         {
361             start_angle = start_angle + div_angle -
362             ↪ CONFIG_DEGREE_MAX;
363         }
364         else
365         {
366             start_angle += div_angle;
367         }
368         start_time += div_time;
369     }
370 }
371 udprepack[urpmain % CONFIG_MAX_REPACK].maxdots = tempdots;
372 }
373 // ĀĒŭāōČŌāŇĚ
374 else
375 {
376     {
377         (void)pack;
378         u32_t start_time = pack[0].timestamp;
379         u32_t end_time = pack[1].timestamp;
380
381         u16_t tempdots = udprepack[urpmain % CONFIG_MAX_REPACK].
382             ↪ maxdots;
383         u32_t div_time = (end_time - start_time) /
384             ↪ CONFIG_BLOCK_DEPTHS / CONFIG_UDP_BLOCKS;
385
386         udprepack[urpmain % CONFIG_MAX_REPACK].interval += div_time
387             ↪ ;
388
389         for (u32_t i = 0; i < start; i++)

```

```

375     {
376         u16_t start_angle = pack[0].depths[i].azimuth;
377         u16_t end_angle = pack[0].depths[i + 1].azimuth;
378         u16_t div_angle;
379         if (end_angle >= start_angle)
380         {
381             div_angle = (end_angle - start_angle) /
382                 ↪ CONFIG_BLOCK_DEPTHS;
383         }
384         else
385         {
386             div_angle = (CONFIG_DEGREE_MAX - start_angle +
387                 ↪ end_angle) / CONFIG_BLOCK_DEPTHS;
388         }
389         for (int j = 0; j < CONFIG_BLOCK_DEPTHS; j++)
390         {
391             udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
392                 ↪ tempdots % CONFIG_CIRCLE_DOTS].timestamp =
393                 ↪ start_time;
394             udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
395                 ↪ tempdots % CONFIG_CIRCLE_DOTS].angle =
396                 ↪ start_angle;
397             udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
398                 ↪ tempdots % CONFIG_CIRCLE_DOTS].distance =
399                 ↪ pack->depths[i].depth[j].distance0;
400             udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud[
401                 ↪ tempdots % CONFIG_CIRCLE_DOTS].rssi = pack->
402                 ↪ depths[i].depth[j].rssi0;
403             udprepack[urpmain % CONFIG_MAX_REPACK].interval +=
404                 ↪ div_time;
405             tempdots++;
406             if ((start_angle + div_angle) >= CONFIG_DEGREE_MAX)
407             {
408                 start_angle = start_angle + div_angle -
409                     ↪ CONFIG_DEGREE_MAX;
410             }
411             else
412             {
413                 start_angle += div_angle;
414             }
415             start_time += div_time;
416         }
417     }
418     if((tempdots==1440)|| (tempdots==3600))
419     {
420         bool lost = false;
421         if (tempdots == 1440)
422         {
423             for (int i = 180; i < (1440 - 17); i++)
424             {
425                 if (udprepack[urpmain % CONFIG_MAX_REPACK].
426                     ↪ dotcloud[i+1].angle < udprepack[urpmain %
427                     ↪ CONFIG_MAX_REPACK].dotcloud[i].angle)
428                 {
429                     lost = true;
430                 }
431             }
432         }
433     }

```

```

419     else
420     {
421         for (int i = 450; i < (3600 - 17); i++)
422         {
423             if (udprepack[urpmain % CONFIG_MAX_REPACK].
424                 ↪ dotcloud[i + 1].angle < udprepack[urpmain
425                 ↪ % CONFIG_MAX_REPACK].dotcloud[i].angle)
426             {
427                 lost = true;
428             }
429         }
430     }
431
432     if (lost == false)
433     {
434         udprepack[urpmain % CONFIG_MAX_REPACK].interval /=
435         ↪ tempdots;
436         udprepack[urpmain % CONFIG_MAX_REPACK].maxdots =
437         ↪ tempdots;
438         urpmain++;
439     }
440 }
441 }
442 {
443     udprepack[urpmain % CONFIG_MAX_REPACK].maxdots = 0;
444
445     (void)pack;
446     u32_t start_time = pack[0].timestamp;
447     u32_t end_time = pack[1].timestamp;
448
449     u16_t tempdots = udprepack[urpmain % CONFIG_MAX_REPACK].
450     ↪ maxdots;
451     u32_t div_time = (end_time - start_time) /
452     ↪ CONFIG_BLOCK Depths / CONFIG_UDP_BLOCKS;
453
454     for (u32_t i = start; i < CONFIG_UDP_BLOCKS; i++)
455     {
456         if (i < (CONFIG_UDP_BLOCKS - 1))
457         {
458             u16_t start_angle = pack[0].depths[i].azimuth;
459             u16_t end_angle = pack[0].depths[i + 1].azimuth;
460             u16_t div_angle;
461             if (end_angle >= start_angle)
462             {
463                 div_angle = (end_angle - start_angle) /
464                 ↪ CONFIG_BLOCK Depths;
465             }
466             else
467             {
468                 div_angle = (CONFIG_DEGREE_MAX - start_angle +
469                 ↪ end_angle) / CONFIG_BLOCK Depths;
470             }
471             for (int j = 0; j < CONFIG_BLOCK Depths; j++)
472             {
473                 udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud
474                 ↪ [tempdots % CONFIG_CIRCLE_DOTS].timestamp
475                 ↪ = start_time;
476                 udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud

```

```

467         ↪ [tempdots % CONFIG_CIRCLE_DOTS].angle =
468         ↪ start_angle;
469         udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud
470         ↪ [tempdots % CONFIG_CIRCLE_DOTS].distance
471         ↪ = pack->depths[i].depth[j].distance0;
472         udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud
473         ↪ [tempdots % CONFIG_CIRCLE_DOTS].rsssi =
474         ↪ pack->depths[i].depth[j].rsssi0;
475         udprepack[urpmain % CONFIG_MAX_REPACK].interval
476         ↪ += div_time;
477         tempdots++;
478         if ((start_angle + div_angle) >=
479             ↪ CONFIG_DEGREE_MAX)
480         {
481             start_angle = start_angle + div_angle -
482             ↪ CONFIG_DEGREE_MAX;
483         }
484         else
485         {
486             start_angle += div_angle;
487         }
488         start_time += div_time;
489     }
490 }
491 else
492 {
493     u16_t start_angle = pack[0].depths[i].azimuth;
494     u16_t end_angle = pack[1].depths[0].azimuth;
495     u16_t div_angle;
496     if (end_angle >= start_angle)
497     {
498         div_angle = (end_angle - start_angle) /
499         ↪ CONFIG_BLOCK_DEPTHS;
500     }
501     else
502     {
503         div_angle = (CONFIG_DEGREE_MAX - start_angle +
504         ↪ end_angle) / CONFIG_BLOCK_DEPTHS;
505     }
506     for (u32_t j = 0; j < CONFIG_BLOCK_DEPTHS; j++)
507     {
508         udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud
509         ↪ [tempdots % CONFIG_CIRCLE_DOTS].timestamp
510         ↪ = start_time;
511         udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud
512         ↪ [tempdots % CONFIG_CIRCLE_DOTS].angle =
513         ↪ start_angle;
514         udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud
515         ↪ [tempdots % CONFIG_CIRCLE_DOTS].distance
516         ↪ = pack->depths[i].depth[j].distance0;
517         udprepack[urpmain % CONFIG_MAX_REPACK].dotcloud
518         ↪ [tempdots % CONFIG_CIRCLE_DOTS].rsssi =
519         ↪ pack->depths[i].depth[j].rsssi0;
520         udprepack[urpmain % CONFIG_MAX_REPACK].interval
521         ↪ += div_time;
522         tempdots++;
523         if ((start_angle + div_angle) >=
524             ↪ CONFIG_DEGREE_MAX)

```

```

504         {
505             start_angle = start_angle + div_angle -
                    ↪ CONFIG_DEGREE_MAX;
506         }
507         else
508         {
509             start_angle += div_angle;
510         }
511         start_time += div_time;
512     }
513 }
514 }
515     udprepack[urpmain % CONFIG_MAX_REPACK].maxdots = tempdots;
516 }
517 }
518 }

```

Listing 22: LakiBeamUDP.cpp

High-level Handler:

- Filters points outside valid range or intensity threshold.
- Converts raw units to (angle, distance, intensity) in metric units.
- Provides a thread-safe getter for the latest full scan.
- Optional dump-to-CSV facility for offline SLAM tuning.

```

1  #ifndef LIDAR_HANDLER_HPP
2  #define LIDAR_HANDLER_HPP
3
4  #include "LakiBeamUDP.h"                // â€œ SDK header
5  #include <vector>
6  #include <thread>
7  #include <mutex>
8  #include <atomic>
9
10 /** A single polar sample from the LiDAR */
11 struct LidarPoint {
12     float azimuth;    // deg    (0 â€œ 360)
13     float distance;   // m      (0 = invalid)
14     uint8_t rssi;     // 0 â€œ 255
15 };
16
17 /** Thin wrapper around Richbeam's LakiBeamUDP that:
18  *   â€œ runs the SDK in a background thread
19  *   â€œ converts each completed 360â€œ frame into std::vector<
20      ↪ LidarPoint>
21  *   â€œ offers getLatestScan() (thread-safe copy)
22      ↪ */
23 class LidarHandler {
24 public:
25     explicit LidarHandler(const std::string& local_ip   = "0.0.0.0",
26                           const std::string& local_port = "2368",
27                           const std::string& laser_ip   = "
                    ↪ 192.168.198.2",
28                           const std::string& laser_port = "2368");
29     ~LidarHandler();

```

```

28
29  /** Non-blocking; returns an empty vector if no new scan is ready.
    ↪ */
30  std::vector<LidarPoint> getLatestScan();
31  void dumpNextScan(const std::string &filename, std::vector<
    ↪ LidarPoint> &scan);
32
33 private:
34     void worker(); // background thread
35
36     std::unique_ptr<LakiBeamUDP> driver_;
37     std::thread th_;
38     std::mutex mtx_;
39     std::vector<LidarPoint> latest_;
40     std::atomic<bool> running_{true};
41 };
42
43 #endif // LIDAR_HANDLER_HPP

```

Listing 23: lidar_handler.hpp

```

1  #include "lidar_handler.hpp"
2  #include <chrono>
3  #include <cmath>
4  #include <fstream>
5  #include <algorithm>
6
7  using namespace std::chrono_literals;
8
9  // helpers to convert SDK raw units
10 static inline float mm2m(uint16_t mm) { return mm * 0.001f; }
11 static inline float hunDeg2deg(uint16_t hd) { return hd * 0.01f; }
12
13 LidarHandler::LidarHandler(const std::string& local_ip,
14                             const std::string& local_port,
15                             const std::string& laser_ip,
16                             const std::string& laser_port)
17 {
18     driver_ = std::make_unique<LakiBeamUDP>(local_ip, local_port,
19                                              laser_ip, laser_port);
20     th_ = std::thread(&LidarHandler::worker, this);
21 }
22
23 LidarHandler::~LidarHandler()
24 {
25     running_ = false;
26     if (th_.joinable()) th_.join();
27 }
28
29 void LidarHandler::dumpNextScan(const std::string &filename, std::
    ↪ vector<LidarPoint> &scan)
30 {
31     // 1) grab the latest scan
32     scan = getLatestScan();
33     if (scan.empty()) return;
34
35     // 2) sort by azimuth
36     std::sort(scan.begin(), scan.end(),

```



```

37         [](auto &a, auto &b){ return a.azimuth < b.azimuth; });
38
39     // 3) open file in append mode
40     std::ofstream ofs(filename, std::ios::app);
41     if (!ofs.is_open()) return; // could log an error
42
43     // 4) write distances, space-separated, one line per scan
44     for (size_t i = 0; i < scan.size(); ++i) {
45         ofs << scan[i].distance
46             << (i + 1 < scan.size() ? ' ' : '\n');
47     }
48     // file closed automatically when ofs goes out of scope
49 }
50
51 void LidarHandler::worker()
52 {
53     repack_t frame; // SDK struct
54     while (running_)
55     {
56         if (driver_ -> get_repackedpack(frame)) // = one 360° scan
57         {
58             std::vector<LidarPoint> tmp;
59             tmp.reserve(frame.maxdots);
60
61             for (uint16_t i = 0; i < frame.maxdots; ++i)
62             {
63                 const auto& d = frame.dotcloud[i];
64                 if (d.distance == 0) continue; // invalid / out-of-
65                 ↪ range
66
67                 tmp.push_back({ hunDeg2deg(d.angle),
68                               mm2m(d.distance),
69                               d.rssi });
70
71                 // swap into shared buffer
72                 std::lock_guard<std::mutex> lk(mtx_);
73                 latest_.swap(tmp);
74             }
75
76             std::this_thread::sleep_for(2ms); // avoid busy spin
77         }
78     }
79
80     std::vector<LidarPoint> LidarHandler::getLatestScan()
81     {
82         std::lock_guard<std::mutex> lk(mtx_);
83         return latest_; // copy (small:
84         ↪ 3600 pts)
85     }

```

Listing 24: lidar_handler.cpp

3.3.2 Serial-Stream Sensor Parser

Even beyond LiDAR, the MCU streams wheel encoder deltas and IMU samples over UART. Our serial parser decodes and calibrates these:

- Converts raw ticks $\rightarrow$ meters and gyro counts $\rightarrow$ rad/s.
- Applies bias removal (zero-offset for stationary detection).
- Flags stale or repeated packets to prevent filter divergence.

3.4 Localization & Sensor Fusion

3.4.1 Extended Kalman Filter (EKF)

Role & Design

Fusing encoders, gyro, accelerometer, and optionally LiDAR odometry yields a robust 5D state $(x, y, \theta, v, \omega)$. We use a discrete EKF with differential-drive kinematics.

- **Prediction Step:** Motion model integrates wheel-based forward velocity and yaw rate to predict pose. Covariance propagation uses process noise terms.
- **Update Step:** Incorporates gyro rate and accelerometer heading with respective measurement noise. Encoder-based velocity measurements correct drift in linear speed. Zero-Velocity Update (ZUPT) disables velocity updates when sensors indicate full stop, collapsing velocity uncertainty.
- **Covariance Management:** Tuning knobs in `config.hpp` trade responsiveness vs. noise rejection. If a sensor dropout is detected (e.g. LiDAR packet loss exceeds threshold), that measurement's Kalman gain is forced to zero.

```

1 #ifndef SENSOR_FUSION_HPP
2 #define SENSOR_FUSION_HPP
3
4 #include <array>
5 #include <vector>
6 #include <Eigen/Dense>
7
8 class SensorFusion {
9 public:
10     // Constructor
11     SensorFusion(double wheelRadius,
12                 double wheelBase,
13                 double dt = 0.01, // Default 100
14                               ↪ Hz (10ms)
15                 double processNoisePos = 1e-4,
16                 double processNoiseAng = 1e-5,
17                 double processNoiseVel = 1e-3,
18                 double gyroNoise = 1e-4,
19                 double imuNoise = 1e-3,
20                 double accelNoise = 1e-2);
21
22     // Prediction step using encoder data
23     void predict(double leftVelocity, double rightVelocity);
24     void predictWithZUPT(double leftVelocity, double rightVelocity,
25                          ↪ bool isStationary);
26
27     // Comprehensive sensor fusion step (recommended usage)
28     void sensorFusionStep(double leftVel, double rightVel,
29                          double gyroZ = 0.0, bool hasGyro = false,
30                          double imuYaw = 0.0, bool hasIMU = false,

```

```

29         double accelX = 0.0, double accelY = 0.0, bool
30             ↪ hasAccel = false,
31             bool hasEncoders = true);
32
33     // Update steps with different sensors
34     void updateWithGyro(double gyroZ);
35     void updateWithIMU(double imuYaw);
36     void updateWithAccelerometer(double accelX, double accelY); // 2D
37     ↪ accelerometer for ZUPT
38     void updateWithEncoders(double leftVel, double rightVel);
39     void updateWithEncodersEnhanced(double leftVel, double rightVel,
40                                     double accelX, double accelY); //
41     ↪ With slip detection
42
43     // Get current state estimates
44     std::array<double, 3> getPose() const; // [x, y, theta]
45     std::array<double, 3> getVelocities() const; // [vx, vy, omega]
46     Eigen::VectorXd getFullState() const; // Full 6D state
47     Eigen::MatrixXd getCovariance() const; // Covariance
48     ↪ matrix
49
50     // Get specific state components
51     double getX() const { return state_(0); }
52     double getY() const { return state_(1); }
53     double getTheta() const { return state_(2); }
54     double getVx() const { return state_(3); }
55     double getVy() const { return state_(4); }
56     double getOmega() const { return state_(5); }
57
58     // Utility functions
59     void printState() const;
60     void printCovariance() const;
61     void printDiagnostics() const; // Comprehensive debugging output
62     void resetState();
63     bool isInitialized() const { return initialized_; }
64
65 private:
66     // Robot parameters
67     double wheelRadius_;
68     double wheelBase_;
69     double dt_;
70
71     // State vector: [x, y, theta, vx, vy, omega] (6D)
72     Eigen::VectorXd state_; // State estimate
73     Eigen::MatrixXd P_; // Covariance matrix
74
75     // Noise matrices
76     Eigen::MatrixXd Q_; // Process noise
77     Eigen::MatrixXd R_gyro_; // Gyro measurement noise
78     Eigen::MatrixXd R_imu_; // IMU measurement noise
79     Eigen::MatrixXd R_accel_; // Accelerometer noise
80     Eigen::MatrixXd R_encoder_; // Encoder noise
81
82     // State tracking
83     bool initialized_;
84
85     // Motion model functions
86     Eigen::VectorXd motionModel(const Eigen::VectorXd& state,

```



```

29     ↪ uncertainties
30     P_ = Eigen::MatrixXd::Zero(6, 6);
31     P_(0,0) = 0.1;    // x position uncertainty: 10cm
32     P_(1,1) = 0.1;    // y position uncertainty: 10cm
33     P_(2,2) = 0.1;    // theta uncertainty: ~6 degrees
34     P_(3,3) = 0.01;   // vx velocity uncertainty: 10cm/s
35     P_(4,4) = 0.01;   // vy velocity uncertainty: 10cm/s
36     P_(5,5) = 0.01;   // omega uncertainty: ~0.6 deg/s
37
38     // Process noise matrix Q
39     Q_ = Eigen::MatrixXd::Zero(6, 6);
40     Q_(0,0) = processNoisePos; // x position noise
41     Q_(1,1) = processNoisePos; // y position noise
42     Q_(2,2) = processNoiseAng; // theta noise
43     Q_(3,3) = processNoiseVel; // vx noise
44     Q_(4,4) = processNoiseVel; // vy noise
45     Q_(5,5) = processNoiseAng; // omega noise
46
47     // Measurement noise matrices
48     R_gyro_ = Eigen::MatrixXd::Identity(1, 1) * gyroNoise;
49     R_imu_ = Eigen::MatrixXd::Identity(1, 1) * imuNoise;
50     R_accel_ = Eigen::MatrixXd::Identity(2, 2) * accelNoise;
51     R_encoder_ = Eigen::MatrixXd::Identity(2, 2);
52     R_encoder_(0,0) = processNoiseVel; // Forward velocity noise
53     R_encoder_(1,1) = processNoiseAng; // Angular velocity noise
54 }
55
56 void SensorFusion::predict(double leftVelocity, double rightVelocity) {
57     if (!initialized_) {
58         initialized_ = true;
59         return;
60     }
61
62     // Apply motion model
63     Eigen::VectorXd predicted_state = motionModel(state_, leftVelocity,
64         ↪ rightVelocity);
65
66     // Get motion Jacobian
67     Eigen::MatrixXd F = getMotionJacobian(state_, leftVelocity,
68         ↪ rightVelocity);
69
70     // Predict covariance
71     Eigen::MatrixXd predicted_P = F * P_ * F.transpose() + Q_;
72
73     // Update state and covariance
74     state_ = wrapStateToPi(predicted_state);
75     P_ = predicted_P;
76 }
77
78 void SensorFusion::predictWithZUPT(double leftVelocity, double
79     ↪ rightVelocity, bool isStationary) {
80     if (!initialized_) {
81         initialized_ = true;
82         return;
83     }
84
85     // If stationary (ZUPT active), ignore encoder velocities and use
86     ↪ current velocities

```

```

82     double effectiveLeftVel = isStationary ? 0.0 : leftVelocity;
83     double effectiveRightVel = isStationary ? 0.0 : rightVelocity;
84
85     // Apply motion model with potentially zeroed encoder inputs
86     Eigen::VectorXd predicted_state = motionModel(state_,
87         ↪ effectiveLeftVel, effectiveRightVel);
88
89     // Get motion Jacobian
90     Eigen::MatrixXd F = getMotionJacobian(state_, effectiveLeftVel,
91         ↪ effectiveRightVel);
92
93     // Predict covariance
94     Eigen::MatrixXd predicted_P = F * P_ * F.transpose() + Q_;
95
96     // Update state and covariance
97     state_ = wrapStateToPi(predicted_state);
98     P_ = predicted_P;
99 }
100
101 Eigen::VectorXd SensorFusion::motionModel(const Eigen::VectorXd& state,
102     double vLeft, double vRight)
103     ↪ const {
104
105     double x = state(0);
106     double y = state(1);
107     double theta = state(2);
108     double vx = state(3);
109     double vy = state(4);
110     double omega = state(5);
111
112     // Calculate robot velocities from wheel velocities
113     double v_robot = (vLeft + vRight) / 2.0;
114     double omega_robot = (vRight - vLeft) / wheelBase_;
115
116     // Predict new state
117     Eigen::VectorXd new_state(6);
118
119     // Position update (integrate velocity in robot frame to global
120     ↪ frame)
121     // Transform robot velocities to global frame and integrate
122     double v_global_x = vx * cos(theta) - vy * sin(theta);
123     double v_global_y = vx * sin(theta) + vy * cos(theta);
124
125     new_state(0) = x + v_global_x * dt_;
126     new_state(1) = y + v_global_y * dt_;
127     new_state(2) = theta + omega * dt_;
128
129     // Velocity update (based on wheel commands with some smoothing)
130     double alpha = 0.8; // Smoothing factor
131     new_state(3) = alpha * vx + (1.0 - alpha) * v_robot; //
132     ↪ Forward velocity
133     new_state(4) = alpha * vy; //
134     ↪ Lateral velocity (decay)
135     new_state(5) = alpha * omega + (1.0 - alpha) * omega_robot; //
136     ↪ Angular velocity
137
138     return new_state;
139 }

```

```

133 Eigen::MatrixXd SensorFusion::getMotionJacobian(const Eigen::VectorXd&
    ↪ state,
134
    double vLeft, double
    ↪ vRight) const {
135     double theta = state(2);
136     double vx = state(3);
137     double vy = state(4);
138
139     Eigen::MatrixXd F = Eigen::MatrixXd::Identity(6, 6);
140
141     // Position derivatives
142     F(0,2) = -(vx * sin(theta) + vy * cos(theta)) * dt_; // dx/dtheta
143     F(0,3) = cos(theta) * dt_; // dx/dvx
144     F(0,4) = -sin(theta) * dt_; // dx/dvy
145
146     F(1,2) = (vx * cos(theta) - vy * sin(theta)) * dt_; // dy/dtheta
147     F(1,3) = sin(theta) * dt_; // dy/dvx
148     F(1,4) = cos(theta) * dt_; // dy/dvy
149
150     F(2,5) = dt_; // dtheta/
    ↪ domega
151
152     // Velocity smoothing
153     double alpha = 0.8;
154     F(3,3) = alpha; // dvx/dvx
155     F(4,4) = alpha; // dvy/dvy
156     F(5,5) = alpha; // domega/domega
157
158     return F;
159 }
160
161 void SensorFusion::updateWithGyro(double gyroZ) {
162     // Measurement model: z = omega (angular velocity)
163     double predicted = gyroMeasurementModel(state_);
164     double innovation = gyroZ - predicted;
165
166     // Measurement Jacobian
167     Eigen::MatrixXd H = gyroMeasurementJacobian();
168
169     // Innovation covariance
170     Eigen::MatrixXd S = H * P_ * H.transpose() + R_gyro_;
171
172     // Kalman gain with regularization to avoid singular matrices
173     if (S.determinant() < 1e-10) {
174         S += Eigen::MatrixXd::Identity(1, 1) * 1e-6; // Add small
    ↪ regularization
175     }
176     Eigen::MatrixXd K = P_ * H.transpose() * S.inverse();
177
178     // Update state and covariance using Joseph form for numerical
    ↪ stability
179     Eigen::MatrixXd I_KH = Eigen::MatrixXd::Identity(6, 6) - K * H;
180     state_ = state_ + K * innovation;
181     P_ = I_KH * P_ * I_KH.transpose() + K * R_gyro_ * K.transpose();
182
183     state_ = wrapStateToPi(state_);
184 }
185

```

```

186 double SensorFusion::gyroMeasurementModel(const Eigen::VectorXd& state)
    ↪ const {
187     return state(5); // Return omega (angular velocity)
188 }
189
190 Eigen::MatrixXd SensorFusion::gyroMeasurementJacobian() const {
191     Eigen::MatrixXd H = Eigen::MatrixXd::Zero(1, 6);
192     H(0,5) = 1.0; // dh/domega = 1
193     return H;
194 }
195
196 void SensorFusion::updateWithIMU(double imuYaw) {
197     // Measurement model: z = theta (orientation)
198     double predicted = imuMeasurementModel(state_);
199     double innovation = wrapToPi(imuYaw - predicted);
200
201     // Measurement Jacobian
202     Eigen::MatrixXd H = imuMeasurementJacobian();
203
204     // Innovation covariance
205     Eigen::MatrixXd S = H * P_ * H.transpose() + R_imu_;
206
207     // Kalman gain with regularization to avoid singular matrices
208     if (S.determinant() < 1e-10) {
209         S += Eigen::MatrixXd::Identity(1, 1) * 1e-6; // Add small
            ↪ regularization
210     }
211     Eigen::MatrixXd K = P_ * H.transpose() * S.inverse();
212
213     // Update state and covariance using Joseph form for numerical
            ↪ stability
214     Eigen::MatrixXd I_KH = Eigen::MatrixXd::Identity(6, 6) - K * H;
215     Eigen::VectorXd state_update = K * innovation;
216     state_ = state_ + state_update;
217     P_ = I_KH * P_ * I_KH.transpose() + K * R_imu_ * K.transpose();
218
219     state_ = wrapStateToPi(state_);
220 }
221
222 double SensorFusion::imuMeasurementModel(const Eigen::VectorXd& state)
    ↪ const {
223     return state(2); // Return theta (orientation)
224 }
225
226 Eigen::MatrixXd SensorFusion::imuMeasurementJacobian() const {
227     Eigen::MatrixXd H = Eigen::MatrixXd::Zero(1, 6);
228     H(0,2) = 1.0; // dh/dtheta = 1
229     return H;
230 }
231
232 void SensorFusion::updateWithAccelerometer(double accelX, double accelY
    ↪ ) {
233     // Use accelerometer for Zero Velocity Update (ZUPT) in 2D
234     // If total acceleration is very small, assume robot is stationary
235     double accel_magnitude = std::sqrt(accelX*accelX + accelY*accelY);
236
237     // Apply ZUPT if acceleration is very small (robot not accelerating
            ↪ much)

```



```

238 // Lowered threshold to be more sensitive to stationary conditions
239 if (accel_magnitude < 0.05) { // Less than 0.05 m/s2 indicates
    ↪ near-stationary
240 // Measurement: velocities should be zero
241 Eigen::Vector2d z_measured = Eigen::Vector2d::Zero();
242 Eigen::Vector2d z_predicted;
243 z_predicted(0) = state_(3); // vx
244 z_predicted(1) = state_(4); // vy
245
246 Eigen::Vector2d innovation = z_measured - z_predicted;
247
248 // Measurement Jacobian for velocity
249 Eigen::MatrixXd H = Eigen::MatrixXd::Zero(2, 6);
250 H(0,3) = 1.0; // dvx/dvx = 1
251 H(1,4) = 1.0; // dvy/dvy = 1
252
253 // Use very low noise for ZUPT - we're confident about zero
    ↪ velocity
254 Eigen::MatrixXd R_zupt = Eigen::MatrixXd::Identity(2, 2) * 1e
    ↪ -4; // Very confident in ZUPT
255
256 // Innovation covariance
257 Eigen::MatrixXd S = H * P_ * H.transpose() + R_zupt;
258
259 // Kalman gain with regularization to avoid singular matrices
260 if (S.determinant() < 1e-10) {
261     S += Eigen::MatrixXd::Identity(2, 2) * 1e-6; // Add small
    ↪ regularization
262 }
263 Eigen::MatrixXd K = P_ * H.transpose() * S.inverse();
264
265 // Update - force velocities towards zero using Joseph form
266 Eigen::MatrixXd I_KH = Eigen::MatrixXd::Identity(6, 6) - K * H;
267 state_ = state_ + K * innovation;
268 P_ = I_KH * P_ * I_KH.transpose() + K * R_zupt * K.transpose();
269
270 // Additional: Directly constrain velocities for more
    ↪ aggressive ZUPT
271 // This ensures that when ZUPT is active, velocities are truly
    ↪ forced to near-zero
272 state_(3) = state_(3) * 0.1; // Heavily damp vx
273 state_(4) = state_(4) * 0.1; // Heavily damp vy
274 state_(5) = state_(5) * 0.1; // Heavily damp omega
275 }
276 }
277
278 void SensorFusion::updateWithEncoders(double leftVel, double rightVel)
    ↪ {
279 // Measurement model: velocities derived from encoders
280 Eigen::Vector2d z_measured;
281 z_measured(0) = (leftVel + rightVel) / 2.0; // Forward
    ↪ velocity
282 z_measured(1) = (rightVel - leftVel) / wheelBase_; // Angular
    ↪ velocity
283
284 Eigen::Vector2d z_predicted = encoderMeasurementModel(state_);
285 Eigen::Vector2d innovation = z_measured - z_predicted;
286

```

```

287 // Measurement Jacobian
288 Eigen::MatrixXd H = encoderMeasurementJacobian();
289
290 // Innovation covariance
291 Eigen::MatrixXd S = H * P_ * H.transpose() + R_encoder_;
292
293 // Kalman gain with regularization to avoid singular matrices
294 if (S.determinant() < 1e-10) {
295     S += Eigen::MatrixXd::Identity(2, 2) * 1e-6; // Add small
296         ↪ regularization
297 }
298 Eigen::MatrixXd K = P_ * H.transpose() * S.inverse();
299
300 // Update using Joseph form for numerical stability
301 Eigen::MatrixXd I_KH = Eigen::MatrixXd::Identity(6, 6) - K * H;
302 state_ = state_ + K * innovation;
303 P_ = I_KH * P_ * I_KH.transpose() + K * R_encoder_ * K.transpose();
304
305 // Ensure angles are properly wrapped after update
306 state_ = wrapStateToPi(state_);
307 }
308
309 void SensorFusion::updateWithEncodersEnhanced(double leftVel, double
310     ↪ rightVel,
311
312     double accelX, double
313     ↪ accelY) {
314
315     // Detect wheel slip/lift conditions before using encoder data
316     if (detectWheelSlipOrLift(leftVel, rightVel, accelX, accelY)) {
317         // Reduce trust in encoder measurements by increasing noise
318         Eigen::MatrixXd R_reduced_trust = R_encoder_ * 10.0; // 10x
319             ↪ higher noise
320
321         // Still update but with much lower confidence
322         Eigen::Vector2d z_measured;
323         z_measured(0) = (leftVel + rightVel) / 2.0; //
324             ↪ Forward velocity
325         z_measured(1) = (rightVel - leftVel) / wheelBase_; //
326             ↪ Angular velocity
327
328         Eigen::Vector2d z_predicted = encoderMeasurementModel(state_);
329         Eigen::Vector2d innovation = z_measured - z_predicted;
330
331         // Measurement Jacobian
332         Eigen::MatrixXd H = encoderMeasurementJacobian();
333
334         // Innovation covariance with increased noise
335         Eigen::MatrixXd S = H * P_ * H.transpose() + R_reduced_trust;
336
337         // Regularization to avoid singular matrices
338         if (S.determinant() < 1e-10) {
339             S += Eigen::MatrixXd::Identity(2, 2) * 1e-6;
340         }
341         Eigen::MatrixXd K = P_ * H.transpose() * S.inverse();
342
343         // Update state and covariance using Joseph form for numerical
344             ↪ stability
345         Eigen::MatrixXd I_KH = Eigen::MatrixXd::Identity(6, 6) - K * H;
346         state_ = state_ + K * innovation;
347

```

```

338     P_ = I_KH * P_ * I_KH.transpose() + K * R_reduced_trust * K.
        ↪ transpose();
339
340 } else {
341     // Normal encoder update with full trust
342     updateWithEncoders(leftVel, rightVel);
343 }
344
345 // Ensure angles are properly wrapped after update
346 state_ = wrapStateToPi(state_);
347 }
348
349 bool SensorFusion::detectWheelSlipOrLift(double leftVel, double
        ↪ rightVel,
350                                         double accelX, double accelY)
        ↪ const {
351     // Calculate expected acceleration from encoder velocities
352     double v_robot = (leftVel + rightVel) / 2.0;
353     double omega_robot = (rightVel - leftVel) / wheelBase_;
354
355     // Estimate expected acceleration in robot frame
356     double expected_accel_forward = (v_robot - state_(3)) / dt_; //
        ↪ Change in forward velocity
357     double expected_accel_lateral = -state_(3) * omega_robot; //
        ↪ Centripetal acceleration
358
359     // Transform to global frame for comparison with IMU
360     double cos_theta = std::cos(state_(2));
361     double sin_theta = std::sin(state_(2));
362     double expected_accel_x = expected_accel_forward * cos_theta -
        ↪ expected_accel_lateral * sin_theta;
363     double expected_accel_y = expected_accel_forward * sin_theta +
        ↪ expected_accel_lateral * cos_theta;
364
365     // Calculate acceleration discrepancy
366     double accel_diff_x = std::abs(accelX - expected_accel_x);
367     double accel_diff_y = std::abs(accelY - expected_accel_y);
368     double total_accel_error = std::sqrt(accel_diff_x*accel_diff_x +
        ↪ accel_diff_y*accel_diff_y);
369
370     // Thresholds for slip/lift detection
371     double accel_error_threshold = 0.5; // m/s2 - significant
        ↪ discrepancy
372     double velocity_unreasonable_threshold = 3.0; // m/s -
        ↪ unreasonably high velocities
373
374     // Detect slip/lift conditions
375     bool high_accel_error = (total_accel_error > accel_error_threshold)
        ↪ ;
376     bool unreasonable_velocities = (std::abs(leftVel) >
        ↪ velocity_unreasonable_threshold ||
377                                     std::abs(rightVel) >
        ↪ velocity_unreasonable_threshold
        ↪ );
378     bool velocity_mismatch = (std::abs(leftVel - rightVel) > 2.0 * std
        ↪ ::abs(state_(5)) * wheelBase_);
379
380     return (high_accel_error || unreasonable_velocities ||

```

```

    ↪ velocity_mismatch);
381 }
382
383 double SensorFusion::wrapToPi(double angle) const {
384     while (angle > M_PI) angle -= 2.0 * M_PI;
385     while (angle < -M_PI) angle += 2.0 * M_PI;
386     return angle;
387 }
388
389 Eigen::VectorXd SensorFusion::wrapStateToPi(const Eigen::VectorXd&
    ↪ state) const {
390     Eigen::VectorXd wrapped_state = state;
391     wrapped_state(2) = wrapToPi(state(2)); // Wrap theta
392     return wrapped_state;
393 }
394
395 std::array<double, 3> SensorFusion::getPose() const {
396     return {state_(0), state_(1), state_(2)};
397 }
398
399 std::array<double, 3> SensorFusion::getVelocities() const {
400     return {state_(3), state_(4), state_(5)};
401 }
402
403 Eigen::VectorXd SensorFusion::getFullState() const {
404     return state_;
405 }
406
407 Eigen::MatrixXd SensorFusion::getCovariance() const {
408     return P_;
409 }
410
411 void SensorFusion::printStats() const {
412     std::cout << std::fixed << std::setprecision(4)
413         << "EKF STATE -> x: " << state_(0) << " m, y: " << state_
    ↪ (1)
414         << " m,  $\dot{y}$ : " << state_(2) << " rad (" << (state_(2) *
    ↪ 180.0 / M_PI) << "  $\dot{r}$ )" << std::endl;
415     std::cout << "VELOCITIES -> vx: " << state_(3) << " m/s, vy: " <<
    ↪ state_(4)
416         << " m/s,  $\dot{L}$ : " << state_(5) << " rad/s (" << (state_(5)
    ↪ * 180.0 / M_PI) << "  $\dot{r}$ /s)" << std::endl;
417 }
418
419 void SensorFusion::printCovariance() const {
420     std::cout << "\n=== EKF COVARIANCE MATRIX ===\n";
421     std::cout << "Diagonal elements (uncertainties):\n";
422     std::cout << std::fixed << std::setprecision(6);
423     std::cout << "  $\hat{C}_x$  = " << P_(0,0) << " m $\hat{s}$ \n";
424     std::cout << "  $\hat{C}_y$  = " << P_(1,1) << " m $\hat{s}$ \n";
425     std::cout << "  $\hat{C}_{\dot{y}}$  = " << P_(2,2) << " rad $\hat{s}$ \n";
426     std::cout << "  $\hat{C}_{vx}$  = " << P_(3,3) << " (m/s) $\hat{s}$ \n";
427     std::cout << "  $\hat{C}_{vy}$  = " << P_(4,4) << " (m/s) $\hat{s}$ \n";
428     std::cout << "  $\hat{C}_{\dot{L}}$  = " << P_(5,5) << " (rad/s) $\hat{s}$ \n";
429
430     std::cout << "\nStandard deviations:\n";
431     std::cout << "  $\hat{C}_x$  = " << std::sqrt(P_(0,0)) << " m\n";
432     std::cout << "  $\hat{C}_y$  = " << std::sqrt(P_(1,1)) << " m\n";

```



```

517         updateWithEncodersEnhanced(leftVel, rightVel, accelX,
518             ↪ accelY);
519     } else {
520         updateWithEncoders(leftVel, rightVel); // Fallback to
521             ↪ normal update
522     }
523 }
524
525 if (hasIMU) {
526     updateWithIMU(imuYaw);
527 }
528
529 if (hasAccel) {
530     updateWithAccelerometer(accelX, accelY);
531 }
532
533 // Ensure final angle wrapping after all updates
534 state_ = wrapStateToPi(state_);
535 }
536
537 // ===== Missing Encoder Measurement Functions =====
538 Eigen::Vector2d SensorFusion::encoderMeasurementModel(const Eigen::
539     ↪ VectorXd& state) const {
540     // Extract velocities from state [x, y, theta, vx, vy, omega]
541     double vx = state(3);
542     double vy = state(4);
543     double omega = state(5);
544     double theta = state(2);
545
546     // Convert from robot velocity to wheel velocities
547     // Forward velocity in robot frame
548     double v_forward = vx * cos(theta) + vy * sin(theta);
549
550     // Calculate expected wheel velocities
551     double leftVel = v_forward - (wheelBase_ / 2.0) * omega;
552     double rightVel = v_forward + (wheelBase_ / 2.0) * omega;
553
554     Eigen::Vector2d z_predicted;
555     z_predicted << leftVel, rightVel;
556     return z_predicted;
557 }
558
559 Eigen::MatrixXd SensorFusion::encoderMeasurementJacobian() const {
560     // Jacobian matrix for encoder measurements
561     // State: [x, y, theta, vx, vy, omega]
562     // Measurement: [leftVel, rightVel]
563
564     Eigen::MatrixXd H = Eigen::MatrixXd::Zero(2, 6);
565
566     double theta = state_(2);
567     double cos_theta = cos(theta);
568     double sin_theta = sin(theta);
569     double vx = state_(3);
570     double vy = state_(4);
571
572     // Partial derivatives for left wheel velocity
573     H(0, 2) = -vx * sin_theta + vy * cos_theta; // d/dtheta
574     H(0, 3) = cos_theta; // d/dvx

```

```

572     H(0, 4) = sin_theta;                                // d/dvy
573     H(0, 5) = -wheelBase_ / 2.0;                       // d/domega
574
575     // Partial derivatives for right wheel velocity
576     H(1, 2) = -vx * sin_theta + vy * cos_theta;        // d/dtheta
577     H(1, 3) = cos_theta;                                // d/dvx
578     H(1, 4) = sin_theta;                                // d/dvy
579     H(1, 5) = wheelBase_ / 2.0;                         // d/domega
580
581     return H;
582 }

```

Listing 26: SensorFusion.cpp

3.4.2 Localization Orchestration

Role & Design

RobotLocalization.cpp wraps the EKF:

- Initializes filter with initial pose and uncertainty.
- Runs a 100 Hz loop that calls predict/update in lockstep with sensor acquisition.
- Detects sensor failure- switches to dead-reckoning with covariance inflation.
- Logs timestamped state and covariance diagonals to CSV if LOG_ENABLE.

```

1  #pragma once
2  #include "SensorFusion.hpp"
3  #include "../communication/serial_com.hpp"
4  #include "robot_utils.hpp"
5  #include "../lidar/lidar_handler.hpp"
6  #include <fstream>
7  #include <chrono>
8  #include <array>
9  #include <string>
10
11 class RobotLocalization {
12 private:
13     // default loop period = 10 ms
14     static constexpr double DEFAULT_DT = 0.01;
15
16     std::chrono::steady_clock::time_point lastUpdate_;
17     double dt_;
18
19     std::ofstream logFile_;
20     bool enableLogging_;
21
22 public:
23     RobotLocalization(bool enableLogging = true);
24     ~RobotLocalization();
25
26     void updateEKF(const SensorPacket& sensor);
27     void printStatus(const SensorPacket& sensor);
28     void logData    (const SensorPacket& sensor);
29
30     std::array<double,3> getPose() const;

```



```

31     std::array<double,3> getVelocities() const;
32
33 private:
34     std::string getCurrentTimestamp();
35 };

```

Listing 27: RobotLocalization.hpp

```

1  #include "RobotLocalization.hpp"
2  #include <cmath>
3
4  // use your global EKF instance
5  extern SensorFusion g_ekf;
6
7  RobotLocalization::RobotLocalization(bool enableLogging)
8      : dt_(DEFAULT_DT), // <-- use the class's own
9        ↪ DEFAULT_DT
10      enableLogging_(enableLogging)
11  {
12      lastUpdate_ = std::chrono::steady_clock::now();
13      if (enableLogging_) {
14          auto ts = getCurrentTimestamp();
15          logFile_.open("robot_pose_" + ts + ".csv");
16          logFile_ << "timestamp,x,y,theta,vx,vy,omega,leftVel,rightVel,
17                      ↪ imuYaw,gyroZ,accelX,accelY\n";
18      }
19  }
20
21  RobotLocalization::~RobotLocalization() {
22      if (logFile_.is_open())
23          logFile_.close();
24  }
25
26  void RobotLocalization::updateEKF(const SensorPacket& sensor) {
27      // compute loop dt
28      auto now = std::chrono::steady_clock::now();
29      double delta = std::chrono::duration<double>(now - lastUpdate_).
30          ↪ count();
31      lastUpdate_ = now;
32      if (delta > 0.1) delta = dt_; // reset if too large
33      else if (delta < 0.001) delta = 0.001; // floor
34      dt_ = delta;
35
36      // wheel velocities
37      auto [leftVel, rightVel] = RobotUtils::getWheelVelocities(sensor,
38          ↪ dt_);
39
40      // ZUPT condition
41      double accelMag = std::sqrt(sensor.accelX*sensor.accelX + sensor.
42          ↪ accelY*sensor.accelY);
43      bool lowMotion = (std::abs(leftVel) < 0.05 && std::abs(
44          ↪ rightVel) < 0.05);
45      bool reasonableAccel = (accelMag >= 0.0 && accelMag < 0.5);
46      bool zuptActive = (lowMotion && reasonableAccel && accelMag <
47          ↪ 0.05);
48
49      // 1) predict
50      g_ekf.predictWithZUPT(leftVel, rightVel, zuptActive);

```

```

44
45 // 2) IMU yaw
46 if (sensor.yaw != 0.0f && std::abs(sensor.yaw) < 360.0f)
47     g_ekf.updateWithIMU(RobotUtils::degToRad(sensor.yaw));
48
49 // 3) gyro Z
50 if (sensor.gyroZ != 0.0f && std::abs(sensor.gyroZ) < 10.0)
51     g_ekf.updateWithGyro(sensor.gyroZ);
52
53 // 4) ZUPT accelerometer
54 if (zuptActive)
55     g_ekf.updateWithAccelerometer(sensor.accelX, sensor.accelY);
56
57 // 5) encoder update
58 bool encOk = (std::abs(leftVel) < 2.0 && std::abs(rightVel) < 2.0);
59 if (encOk && !zuptActive && (std::abs(leftVel)>0.01 || std::abs(
    ↪ rightVel)>0.01))
60     g_ekf.updateWithEncoders(leftVel, rightVel);
61 }
62
63 void RobotLocalization::printStatus(const SensorPacket& sensor) {
64     auto pose = g_ekf.getPose();
65     auto vel = g_ekf.getVelocities();
66     auto [lVel, rVel] = RobotUtils::getWheelVelocities(sensor, dt_);
67     double accelMag = std::sqrt(sensor.accelX*sensor.accelX + sensor.
    ↪ accelY*sensor.accelY);
68     double avgVel = (lVel + rVel)/2.0;
69     bool encOk = (std::abs(lVel) < 2.0 && std::abs(rVel) < 2.0);
70     bool lowMotion = (std::abs(lVel)<0.05 && std::abs(rVel)<0.05);
71     bool zuptActive = (lowMotion && accelMag < 0.05);
72
73     std::cout << "\n===== ROBOT STATUS =====\n";
74     std::cout << std::fixed << std::setprecision(3);
75     std::cout << "POSE: x=" << pose[0] << "m, y=" << pose[1] << "m,  $\hat{y}$ =
    ↪ "
76         << RobotUtils::radToDeg(pose[2]) << "°\n";
77     std::cout << "VELOCITIES: vx=" << vel[0] << "m/s, vy=" << vel[1]
78         << "m/s,  $\hat{L}$ =" << RobotUtils::radToDeg(vel[2]) << "°/s\n"
    ↪ ;
79     std::cout << "ENCODERS: L=" << lVel << "m/s, R=" << rVel
80         << "m/s (avg=" << avgVel << ")\n";
81     std::cout << "SENSORS: yaw=" << sensor.yaw << "° $\hat{r}$ , gyroZ=" <<
    ↪ sensor.gyroZ << "rad/s\n";
82     std::cout << "ACCEL: X=" << sensor.accelX << "m/s $\hat{A}$  $\hat{s}$ , Y=" << sensor.
    ↪ accelY
83         << "m/s $\hat{A}$  $\hat{s}$  (mag=" << accelMag << ")\n";
84     std::cout << "DIAGNOSTICS: dt=" << dt_ << "s, Battery=" << sensor.
    ↪ vbat1 << "mV\n";
85     std::cout << "STATUS: Encoders=" << (encOk ? "OK" : "UNRELIABLE")
86         << ", Motion=" << (lowMotion ? "LOW" : "HIGH")
87         << ", ZUPT=" << (zuptActive ? "ACTIVE" : "INACTIVE") <<
    ↪ "\n";
88     std::cout << "=====\n" << std::endl;
89 }
90
91 void RobotLocalization::logData(const SensorPacket& sensor) {
92     if (!logFile_.is_open()) return;
93

```

```

94     auto pose          = g_ekf.getPose();
95     auto vel           = g_ekf.getVelocities();
96     auto [lVel, rVel] = RobotUtils::getWheelVelocities(sensor, dt_);
97     auto now_ms = std::chrono::duration_cast<std::chrono::milliseconds>
    ↪ >(
98         std::chrono::system_clock::now()
99         .time_since_epoch()).count();
100     uint32_t ts = static_cast<uint32_t>(now_ms & 0xFFFFFFFF);
101
102     logFile_ << ts << ", "
103         << pose[0] << ", " << pose[1] << ", " << pose[2] << ", "
104         << vel[0] << ", " << vel[1] << ", " << vel[2] << ", "
105         << lVel << ", " << rVel << ", "
106         << sensor.yaw << ", " << sensor.gyroZ << ", "
107         << sensor.accelX << ", " << sensor.accelY << "\n";
108     logFile_.flush();
109 }
110
111 std::string RobotLocalization::getCurrentTimestamp() {
112     auto t = std::chrono::system_clock::to_time_t(
113         std::chrono::system_clock::now());
114     std::tm tm = *std::localtime(&t);
115     std::ostringstream ss;
116     ss << std::put_time(&tm, "%Y%m%d_%H%M%S");
117     return ss.str();
118 }
119
120 std::array<double, 3> RobotLocalization::getPose() const {
121     return g_ekf.getPose();
122 }
123 std::array<double, 3> RobotLocalization::getVelocities() const {
124     return g_ekf.getVelocities();
125 }

```

Listing 28: RobotLocalization.cpp

3.5 Path Planning

3.5.1 C++ Implementation (A* & PRM)

Role & Design

maincp.cpp delivers grid and sampling-based planners on a 2D occupancy grid.

- Preprocess map: threshold, inflate obstacles by robot radius.
- **A***: Eight-connected grid search with Euclidean cost + heuristic.
- **PRM**: Sample N points, connect within radius R, graph search for shortest path.
- Compare path lengths; choose shorter; detect waypoints where heading change $>10^\circ$ for profiling.

Planned Path Visualization

To illustrate the effectiveness of our two planning approaches, we overlay the computed trajectories on the same occupancy map.

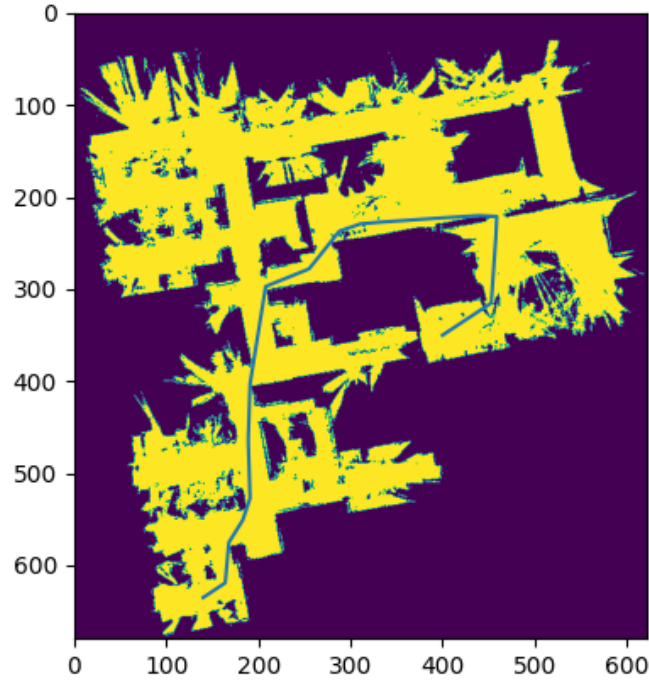


Figure 1: A* Planned Path overlaid on the inflated occupancy grid. The yellow line follows the shortest grid-based route, hugging the free-space boundary and making precise 45° and 90° turns at corridor junctions.

In Figure 1, the A* algorithm searches cell-by-cell across an 8-connected grid. Starting from the bottom-left, it maintains one-cell clearance from walls, navigates narrow passages, and guarantees minimal path length under the grid metric.

Figure 2 shows the Probabilistic Roadmap solution. A sparse set of samples is connected into a graph, then searched to produce a path that:

- Cuts more directly across large open areas via straight-line connections.
- Introduces fewer, but sometimes longer, segments compared to the grid-constrained A* path.
- Matches A* performance in tight corridors, yet yields smoother trajectories in sparse regions.

By comparing these visualizations, we see that A* guarantees the shortest grid-based route, while PRM exploits open spaces for more natural "as-the-crow-flies" segments.

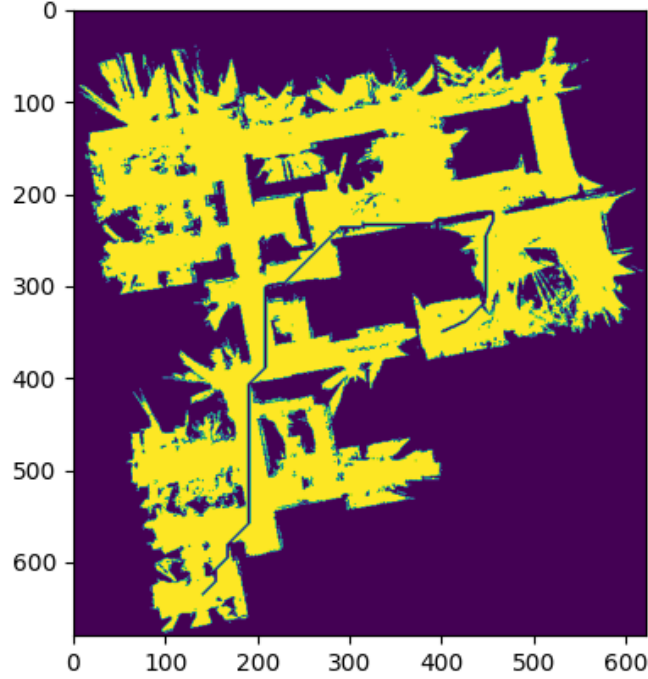


Figure 2: PRM Planned Path on the same map. Randomly sampled waypoints yield longer straight segments in open areas, with turns only at connection nodes.

To further demonstrate the robustness of our planning algorithms, Figures 3 and 4 overlay a different start-goal pair on the same occupancy map.

In Figure 3, the A* planner begins at the green start marker and systematically expands through adjacent free cells. Its tightly zig-zagging red polyline reflects the grid's connectivity constraints, ensuring that every move is optimal in terms of grid distance, even in complex, cluttered sections of the map.

Figure 4 shows the PRM solution for this new start-goal configuration. By sprinkling samples across the free space and linking neighbors, PRM constructs a graph whose shortest-path search yields a red trajectory with smoother, "as-the-crow-flies" edges. It retains corridor-following behavior where necessary but dramatically reduces the number of turns and stop-start motions compared to A*.

Comparing Figures 3 and 4 highlights the trade-off between guaranteed grid-optimality and smooth, continuous trajectories. While A* ensures the minimal cell-based route, PRM delivers a more natural path with fewer abrupt heading changes, improving kinematic feasibility and average speed in environments combining narrow passages and open areas.

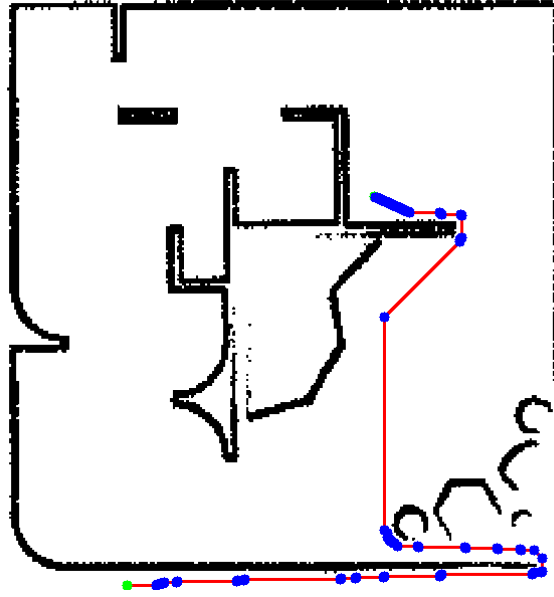


Figure 3: A* planned path for an alternate route. The A* algorithm again searches cell-by-cell on the 8-connected grid, maintaining one-cell clearance from walls, navigating narrow corridors with precise 45° and 90° turns, and guaranteeing the shortest possible route under the discrete grid metric.

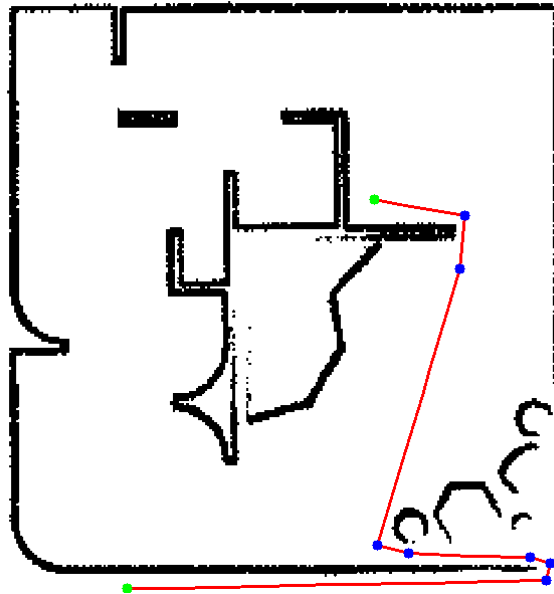


Figure 4: PRM planned path for the same alternate scenario. The Probabilistic Roadmap samples free-space nodes and connects them with straight-line edges, producing longer, direct segments in open areas and fewer waypoints in tight passages.

```

1 // PathFinder.cpp
2 // A* and PRM path planning with turning point detection using OpenCV
3
4 #include <opencv2/opencv.hpp>
5 #include <iostream>
6 #include <vector>
7 #include <queue>
8 #include <map>
9 #include <set>
10 #include <cmath>
11 #include <random>
12
13 using namespace cv;
14 using namespace std;
15
16 struct Pt {
17     int x, y;
18     bool operator<(Pt const &o) const {
19         return x<o.x || (x==o.x && y<o.y);
20     }
21     bool operator==(Pt const &o) const {
22         return x==o.x && y==o.y;
23     }
24 };
25
26 // hash for unordered_map if needed
27 struct PtHash { size_t operator()(Pt const &p) const { return (p.x<<16)
    ↪ ^ p.y; } };
28
29 // Euclidean distance
30 static double dist(Pt a, Pt b) {
31     double dx = a.x - b.x;
32     double dy = a.y - b.y;
33     return sqrt(dx*dx + dy*dy);
34 }
35
36 // Find nearest free pixel if start/goal blocked
37 Pt findNearestFree(Mat const &grid, Pt pt, int max_radius=20) {
38     if (grid.at<uchar>(pt.x, pt.y) == 255)
39         return pt;
40     int rows=grid.rows, cols=grid.cols;
41     for(int r=1; r<=max_radius; ++r){
42         for(int dx=-r; dx<=r; ++dx){
43             for(int dy=-r; dy<=r; ++dy){
44                 int x = pt.x+dx, y=pt.y+dy;
45                 if(x>=0&&x<rows&&y>=0&&y<cols && grid.at<uchar>(x,y)
    ↪ ==255)
46                     return Pt{x,y};
47             }
48         }
49     }
50     return pt;
51 }
52
53 // A* planner
54 class AStar {
55 public:

```

```

56 AStar(Mat const &g): grid(g) {}
57
58 vector<Pt> search(Pt s, Pt goal) {
59     s = findNearestFree(grid,s);
60     goal = findNearestFree(grid,goal);
61     map<Pt,double> gScore, fScore;
62     map<Pt,Pt> pred;
63     auto cmp = [&](Pt a, Pt b){ return fScore[a] > fScore[b]; };
64     priority_queue<Pt, vector<Pt>, decltype(cmp)> open(cmp);
65
66     gScore[s]=0;
67     fScore[s]=dist(s,goal);
68     open.push(s);
69
70     set<Pt> inOpen;
71     inOpen.insert(s);
72
73     vector<Pt> dirs
74         ↪ ={{1,0},{1,1},{0,1},{-1,1},{-1,0},{-1,-1},{0,-1},{1,-1}};
75
76     while(!open.empty()){
77         Pt u=open.top(); open.pop(); inOpen.erase(u);
78         if(u==goal) return reconstruct(pred,s,goal);
79         for(auto &d:dirs){
80             Pt v{u.x+d.x,u.y+d.y};
81             if(v.x<0||v.x>=grid.rows||v.y<0||v.y>=grid.cols)
82                 continue;
83             if(grid.at<uchar>(v.x,v.y)!=255) continue;
84             double tentative = gScore[u] + dist(u,v);
85             if(!gScore.count(v) || tentative < gScore[v]){
86                 pred[v]=u;
87                 gScore[v]=tentative;
88                 fScore[v]=tentative + dist(v,goal);
89                 if(!inOpen.count(v)){
90                     open.push(v);
91                     inOpen.insert(v);
92                 }
93             }
94         }
95     }
96     return {};
97
98 private:
99     Mat grid;
100     vector<Pt> reconstruct(map<Pt,Pt> &pred, Pt s, Pt g) {
101         vector<Pt> path;
102         Pt cur=g;
103         while(!(cur==s)){
104             path.push_back(cur);
105             cur = pred[cur];
106         }
107         path.push_back(s);
108         reverse(path.begin(), path.end());
109         return path;
110     }
111 };
112

```



```

113 // Simple PRM planner
114 class PRM {
115 public:
116     PRM(Mat const &g): grid(g), gen(random_device{}()) {}
117
118     void build(int N, double dmax) {
119         uniform_int_distribution<> rx(0,grid.rows-1), ry(0,grid.cols-1)
120         ↪ ;
121         while(nodes.size()<N){
122             Pt p{rx(gen), ry(gen)};
123             if(grid.at<uchar>(p.x,p.y)==255 && !conn.count(p)){
124                 conn[p]={};
125                 nodes.push_back(p);
126                 for(auto &u: nodes){
127                     if(!(u==p) && dist(u,p)<=dmax && connectable(u,p)){
128                         conn[u].push_back({p,dist(u,p)});
129                         conn[p].push_back({u,dist(u,p)});
130                     }
131                 }
132             }
133         }
134
135         vector<Pt> findPath(Pt s, Pt goal, double dmax) {
136             s = findNearestFree(grid,s);
137             goal = findNearestFree(grid,goal);
138             conn[s]={}; conn[goal]={};
139             // connect s and goal
140             for(auto &u: nodes){
141                 if(dist(u,s)<=dmax && connectable(u,s)) conn[u].push_back({
142                     ↪ s,dist(u,s)}), conn[s].push_back({u,dist(u,s)});
143                 if(dist(u,goal)<=dmax && connectable(u,goal)) conn[u].
144                     ↪ push_back({goal,dist(u,goal)}), conn[goal].push_back
145                     ↪ ({u,dist(u,goal)});
146             }
147             // A* on roadmap
148             map<Pt,double> gScore,fScore;
149             map<Pt,Pt> pred;
150             auto cmp=[&](Pt a, Pt b){ return fScore[a]>fScore[b]; };
151             priority_queue<Pt, vector<Pt>, decltype(cmp)> open(cmp);
152             gScore[s]=0; fScore[s]=dist(s,goal);
153             open.push(s);
154             set<Pt> inOpen; inOpen.insert(s);
155             while(!open.empty()){
156                 Pt u=open.top(); open.pop(); inOpen.erase(u);
157                 if(u==goal) return reconstruct(pred,s,goal);
158                 for(auto &edge:conn[u]){
159                     Pt v=edge.first; double w=edge.second;
160                     double tentative = gScore[u]+w;
161                     if(!gScore.count(v)||tentative<gScore[v]){
162                         pred[v]=u; gScore[v]=tentative;
163                         fScore[v]=tentative+dist(v,goal);
164                         if(!inOpen.count(v)) open.push(v), inOpen.insert(v)
165                         ↪ ;
166                     }
167                 }
168             }
169             return {};
170         }
171     }
172 };

```

```

166     }
167
168 private:
169     Mat grid;
170     vector<Pt> nodes;
171     map<Pt, vector<pair<Pt, double>>> conn;
172     mt19937 gen;
173
174     bool connectable(Pt a, Pt b){
175         int steps = max(abs(b.x-a.x), abs(b.y-a.y));
176         for(int i=0; i<=steps; ++i){
177             double t=(double)i/steps;
178             int x=round(a.x+(b.x-a.x)*t), y=round(a.y+(b.y-a.y)*t);
179             if(grid.at<uchar>(x,y)==0) return false;
180         }
181         return true;
182     }
183
184     vector<Pt> reconstruct(map<Pt,Pt> &pred, Pt s, Pt g){
185         vector<Pt> path; Pt cur=g;
186         while(!(cur==s)) { path.push_back(cur); cur=pred[cur]; }
187         path.push_back(s); reverse(path.begin(), path.end()); return path
188         ↪ ;
189     }
190 };
191
192 int main(){
193     // Load and preprocess image
194     Mat img = imread("bxKHP.png", IMREAD_GRAYSCALE);
195     resize(img, img, Size(600,600), 0,0, INTER_AREA);
196     Mat mask = (img<128);
197     mask *= 255;
198
199     // Inflate obstacles
200     Mat inflated = mask.clone();
201     int r=6;
202     for(int x=0; x<mask.rows; ++x) for(int y=0; y<mask.cols; ++y) if(mask.
203     ↪ at<uchar>(x,y)==0){
204         int x0=max(0,x-r), x1=min(mask.rows-1,x+r);
205         int y0=max(0,y-r), y1=min(mask.cols-1,y+r);
206         inflated(Range(x0,x1), Range(y0,y1)).setTo(0);
207     }
208
209     Pt start{595,140}, goal{208,386};
210     // Run planners
211     AStar astar(inflated);
212     auto path1 = astar.search(start, goal);
213     PRM prm(inflated);
214     prm.build(2500,75);
215     auto path2 = prm.findPath(start, goal, 75);
216     double l1=1e9, l2=1e9;
217     for(int i=1; i<(int)path1.size(); ++i) l1+= dist(path1[i-1], path1[i])
218     ↪ ;
219     for(int i=1; i<(int)path2.size(); ++i) l2+= dist(path2[i-1], path2[i])
220     ↪ ;
221     auto chosen = l1<=l2?path1:path2;
222
223     cout<<"Chosen: " <<(l1<=l2?"A*":"PRM")<<"\n";

```

```

220 // Detect turns
221 vector<tuple<int,Pt,double>> turns;
222 for(int i=1;i+1<(int)chosen.size();++i){
223     Vec2d v1(chosen[i].x-chosen[i-1].x, chosen[i].y-chosen[i-1].y);
224     Vec2d v2(chosen[i+1].x-chosen[i].x, chosen[i+1].y-chosen[i].y);
225     double n1=norm(v1), n2=norm(v2);
226     if(n1<1e-6||n2<1e-6) continue;
227     double angle=degrees(acos(clamp((v1.dot(v2))/(n1*n2), -1.0,1.0)
228         ↪ ));
229     if(angle>=10) turns.emplace_back(i,chosen[i],angle);
230 }
231 // Include start/end
232 vector<pair<Pt,double>> keyPts;
233 keyPts.emplace_back(start,0);
234 for(auto &t:turns){ keyPts.emplace_back(get<1>(t),get<2>(t)); }
235 keyPts.emplace_back(goal,0);
236
237 // Distances between keyPts
238 cout<<"KeyPoint distances:\n";
239 for(int i=0;i+1<(int)keyPts.size();++i){
240     Pt p1=keyPts[i].first, p2=keyPts[i+1].first;
241     // sum along chosen path between indices
242     int idx1 = (i==0?0:get<0>(turns[i-1]));
243     int idx2 = (i+1==(int)keyPts.size()-1?chosen.size()-1:get<0>(
244         ↪ turns[i]));
245     double d=0;
246     for(int j=idx1+1;j<=idx2;++j) d+=dist(chosen[j-1],chosen[j]);
247     cout<<" ("<<p1.x<<" "<<p1.y<<" "<<p2.x<<" "<<p2.y<<">: "<<d<<
248         ↪ "\n";
249 }
250 return 0;
251 }
252 // End of PathFinder.cpp

```

Listing 29: maincp.cpp

3.5.2 Python Reference Implementation

Role & Design

main.py mirrors C++ logic with NumPy, NetworkX, and Matplotlib:

- Binarize and inflate map in Python for rapid prototyping.
- Build and search PRM graphs with NetworkX.
- Visualize occupancy and final path with turning points overlay.

```

1 import numpy as np
2 import math
3 import matplotlib.pyplot as plt
4 import networkx as nx
5 import random as rn
6 from PIL import Image
7

```

[illegible]

[illegible][illegible]

```

79 class PRM:
80     def __init__(self, grid): self.grid=grid; self.G=nx.Graph()
81     def sample(self):
82         rows,cols=self.grid.shape
83         while True:
84             pt=(rn.randrange(rows), rn.randrange(cols))
85             if self.grid[pt]==255 and pt not in self.G: return pt
86     def connectable(self,v1,v2):
87         steps=max(abs(v2[0]-v1[0]), abs(v2[1]-v1[1]))
88         for i in range(steps+1):
89             t=i/steps; x=int(round(v1[0]+(v2[0]-v1[0])*t)); y=int(round
           ↪ (v1[1]+(v2[1]-v1[1])*t))
90             if self.grid[x,y]==0: return False
91         return True
92     def build(self,N,dmax):
93         while self.G.number_of_nodes()
```

[illegible]

```

157 plt.imshow(mask, cmap='gray', origin='upper')
158 if chosen:
159     xs, ys = zip(*[(p[1], p[0]) for p in chosen])
160     plt.plot(xs, ys, '-r')
161 # plot key points
162 for _, coord, _ in key_points:
163     plt.scatter(coord[1], coord[0], c='blue', s=50)
164 plt.scatter(start[1], start[0], c='green', s=50)
165 plt.scatter(goal[1], goal[0], c='green', s=50)
166 plt.title("Path & Key Points (incl. start/goal)")
167 plt.axis('off')
168 plt.show()
169
170 # end of script
171 plt.figure(figsize=(6,6)); plt.imshow(mask,cmap='gray',origin='
    ↳ upper')
172 if chosen:
173     xs,ys=zip(*[(p[1],p[0]) for p in chosen]); plt.plot(xs,ys,'-r')
174 for _,coord,_ in turns: plt.scatter(coord[1],coord[0],c='blue',s
    ↳ =50)
175 plt.scatter(start[1],start[0],c='green',s=50); plt.scatter(goal[1],
    ↳ goal[0],c='green',s=50)
176 plt.title("Path & Turning Points"); plt.axis('off'); plt.show()

```

Listing 30: main.py

3.6 Mapping (SLAM)

Two complementary SLAM strategies: an **Online SLAM** for low-latency mapping within the control loop, and an **Offline SLAM** pipeline for batch global optimization can be implemented. Both approaches consume logged LiDAR scans and pose estimates, but differ in their computational complexity, consistency guarantees, and output fidelity.

3.6.1 Online SLAM (Real-Time Occupancy Grid)

In the 10 ms control loop, it maintains a lightweight occupancy grid that updates at 10 Hz. This map drives reactive obstacle avoidance and provides live telemetry to remote dashboards.

On each mapping cycle:

- Retrieve the latest EKF-fused pose.
- Ray-trace every LiDAR beam using Bresenham's algorithm:
 - *Free updates* for cells along the beam (misses).
 - *Occupied updates* for the endpoint (hit).
- Apply log-odds increments to each cell, bounded to prevent numerical overflow.
- Optionally run a fast median filter to smooth isolated spurious hits.

Key Attributes:

- *Latency:* Map updates complete in a few milliseconds suitable for real-time collision checks.
- *Resource Usage:* Fixed-size grid and simple ray-tracing ensure predictable CPU and memory usage.

- *Drift*: No loop closures; odometry errors accumulate over time.

```

1 #pragma once
2 /*****
3  * OccupancyGrid 2-D log-odds map suitable for SLAM / navigation.
4  * - fixed-size, axis-aligned grid
5  * - Bresenham ray tracing update from 2-D LiDAR scans
6  * - thread-safe (internal mutex)
7  * - image export for quick visualisation
8  * - binary (de)serialisation helpers
9  * - NEW: serialize() helper to publish the grid over UDP or ROS2.
10 *****/
11 #include <vector>
12 #include <mutex>
13 #include <cmath>
14 #include <opencv2/core.hpp>
15
16 #include "lidar/lidar_handler.hpp" // LidarPoint
17 #include "localization/robot_utils.hpp" // RobotPose
18
19 /* 2-D Log-odds parameters (tweak to taste)
20    ↳ 2-D log-odds parameters (tweak to taste)
21    ↳ */
22 constexpr float LOG_ODD_OCCUPIED = 3.5f; // + 2 more certain
23    ↳ occupied
24 constexpr float LOG_ODD_FREE = -3.5f; // 2 more certain
25    ↳ free
26 constexpr float LOG_ODD_MAX = 10.0f;
27 constexpr float LOG_ODD_MIN = -10.0f;
28 /*
29    ↳ 2-D log-odds parameters (tweak to taste)
30    ↳ */
31
32 struct RobotPose {
33     float x{0.f};
34     float y{0.f};
35     float theta{0.f};
36 };
37
38 class OccupancyGrid
39 {
40 public:
41     /** width_m 2 height_m [metres] at given resolution [metres/cell].
42         origin_{x,y}_m is the world-space position of cell (0,0) centre
43         ↳ .
44         ↳ */
45     OccupancyGrid(float width_m,
46                   float height_m,
47                   float resolution_m,
48                   float origin_x_m = 0.f,
49                   float origin_y_m = 0.f);
50
51     /* 2-D Log-odds parameters (tweak to taste)
52        ↳ 2-D log-odds parameters (tweak to taste)
53        ↳ */
54     void updateGrid(const std::vector<LidarPoint>& scan,
55                    const RobotPose& robot_pose);
56
57     /** 0 2 free, 1 2 occupied, 0.5 2 unknown.
58         ↳ 2-D log-odds parameters (tweak to taste)
59         ↳ */

```



```

48     ↪          */
49     float getOccupancy(float x_m, float y_m) const;
50     /** BGR image: white = free, black = occupied, grey = unknown.
51     ↪ */
52     cv::Mat toImage() const;
53     /** Persistence
54     ↪ */
55     void saveToFile(const std::string& filename) const;
56     void loadFromFile(const std::string& filename);
57     void clear();    ///< reset to all unknown
58     /** New: helpers for map publishers
59     ↪ */
60     /** Packs the grid (row-major) into out as:
61     ↪     - each byte 0-100 occupancy probability [%]
62     ↪     - value 255 unknown
63     ↪          */
64     bool serialize(std::vector<uint8_t>& out) const;
65     /** Lightweight metadata accessors (handy for headers) */
66     inline int    widthCells() const { return width_cells_; }
67     inline int    heightCells() const { return height_cells_; }
68     inline float  resolution() const { return resolution_; }
69     inline float  originX() const { return origin_x_; }
70     inline float  originY() const { return origin_y_; }
71 private:
72     /** helpers -----
73     ↪ */
74     bool worldToGrid(float x_m, float y_m,
75     ↪ int& grid_x, int& grid_y) const;
76     void gridToWorld(int grid_x, int grid_y,
77     ↪ float& x_m, float& y_m) const;
78     float logOddsToProb(float log_odds) const;
79     void raycast(const RobotPose& start,
80     ↪ float end_x, float end_y,
81     ↪ std::vector<std::pair<int,int>>& cells) const;
82     /** geometry -----
83     ↪ */
84     float width_m_, height_m_, resolution_;
85     float origin_x_, origin_y_;
86     int width_cells_, height_cells_;
87     /** storage -----
88     ↪ */
89     mutable std::mutex grid_mutex_;
90     std::vector<std::vector<float>> log_odds_;    // [row][col]
91 };

```

Listing 31: Occupancy-Grid class interface (occupancy_grid.hpp)

[illegible]

```

190 {
191     gx = static_cast<int>((x_m - origin_x_) / resolution_);
192     gy = static_cast<int>((y_m - origin_y_) / resolution_);
193     return (gx >= 0 && gx < width_cells_ &&
194            gy >= 0 && gy < height_cells_);
195 }
196
197 void OccupancyGrid::gridToWorld(int gx, int gy,
198                                float& x_m, float& y_m) const
199 {
200     x_m = origin_x_ + (gx + 0.5f)*resolution_;
201     y_m = origin_y_ + (gy + 0.5f)*resolution_;
202 }
203
204 float OccupancyGrid::logOddsToProb(float lo) const
205 {
206     return 1.f / (1.f + std::exp(-lo));    //  $\tilde{C}(lo)$ 
207 }
208
209 /* Bresenham ray trace ----- */
210 void OccupancyGrid::raycast(const RobotPose& start,
211                             float end_x, float end_y,
212                             std::vector<std::pair<int,int>>& cells)
213                             ↪ const
214 {
215     cells.clear();
216
217     int x0, y0, x1, y1;
218     if (!worldToGrid(start.x, start.y, x0, y0)
219         || !worldToGrid(end_x, end_y, x1, y1))
220         return;
221
222     int dx = std::abs(x1 - x0), dy = std::abs(y1 - y0);
223     int sx = (x0 < x1) ? 1 : -1;
224     int sy = (y0 < y1) ? 1 : -1;
225     int err = dx - dy;
226
227     for (;;) {
228         cells.emplace_back(x0, y0);
229         if (x0 == x1 && y0 == y1) break;
230
231         int e2 = 2*err;
232         if (e2 > -dy) { err -= dy; x0 += sx; }
233         if (e2 < dx) { err += dx; y0 += sy; }
234     }
235 }

```

Listing 32: Real-time grid update logic (occupancy_grid.cpp)

3.6.2 Offline SLAM (Batch Scan-to-Map Alignment)

For post-mission map generation, we run a separate SLAM process that refines robot poses and rebuilds the occupancy grid with loop-closure and global smoothing:

1. **Initial Mapping:** Insert each logged scan into an `OccupancyMap` with Bresenham updates (hits/misses).
2. **Pose Refinement:** Use `ScanMatcher` to perform a hierarchical grid search, aligning each scan to the current map.
3. **Trajectory Storage:** Record refined poses and timestamps for batch optimization.
4. **Batch Optimization:**
 - *Re-refinement:* Re-match all scans to the evolving map with weighted corrections.
 - *Loop Closure Detection:* Identify revisit pairs within a distance threshold; apply pose graph corrections.
 - *Global Smoothing:* Windowed averaging to remove residual jitter while preserving structure.
5. **Final Map Rebuild:** Clear and re-insert all scans at optimized poses, apply median filtering, and export to PGM.

Key Attributes:

- *Consistency:* Loop-closure and pose-graph smoothing eliminate long-term drift.
- *Fidelity:* Sigmoid-style probability updates and median filtering produce crisp obstacle boundaries.
- *Compute Cost:* Multi-level scan matching and global optimization can take minutes to hours, depending on dataset size.

```
1 #pragma once
2 #include "SimpleSLAM.h"
3 #include <vector>
4 #include <string>
5
6 class OccupancyMap {
7 public:
8     OccupancyMap(double width, double height, double resolution);
9     ~OccupancyMap();
10
11     // Main interface
12     void addScan(const std::vector<LidarPoint>& scan, const Pose& pose)
13         ↪ ;
14     void clear();
15     void saveToFile(const std::string& filename) const;
16     void postProcessMap(); // MAKE SURE THIS LINE EXISTS
17
18     // Rest of declarations...
19     double getOccupancy(double x, double y) const;
20     double getOccupancyGrid(int grid_x, int grid_y) const;
21     int getWidth() const { return width_cells_; }
22     int getHeight() const { return height_cells_; }
23     double getResolution() const { return resolution_; }
```

```

23     void getBounds(double& min_x, double& min_y, double& max_x, double&
        ↪ max_y) const;
24
25     void worldToGrid(double x, double y, int& grid_x, int& grid_y)
        ↪ const;
26     void gridToWorld(int grid_x, int grid_y, double& x, double& y)
        ↪ const;
27     bool isValidGrid(int grid_x, int grid_y) const;
28
29 private:
30     std::vector<std::vector<double>> grid_;
31     std::vector<std::vector<int>> hits_;
32     std::vector<std::vector<int>> misses_;
33
34     int width_cells_, height_cells_;
35     double resolution_;
36     double origin_x_, origin_y_;
37
38     void updateLine(int x0, int y0, int x1, int y1);
39     void updateCell(int grid_x, int grid_y, bool occupied);
40     double logOdds(int hits, int misses) const;
41 };

```

Listing 33: Probability Grid Map class interface (OccupancyMap.h)

```

1  #include "OccupancyMap.h"
2  #include <fstream>
3  #include <iostream>
4  #include <algorithm>
5  #include <iomanip>
6
7  OccupancyMap::OccupancyMap(double width, double height, double
    ↪ resolution)
8      : resolution_(resolution)
9        , origin_x_(-width / 2.0)
10         , origin_y_(-height / 2.0) {
11
12     width_cells_ = static_cast<int>(std::ceil(width / resolution));
13     height_cells_ = static_cast<int>(std::ceil(height / resolution));
14
15     // Initialize grids
16     grid_.resize(height_cells_, std::vector<double>(width_cells_, 0.5))
        ↪ ; // Unknown = 0.5
17     hits_.resize(height_cells_, std::vector<int>(width_cells_, 0));
18     misses_.resize(height_cells_, std::vector<int>(width_cells_, 0));
19
20     std::cout << "OccupancyMap created: " << width_cells_ << "x" <<
        ↪ height_cells_
21             << " cells, resolution: " << resolution_ << "m" << std::
        ↪ endl;
22 }
23
24 OccupancyMap::~OccupancyMap() {
25 }
26
27 void OccupancyMap::addScan(const std::vector<LidarPoint>& scan, const
    ↪ Pose& pose) {
28     if (scan.empty()) return;

```



```

29
30 // Convert robot pose to grid coordinates
31 int robot_grid_x, robot_grid_y;
32 worldToGrid(pose.x, pose.y, robot_grid_x, robot_grid_y);
33
34 if (!isValidGrid(robot_grid_x, robot_grid_y)) {
35     // Try to handle poses near map boundary gracefully
36     std::cerr << "Warning: Robot pose near/outside map bounds: ("
37         << pose.x << ", " << pose.y << ") -> grid ("
38         << robot_grid_x << ", " << robot_grid_y << ")" << std
39         ↪ ::endl;
40
41     // Clamp to map bounds
42     robot_grid_x = std::max(0, std::min(width_cells_ - 1,
43         ↪ robot_grid_x));
44     robot_grid_y = std::max(0, std::min(height_cells_ - 1,
45         ↪ robot_grid_y));
46 }
47
48 // Process each laser beam
49 int valid_points = 0;
50 for (const auto& point : scan) {
51     if (point.distance <= 0.1 || point.distance > 30.0) {
52         continue; // Skip invalid readings
53     }
54
55     // Calculate endpoint in world coordinates
56     // Make sure angle transformation is correct for your LiDAR
57     ↪ coordinate system
58     double global_angle = pose.theta + point.angle;
59     double end_x = pose.x + point.distance * std::cos(global_angle)
60     ↪ ;
61     double end_y = pose.y + point.distance * std::sin(global_angle)
62     ↪ ;
63
64     // Convert to grid coordinates
65     int end_grid_x, end_grid_y;
66     worldToGrid(end_x, end_y, end_grid_x, end_grid_y);
67
68     // Skip points outside map bounds instead of clamping
69     if (!isValidGrid(end_grid_x, end_grid_y)) {
70         continue;
71     }
72
73     // Trace ray from robot to endpoint
74     updateLine(robot_grid_x, robot_grid_y, end_grid_x, end_grid_y);
75     valid_points++;
76 }
77
78 // Debug output for first few scans
79 static int debug_count = 0;
80 if (debug_count < 3) {
81     std::cout << " Processed " << valid_points << "/" << scan.size
82     ↪ ()
83         << " valid LiDAR points for scan " << debug_count <<
84     ↪ std::endl;
85     debug_count++;
86 }

```

```

79 }
80
81 void OccupancyMap::updateLine(int x0, int y0, int x1, int y1) {
82     // Bresenham's line algorithm
83     std::vector<std::pair<int, int>> line_points;
84
85     int dx = std::abs(x1 - x0);
86     int dy = std::abs(y1 - y0);
87     int sx = (x0 < x1) ? 1 : -1;
88     int sy = (y0 < y1) ? 1 : -1;
89     int err = dx - dy;
90
91     int x = x0, y = y0;
92
93     while (true) {
94         if (isValidGrid(x, y)) {
95             line_points.push_back({x, y});
96         }
97
98         if (x == x1 && y == y1) break;
99
100         int e2 = 2 * err;
101         if (e2 > -dy) {
102             err -= dy;
103             x += sx;
104         }
105         if (e2 < dx) {
106             err += dx;
107             y += sy;
108         }
109     }
110
111     // Update cells along the ray
112     for (size_t i = 0; i < line_points.size(); i++) {
113         int gx = line_points[i].first;
114         int gy = line_points[i].second;
115
116         if (i == line_points.size() - 1) {
117             // Last point is occupied (obstacle)
118             updateCell(gx, gy, true);
119         } else {
120             // Intermediate points are free space
121             updateCell(gx, gy, false);
122         }
123     }
124 }
125
126 void OccupancyMap::updateCell(int grid_x, int grid_y, bool occupied) {
127     if (!isValidGrid(grid_x, grid_y)) return;
128
129     if (occupied) {
130         hits_[grid_y][grid_x] += 3; // Stronger weight for obstacles
131     } else {
132         misses_[grid_y][grid_x]++;
133     }
134
135     int hits = hits_[grid_y][grid_x];
136     int misses = misses_[grid_y][grid_x];

```

```

137     int total = hits + misses;
138
139     if (total >= 2) { // Need at least 2 observations
140         double probability = static_cast<double>(hits) / total;
141
142         // Apply sigmoid-like function for better contrast
143         if (probability > 0.6) {
144             probability = 0.8 + 0.15 * (probability - 0.6) / 0.4;
145         } else if (probability < 0.4) {
146             probability = 0.05 + 0.15 * probability / 0.4;
147         } else {
148             probability = 0.2 + 0.6 * (probability - 0.4) / 0.2;
149         }
150
151         grid_[grid_y][grid_x] = probability;
152     }
153 }
154
155 void OccupancyMap::postProcessMap() {
156     std::cout << "Post-processing map for noise reduction..." << std::
157         ↪ endl;
158
159     // Median filter to reduce noise
160     std::vector<std::vector<double>> filtered_grid = grid_;
161
162     int filter_count = 0;
163     for (int y = 1; y < height_cells_ - 1; y++) {
164         for (int x = 1; x < width_cells_ - 1; x++) {
165             // Only filter cells that have been observed
166             if (hits_[y][x] + misses_[y][x] > 0) {
167                 std::vector<double> neighbors;
168
169                 for (int dy = -1; dy <= 1; dy++) {
170                     for (int dx = -1; dx <= 1; dx++) {
171                         neighbors.push_back(grid_[y + dy][x + dx]);
172                     }
173                 }
174
175                 std::sort(neighbors.begin(), neighbors.end());
176                 double old_value = filtered_grid[y][x];
177                 filtered_grid[y][x] = neighbors[4]; // Median of 9
178                 ↪ values
179
180                 if (std::abs(old_value - filtered_grid[y][x]) > 0.1) {
181                     filter_count++;
182                 }
183             }
184         }
185     }
186
187     grid_ = filtered_grid;
188     std::cout << "    Filtered " << filter_count << " noisy cells" << std
189         ↪ ::endl;
190 }
191
192 void OccupancyMap::clear() {
193     for (int y = 0; y < height_cells_; y++) {
194         for (int x = 0; x < width_cells_; x++) {

```

```

192         grid_[y][x] = 0.5;    // Unknown
193         hits_[y][x] = 0;
194         misses_[y][x] = 0;
195     }
196 }
197 }
198
199 double OccupancyMap::getOccupancy(double x, double y) const {
200     int grid_x, grid_y;
201     worldToGrid(x, y, grid_x, grid_y);
202     return getOccupancyGrid(grid_x, grid_y);
203 }
204
205 double OccupancyMap::getOccupancyGrid(int grid_x, int grid_y) const {
206     if (!isValidGrid(grid_x, grid_y)) {
207         return 0.5; // Unknown for out-of-bounds
208     }
209     return grid_[grid_y][grid_x];
210 }
211
212 void OccupancyMap::worldToGrid(double x, double y, int& grid_x, int&
    ↪ grid_y) const {
213     grid_x = static_cast<int>((x - origin_x_) / resolution_);
214     grid_y = static_cast<int>((y - origin_y_) / resolution_);
215 }
216
217 void OccupancyMap::gridToWorld(int grid_x, int grid_y, double& x,
    ↪ double& y) const {
218     x = origin_x_ + (grid_x + 0.5) * resolution_;
219     y = origin_y_ + (grid_y + 0.5) * resolution_;
220 }
221
222 bool OccupancyMap::isValidGrid(int grid_x, int grid_y) const {
223     return grid_x >= 0 && grid_x < width_cells_ &&
224            grid_y >= 0 && grid_y < height_cells_;
225 }
226
227 void OccupancyMap::getBounds(double& min_x, double& min_y, double&
    ↪ max_x, double& max_y) const {
228     min_x = origin_x_;
229     min_y = origin_y_;
230     max_x = origin_x_ + width_cells_ * resolution_;
231     max_y = origin_y_ + height_cells_ * resolution_;
232 }
233
234 void OccupancyMap::saveToFile(const std::string& filename) const {
235     std::ofstream file(filename);
236     if (!file.is_open()) {
237         std::cerr << "Failed to open file: " << filename << std::endl;
238         return;
239     }
240
241     // Calculate statistics
242     int occupied_cells = 0, free_cells = 0, unknown_cells = 0;
243     for (int y = 0; y < height_cells_; y++) {
244         for (int x = 0; x < width_cells_; x++) {
245             double prob = grid_[y][x];
246             if (prob > 0.6) occupied_cells++;

```

```

247         else if (prob < 0.4) free_cells++;
248         else unknown_cells++;
249     }
250 }
251
252 // Write PGM header
253 file << "P2" << std::endl;
254 file << "# Created by SimpleSLAM - Advanced Occupancy Mapping" <<
    ↪ std::endl;
255 file << "# Map resolution: " << resolution_ << " m/cell" << std::
    ↪ endl;
256 file << "# Map bounds: (" << origin_x_ << ", " << origin_y_ << ") to
    ↪ ("
257     << (origin_x_ + width_cells_ * resolution_) << ", "
258     << (origin_y_ + height_cells_ * resolution_) << ")" << std::
    ↪ endl;
259 file << "# Statistics: " << occupied_cells << " occupied, "
260     << free_cells << " free, " << unknown_cells << " unknown cells"
    ↪ << std::endl;
261 file << width_cells_ << " " << height_cells_ << std::endl;
262 file << "255" << std::endl;
263
264 // Write grid data (flip Y axis for standard image coordinates)
265 for (int y = height_cells_ - 1; y >= 0; y--) {
266     for (int x = 0; x < width_cells_; x++) {
267         double prob = grid_[y][x];
268         int total_obs = hits_[y][x] + misses_[y][x];
269         int pixel_value;
270
271         if (total_obs == 0) {
272             // Unexplored - Medium gray
273             pixel_value = 205;
274         } else if (prob > 0.8) {
275             // Strongly occupied - Black
276             pixel_value = 0;
277         } else if (prob > 0.6) {
278             // Moderately occupied - Dark gray
279             pixel_value = 64;
280         } else if (prob < 0.2) {
281             // Strongly free - White
282             pixel_value = 255;
283         } else if (prob < 0.4) {
284             // Moderately free - Light gray
285             pixel_value = 220;
286         } else {
287             // Uncertain - Medium gray
288             pixel_value = 128;
289         }
290
291         file << pixel_value << " ";
292     }
293     file << std::endl;
294 }
295
296 file.close();
297 std::cout << "Enhanced PGM map saved to: " << filename << std::endl;
298 std::cout << " Map contains: " << occupied_cells << " occupied, "
299     << free_cells << " free, " << unknown_cells << " unknown

```

```

300         ↪ cells" << std::endl;
301     }
302     double OccupancyMap::logOdds(int hits, int misses) const {
303         if (hits + misses == 0) return 0.0;
304
305         double prob = static_cast<double>(hits) / (hits + misses);
306         prob = std::max(0.01, std::min(0.99, prob)); // Clamp to avoid log
307         ↪ (0)
308
309         return std::log(prob / (1.0 - prob));
310     }

```

Listing 34: Grid map implementation (OccupancyMap.cpp)

```

1  #pragma once
2  #include "SimpleSLAM.h"
3
4  class OccupancyMap;
5
6  class ScanMatcher {
7  public:
8      ScanMatcher();
9      ~ScanMatcher();
10
11     // Main interface
12     Pose matchScanToMap(const std::vector<LidarPoint>& scan,
13                        const Pose& initial_guess,
14                        const OccupancyMap* map);
15
16     void setMap(const OccupancyMap* map) { map_ = map; }
17
18     // Configuration
19     void setMaxIterations(int max_iter) { max_iterations_ = max_iter; }
20     void setConvergenceThreshold(double threshold) {
21         ↪ convergence_threshold_ = threshold; }
22     void setSearchRange(double xy_range, double theta_range) {
23         search_range_xy_ = xy_range;
24         search_range_theta_ = theta_range;
25     }
26 private:
27     const OccupancyMap* map_;
28     int max_iterations_;
29     double convergence_threshold_;
30     double search_range_xy_;
31     double search_range_theta_;
32
33     // Matching algorithms
34     double calculateMatchScore(const std::vector<LidarPoint>& scan,
35                              const Pose& pose);
36     Pose optimizePose(const std::vector<LidarPoint>& scan,
37                     const Pose& initial_pose);
38
39     // Utility functions
40     std::vector<std::pair<double, double>> scanToCartesian(
41         const std::vector<LidarPoint>& scan, const Pose& pose);
42

```

```

43     Pose perturbPose(const Pose& pose, double dx, double dy, double
        ↪ dtheta);
44     double normalizeAngle(double angle);
45 };

```

Listing 35: Scan matcher interface (ScanMatcher.h)

```

1  #include "ScanMatcher.h"
2  #include "OccupancyMap.h"
3  #include <iostream>
4  #include <algorithm>
5  #include <cmath>
6
7  ScanMatcher::ScanMatcher()
8      : map_(nullptr)
9      , max_iterations_(15)
10     , convergence_threshold_(0.001)
11     , search_range_xy_(0.5)
12     , search_range_theta_(0.2) {
13 }
14
15 ScanMatcher::~ScanMatcher() {
16 }
17
18 Pose ScanMatcher::matchScanToMap(const std::vector<LidarPoint>& scan,
19                                 const Pose& initial_guess,
20                                 const OccupancyMap* map) {
21     if (!map || scan.empty()) {
22         return initial_guess;
23     }
24
25     map_ = map;
26
27     try {
28         return optimizePose(scan, initial_guess);
29     } catch (const std::exception& e) {
30         std::cerr << "Scan matching failed: " << e.what() << std::endl;
31         return initial_guess;
32     }
33 }
34
35 Pose ScanMatcher::optimizePose(const std::vector<LidarPoint>& scan,
36                               const Pose& initial_pose) {
37     Pose best_pose = initial_pose;
38     double best_score = calculateMatchScore(scan, initial_pose);
39
40     // Hierarchical search: coarse then fine
41     std::vector<double> xy_steps = {0.2, 0.1, 0.05}; // Multiple
        ↪ resolution levels
42     std::vector<double> theta_steps = {0.1, 0.05, 0.02};
43
44     for (size_t level = 0; level < xy_steps.size(); level++) {
45         double step_xy = xy_steps[level];
46         double step_theta = theta_steps[level];
47         double current_range_xy = search_range_xy_ / (level + 1);
48         double current_range_theta = search_range_theta_ / (level + 1);
49
50         Pose level_best = best_pose;

```

```

51     double level_best_score = best_score;
52
53     // Grid search around current best pose
54     for (double dx = -current_range_xy; dx <= current_range_xy; dx
55         ↪ += step_xy) {
56         for (double dy = -current_range_xy; dy <= current_range_xy;
57             ↪ dy += step_xy) {
58             for (double dtheta = -current_range_theta; dtheta <=
59                 ↪ current_range_theta; dtheta += step_theta) {
60
61                 Pose test_pose = perturbPose(best_pose, dx, dy,
62                     ↪ dtheta);
63                 double score = calculateMatchScore(scan, test_pose)
64                     ↪ ;
65
66                 if (score > level_best_score) {
67                     level_best = test_pose;
68                     level_best_score = score;
69                 }
70             }
71         }
72     }
73
74     best_pose = level_best;
75     best_score = level_best_score;
76 }
77
78 return best_pose;
79 }
80
81 double ScanMatcher::calculateMatchScore(const std::vector<LidarPoint>&
82     ↪ scan,
83
84     const Pose& pose) {
85
86     if (!map_) return 0.0;
87
88     auto cartesian_points = scanToCartesian(scan, pose);
89     double total_score = 0.0;
90     int valid_points = 0;
91
92     for (const auto& point : cartesian_points) {
93         double x = point.first;
94         double y = point.second;
95
96         // Check map bounds
97         double min_x, min_y, max_x, max_y;
98         map_->getBounds(min_x, min_y, max_x, max_y);
99
100        if (x >= min_x && x <= max_x && y >= min_y && y <= max_y) {
101            double occupancy = map_->getOccupancy(x, y);
102
103            // Improved scoring function
104            if (occupancy > 0.7) {
105                total_score += 2.0; // Strong reward for hitting
106                ↪ occupied areas
107            } else if (occupancy > 0.5) {
108                total_score += 1.0; // Moderate reward for likely
109                ↪ occupied areas
110            } else if (occupancy < 0.3) {

```



```

101         total_score -= 1.0; // Penalty for hitting free space
102     } else {
103         total_score += 0.0; // Neutral for unknown areas
104     }
105
106     valid_points++;
107 }
108 }
109
110 if (valid_points == 0) return 0.0;
111
112 // Normalize score and add bonus for more matched points
113 double normalized_score = total_score / valid_points;
114 double coverage_bonus = static_cast<double>(valid_points) / scan.
    ↪ size();
115
116 return normalized_score + coverage_bonus;
117 }
118
119 std::vector<std::pair<double, double>> ScanMatcher::scanToCartesian(
120     const std::vector<LidarPoint>& scan, const Pose& pose) {
121
122     std::vector<std::pair<double, double>> points;
123     points.reserve(scan.size());
124
125     for (const auto& lidar_point : scan) {
126         if (lidar_point.distance > 0.1 && lidar_point.distance < 30.0)
127             ↪ {
128                 double global_angle = pose.theta + lidar_point.angle;
129                 double x = pose.x + lidar_point.distance * std::cos(
130                     ↪ global_angle);
131                 double y = pose.y + lidar_point.distance * std::sin(
132                     ↪ global_angle);
133                 points.emplace_back(x, y);
134             }
135     }
136
137     return points;
138 }
139
140 Pose ScanMatcher::perturbPose(const Pose& pose, double dx, double dy,
141     ↪ double dtheta) {
142     return Pose(pose.x + dx, pose.y + dy, normalizeAngle(pose.theta +
143         ↪ dtheta));
144 }
145
146 double ScanMatcher::normalizeAngle(double angle) {
147     while (angle > M_PI) angle -= 2 * M_PI;
148     while (angle < -M_PI) angle += 2 * M_PI;
149     return angle;
150 }

```

Listing 36: Hierarchical scan matching (ScanMatcher.cpp)

```

1 #pragma once
2 #include <vector>
3 #include <array>
4 #include <string>

```

```

5  #include <memory>
6
7  #ifdef _WIN32
8  #define _USE_MATH_DEFINES
9  #endif
10 #include <cmath>
11 #ifndef M_PI
12 #define M_PI 3.14159265358979323846
13 #endif
14
15 #include <iostream>
16
17 struct LidarPoint {
18     double angle;      // radians, relative to robot
19     double distance;   // meters
20     int intensity;     // optional
21
22     LidarPoint() : angle(0), distance(0), intensity(0) {}
23     LidarPoint(double a, double d, int i = 0) : angle(a), distance(d),
24         ↪ intensity(i) {}
25 };
26
27 struct Pose {
28     double x, y, theta; // x, y in meters, theta in radians
29
30     Pose() : x(0), y(0), theta(0) {}
31     Pose(double x_, double y_, double theta_) : x(x_), y(y_), theta(
32         ↪ theta_) {}
33 };
34
35 // Forward declarations
36 class OccupancyMap;
37 class ScanMatcher;
38
39 class SimpleSLAM {
40 public:
41     SimpleSLAM();
42     ~SimpleSLAM();
43
44     // Core interface
45     bool initialize(double map_size = 50.0, double resolution = 0.1);
46     bool addScan(const std::vector<LidarPoint>& scan, const Pose&
47         ↪ initial_pose);
48     bool optimizeTrajectory();
49
50     // Results
51     std::vector<Pose> getOptimizedPoses() const;
52     void saveMap(const std::string& filename) const;
53     void saveTrajectory(const std::string& filename) const;
54
55     // Status
56     int getNumScans() const;
57     double getLastProcessingTime() const { return last_processing_time_
58         ↪ ; }
59     bool isInitialized() const { return initialized_; }
60     void printStats() const;
61     void clear();
62 };

```

```

59 private:
60     struct ScanData {
61         std::vector<LidarPoint> points;
62         Pose pose;
63         int id;
64
65         ScanData() : id(0) {}
66         ScanData(const std::vector<LidarPoint>& p, const Pose& pose_,
67                 ↪ int id_)
68             : points(p), pose(pose_), id(id_) {}
69
70     std::vector<ScanData> scans_;
71     std::vector<Pose> optimized_poses_;
72     std::unique_ptr<OccupancyMap> map_;
73     std::unique_ptr<ScanMatcher> matcher_;
74     double last_processing_time_;
75     bool initialized_;
76     int next_scan_id_;
77
78     // Internal methods
79     Pose refinePoseWithScanMatching(const std::vector<LidarPoint>& scan
80     ↪ ,
81                                     const Pose& initial_pose, int
82     ↪ scan_id);
83
84     void updateMap();
85     void detectAndCorrectLoops();
86     void correctLoop(size_t start_idx, size_t end_idx);
87     void smoothTrajectory();
88     double getCurrentTime() const;
89 };

```

Listing 37: Offline SLAM orchestrator interface (SimpleSLAM.h)

```

1  #include "SimpleSLAM.h"
2  #include "OccupancyMap.h"
3  #include "ScanMatcher.h"
4  #include <chrono>
5  #include <iostream>
6  #include <fstream>
7  #include <algorithm>
8  #include <iomanip>
9
10 SimpleSLAM::SimpleSLAM()
11     : last_processing_time_(0.0)
12     , initialized_(false)
13     , next_scan_id_(0) {
14 }
15
16 SimpleSLAM::~SimpleSLAM() {
17 }
18
19 bool SimpleSLAM::initialize(double map_size, double resolution) {
20     try {
21         // Create map
22         map_ = std::make_unique<OccupancyMap>(map_size, map_size,
23     ↪ resolution);
24 }

```

```

24     // Create scan matcher
25     matcher_ = std::make_unique<ScanMatcher>();
26     matcher_>setMap(map_.get());
27     matcher_>setSearchRange(0.5, 0.2); // 50cm, ~11 degrees
28
29     // Clear any existing data
30     scans_.clear();
31     optimized_poses_.clear();
32     next_scan_id_ = 0;
33
34     initialized_ = true;
35
36     std::cout << "SimpleSLAM initialized successfully!" << std::
    ↪ endl;
37     std::cout << "Map size: " << map_size << "x" << map_size << "m"
    ↪ << std::endl;
38     std::cout << "Resolution: " << resolution << "m/cell" << std::
    ↪ endl;
39     std::cout << "Grid size: " << map_->getWidth() << "x" << map_->
    ↪ getHeight() << " cells" << std::endl;
40
41     return true;
42 } catch (const std::exception& e) {
43     std::cerr << "Failed to initialize SimpleSLAM: " << e.what() <<
    ↪ std::endl;
44     return false;
45 }
46 }
47
48 bool SimpleSLAM::addScan(const std::vector<LidarPoint>& scan, const
    ↪ Pose& initial_pose) {
49     if (!initialized_) {
50         std::cerr << "SimpleSLAM not initialized!" << std::endl;
51         return false;
52     }
53
54     if (scan.empty()) {
55         std::cerr << "Empty scan provided!" << std::endl;
56         return false;
57     }
58
59     try {
60         auto start_time = std::chrono::high_resolution_clock::now();
61
62         // Refine pose using scan matching (if we have previous scans)
63         Pose refined_pose = initial_pose;
64         if (!scans_.empty() && matcher_) {
65             refined_pose = refinePoseWithScanMatching(scan,
    ↪ initial_pose, next_scan_id_);
66         }
67
68         // Store scan data
69         ScanData scan_data(scan, refined_pose, next_scan_id_++);
70         scans_.push_back(scan_data);
71         optimized_poses_.push_back(refined_pose);
72
73         // Add to map
74         map_->addScan(scan, refined_pose);

```

```

75
76     auto end_time = std::chrono::high_resolution_clock::now();
77     auto duration = std::chrono::duration_cast<std::chrono::
    ↳ microseconds>(end_time - start_time);
78     last_processing_time_ = duration.count() / 1000.0; // Convert
    ↳ to milliseconds
79
80     if (next_scan_id_ % 10 == 0 || next_scan_id_ <= 5) {
81         std::cout << "Added scan " << (next_scan_id_ - 1)
82             << " at pose (" << std::fixed << std::
    ↳ setprecision(2)
83             << refined_pose.x << ", " << refined_pose.y
84             << ", " << refined_pose.theta * 180.0 / M_PI << "
    ↳ Åř)"
85             << " - " << last_processing_time_ << "ms" << std
    ↳ ::endl;
86     }
87
88     return true;
89 } catch (const std::exception& e) {
90     std::cerr << "Failed to add scan: " << e.what() << std::endl;
91     return false;
92 }
93 }
94
95 Pose SimpleSLAM::refinePoseWithScanMatching(const std::vector<
    ↳ LidarPoint>& scan,
96                                             const Pose& initial_pose,
    ↳ int scan_id) {
97     if (scans_.empty() || !matcher_) {
98         return initial_pose; // No previous scans to match against
99     }
100
101     try {
102         // Use scan matcher to refine pose against map
103         Pose refined_pose = matcher_->matchScanToMap(scan, initial_pose
    ↳ , map_.get());
104
105         // Calculate improvement
106         double dx = refined_pose.x - initial_pose.x;
107         double dy = refined_pose.y - initial_pose.y;
108         double dtheta = refined_pose.theta - initial_pose.theta;
109         double improvement = std::sqrt(dx*dx + dy*dy) + std::abs(dtheta
    ↳ );
110
111         if (improvement > 0.05 && scan_id < 10) { // Only print
    ↳ significant improvements for first few scans
112             std::cout << " Scan matching improved pose by " << std::
    ↳ fixed << std::setprecision(3)
113                 << improvement << " (dx=" << dx << ", dy=" << dy
    ↳ << ", dÅř=" << dtheta*180/M_PI << "Åř)" <<
    ↳ std::endl;
114         }
115
116         return refined_pose;
117     } catch (const std::exception& e) {
118         std::cerr << "Scan matching failed: " << e.what() << std::endl;
119         return initial_pose; // Fall back to initial pose

```

```

120     }
121 }
122
123 bool SimpleSLAM::optimizeTrajectory() {
124     if (!initialized_ || scans_.size() < 5) {
125         std::cout << "Not enough scans for optimization (need at least
126             ↳ 5, have "
127                 << scans_.size() << ")" << std::endl;
128         return false;
129     }
130
131     auto start_time = std::chrono::high_resolution_clock::now();
132
133     try {
134         std::cout << "\n=== Starting Advanced Trajectory Optimization
135             ↳ ===" << std::endl;
136         std::cout << "Processing " << scans_.size() << " scans..." <<
137             ↳ std::endl;
138
139         // PHASE 1: Scan-to-scan matching for consecutive poses
140         std::cout << "Phase 1: Scan-to-map matching refinement..." <<
141             ↳ std::endl;
142
143         int improvements = 0;
144         for (size_t i = 1; i < scans_.size(); i++) {
145             if (matcher_) {
146                 Pose original_pose = optimized_poses_[i];
147
148                 // Match current scan against current map
149                 Pose refined_pose = matcher_->matchScanToMap(
150                     scans_[i].points,
151                     scans_[i].pose,
152                     map_.get()
153                 );
154
155                 // Apply gradual correction to avoid jumps
156                 double weight = 0.3; // 30% correction
157
158                 optimized_poses_[i].x = (1 - weight) * optimized_poses_
159                     ↳ [i].x + weight * refined_pose.x;
160                 optimized_poses_[i].y = (1 - weight) * optimized_poses_
161                     ↳ [i].y + weight * refined_pose.y;
162                 optimized_poses_[i].theta = (1 - weight) *
163                     ↳ optimized_poses_[i].theta + weight * refined_pose
164                     ↳ .theta;
165
166                 scans_[i].pose = optimized_poses_[i];
167
168                 // Count significant improvements
169                 double dx = optimized_poses_[i].x - original_pose.x;
170                 double dy = optimized_poses_[i].y - original_pose.y;
171                 if (std::sqrt(dx*dx + dy*dy) > 0.05) {
172                     improvements++;
173                 }
174             }
175         }
176
177         std::cout << "    Improved " << improvements << " poses" << std::
178             ↳ endl;

```

```

169
170 // PHASE 2: Loop closure detection
171 std::cout << "Phase 2: Loop closure detection..." << std::endl;
172 detectAndCorrectLoops();
173
174 // PHASE 3: Global trajectory smoothing
175 std::cout << "Phase 3: Global trajectory smoothing..." << std::
    ↪ endl;
176 smoothTrajectory();
177
178 // PHASE 4: Rebuild map with optimized poses
179 std::cout << "Phase 4: Rebuilding map with optimized poses..."
    ↪ << std::endl;
180 updateMap();
181
182 auto end_time = std::chrono::high_resolution_clock::now();
183 auto duration = std::chrono::duration_cast<std::chrono::
    ↪ milliseconds>(end_time - start_time);
184 last_processing_time_ = duration.count();
185
186 std::cout << "=== Optimization completed in "
187           << last_processing_time_ << "ms ===" << std::endl;
188
189 return true;
190
191 } catch (const std::exception& e) {
192     std::cerr << "Advanced optimization failed: " << e.what() <<
    ↪ std::endl;
193     return false;
194 }
195 }
196
197 void SimpleSLAM::detectAndCorrectLoops() {
198     int loop_closures = 0;
199
200     // Simple loop closure: check if robot returns to similar position
201     for (size_t i = 0; i < optimized_poses_.size(); i++) {
202         for (size_t j = i + 20; j < optimized_poses_.size(); j++) { //
    ↪ At least 20 scans apart
203
204             double dx = optimized_poses_[i].x - optimized_poses_[j].x;
205             double dy = optimized_poses_[i].y - optimized_poses_[j].y;
206             double distance = std::sqrt(dx*dx + dy*dy);
207
208             if (distance < 1.0) { // Within 1.0m - potential loop
209                 std::cout << " Loop closure detected between scans "
    ↪ << i << " and " << j
210                   << " (distance: " << std::fixed << std::
    ↪ setprecision(2) << distance << "m)" <<
    ↪ std::endl;
211
212                 // Apply loop closure correction
213                 correctLoop(i, j);
214                 loop_closures++;
215
216                 if (loop_closures >= 3) break; // Limit corrections per
    ↪ optimization
217             }

```

```

218     }
219     if (loop_closures >= 3) break;
220 }
221
222 if (loop_closures == 0) {
223     std::cout << "    No loop closures detected" << std::endl;
224 } else {
225     std::cout << "    Applied " << loop_closures << " loop closure
        ↳ corrections" << std::endl;
226 }
227 }
228
229 void SimpleSLAM::correctLoop(size_t start_idx, size_t end_idx) {
230     // Distribute the error over the trajectory segment
231     Pose error;
232     error.x = optimized_poses_[end_idx].x - optimized_poses_[start_idx
        ↳ ].x;
233     error.y = optimized_poses_[end_idx].y - optimized_poses_[start_idx
        ↳ ].y;
234     error.theta = optimized_poses_[end_idx].theta - optimized_poses_[
        ↳ start_idx].theta;
235
236     // Normalize angle
237     while (error.theta > M_PI) error.theta -= 2*M_PI;
238     while (error.theta < -M_PI) error.theta += 2*M_PI;
239
240     size_t segment_length = end_idx - start_idx;
241
242     for (size_t i = start_idx + 1; i < end_idx; i++) {
243         double ratio = static_cast<double>(i - start_idx) /
        ↳ segment_length;
244
245         optimized_poses_[i].x -= ratio * error.x * 0.5; // Apply 50%
        ↳ correction
246         optimized_poses_[i].y -= ratio * error.y * 0.5;
247         optimized_poses_[i].theta -= ratio * error.theta * 0.5;
248
249         scans_[i].pose = optimized_poses_[i];
250     }
251 }
252
253 void SimpleSLAM::smoothTrajectory() {
254     if (optimized_poses_.size() < 3) return;
255
256     // Multiple passes of smoothing
257     for (int pass = 0; pass < 2; pass++) {
258         std::vector<Pose> smoothed_poses = optimized_poses_;
259
260         for (size_t i = 2; i < optimized_poses_.size() - 2; i++) {
261             // 5-point smoothing
262             smoothed_poses[i].x = 0.1 * optimized_poses_[i-2].x + 0.2 *
        ↳ optimized_poses_[i-1].x +
263                 0.4 * optimized_poses_[i].x + 0.2 *
        ↳ optimized_poses_[i+1].x + 0.1 *
        ↳ optimized_poses_[i+2].x;
264             smoothed_poses[i].y = 0.1 * optimized_poses_[i-2].y + 0.2 *
        ↳ optimized_poses_[i-1].y +
265                 0.4 * optimized_poses_[i].y + 0.2 *

```



```

266                                     ↪ optimized_poses_[i+1].y + 0.1 *
267                                     ↪ optimized_poses_[i+2].y;
268
269     // Smooth angle with circular mean
270     double sin_sum = 0.1 * std::sin(optimized_poses_[i-2].theta
271     ↪ ) + 0.2 * std::sin(optimized_poses_[i-1].theta) +
272     0.4 * std::sin(optimized_poses_[i].theta) +
273     ↪ 0.2 * std::sin(optimized_poses_[i
274     ↪ +1].theta) +
275     0.1 * std::sin(optimized_poses_[i+2].theta)
276     ↪ ;
277     double cos_sum = 0.1 * std::cos(optimized_poses_[i-2].theta
278     ↪ ) + 0.2 * std::cos(optimized_poses_[i-1].theta) +
279     0.4 * std::cos(optimized_poses_[i].theta) +
280     ↪ 0.2 * std::cos(optimized_poses_[i
281     ↪ +1].theta) +
282     0.1 * std::cos(optimized_poses_[i+2].theta)
283     ↪ ;
284     smoothed_poses[i].theta = std::atan2(sin_sum, cos_sum);
285 }
286
287     optimized_poses_ = smoothed_poses;
288
289     // Update scan poses
290     for (size_t i = 0; i < scans_.size(); i++) {
291         scans_[i].pose = optimized_poses_[i];
292     }
293 }
294
295     std::cout << "    Applied trajectory smoothing" << std::endl;
296 }
297
298 void SimpleSLAM::updateMap() {
299     if (!map_) return;
300
301     // Clear and rebuild map with optimized poses
302     map_->clear();
303
304     for (const auto& scan_data : scans_) {
305         map_->addScan(scan_data.points, scan_data.pose);
306     }
307
308     // Apply post-processing
309     map_->postProcessMap();
310 }
311
312 std::vector<Pose> SimpleSLAM::getOptimizedPoses() const {
313     return optimized_poses_;
314 }
315
316 void SimpleSLAM::saveMap(const std::string& filename) const {
317     if (map_) {
318         map_->saveToFile(filename);
319     }
320 }
321
322 void SimpleSLAM::saveTrajectory(const std::string& filename) const {
323     std::ofstream file(filename);

```

```

314     if (!file.is_open()) {
315         std::cerr << "Failed to open file: " << filename << std::endl;
316         return;
317     }
318
319     file << "# x y theta" << std::endl;
320     for (const auto& pose : optimized_poses_) {
321         file << std::fixed << std::setprecision(6)
322             << pose.x << " " << pose.y << " " << pose.theta << std::
323                 ↪ endl;
324     }
325
326     file.close();
327     std::cout << "Trajectory saved to: " << filename << std::endl;
328 }
329
330 int SimpleSLAM::getNumScans() const {
331     return static_cast<int>(scans_.size());
332 }
333
334 void SimpleSLAM::clear() {
335     scans_.clear();
336     optimized_poses_.clear();
337     next_scan_id_ = 0;
338
339     if (map_) {
340         map_->clear();
341     }
342 }
343
344 void SimpleSLAM::printStats() const {
345     std::cout << "\n=== SimpleSLAM Statistics ===" << std::endl;
346     std::cout << "Initialized: " << (initialized_ ? "Yes" : "No") <<
347         ↪ std::endl;
348     std::cout << "Number of scans: " << scans_.size() << std::endl;
349     std::cout << "Last processing time: " << last_processing_time_ << "
350         ↪ ms" << std::endl;
351
352     if (map_) {
353         std::cout << "Map size: " << map_->getWidth() << "x" << map_->
354             ↪ getHeight()
355             << " cells" << std::endl;
356         std::cout << "Map resolution: " << map_->getResolution() << "m/
357             ↪ cell" << std::endl;
358
359         double min_x, min_y, max_x, max_y;
360         map_->getBounds(min_x, min_y, max_x, max_y);
361         std::cout << "Map bounds: (" << min_x << ", " << min_y << ") to
362             ↪ ("
363             << max_x << ", " << max_y << ")" << std::endl;
364     }
365
366     if (!optimized_poses_.empty()) {
367         const auto& first = optimized_poses_.front();
368         const auto& last = optimized_poses_.back();
369         std::cout << "Trajectory: (" << first.x << ", " << first.y << "
370             ↪ ) to ("
371             << last.x << ", " << last.y << ")" << std::endl;

```

```

365     double total_distance = 0.0;
366     for (size_t i = 1; i < optimized_poses_.size(); i++) {
367         double dx = optimized_poses_[i].x - optimized_poses_[i-1].x
368             ↪ ;
369         double dy = optimized_poses_[i].y - optimized_poses_[i-1].y
370             ↪ ;
371         total_distance += std::sqrt(dx*dx + dy*dy);
372     }
373     std::cout << "Total distance traveled: " << std::fixed << std::
374     ↪ setprecision(2)
375         << total_distance << "m" << std::endl;
376 }
std::cout << "=====\n" << std::endl;
}

```

Listing 38: Batch SLAM pipeline (SimpleSLAM.cpp)

3.6.3 Comparison: Online vs. Offline SLAM

Characteristic	Online SLAM	Offline SLAM
Latency	Real-time (sub-10 ms updates)	Batch (minutes~hours)
Consistency	Local consistency; drift over time	Global consistency via loop-closure
Fidelity	Moderate; no global smoothing	High; sigmoid updates + median filter
Compute Load	Low; simple ray-tracing	High; multi-level matching + optimization
Memory Footprint	Fixed grid buffer	Large; stores all scans & poses
Use-Case	Reactive navigation, live dashboards	Detailed mapping, post-mission review

Online SLAM excels at keeping the robot safe and responsive, while Offline SLAM delivers a polished, drift-corrected map for documentation, analysis, and advanced planning. Online mapping enables real-time collision avoidance, and offline SLAM produces the final high-fidelity environment model.

3.7 Main Control Loop

3.7.1 Application Entry

This 10 ms periodic loop orchestrates all modules:

1. Read sensor data (UART + LiDAR scan).
2. Update EKF for pose estimation.
3. Trigger planning on new goal; cache waypoints.
4. Send velocity commands via UART.
5. Broadcast sequenced telemetry over UDP.
6. Monitor health: communication timeouts, CRC errors, missed frames; invoke safe-stop or fallback.

On shutdown, the loop terminates cleanly: all threads join, ports close, and logs flush to disk.

```

1 // main.cpp
2 #include "lidar/lidar_handler.hpp"
3 #include <opencv2/opencv.hpp>
4 #include "communication/serial_com.hpp"
5 #include "communication/udp_com.hpp"
6 #include "localization/RobotLocalization.hpp"
7 #include "localization/SensorFusion.hpp"
8 #include "localization/robot_utils.hpp"
9 #include <iostream>
10 #include <iomanip>
11 #include <fstream>
12 #include <boost/filesystem.hpp>
13 #include <chrono>
14 #include <thread>
15 #include <cmath>
16 #include "core/config.hpp"
17
18 namespace fs = boost::filesystem;
19
20 // Global Variables (used by both EKF and Basic modes)
21 ↪
22 ↪
23 SensorPacket          g_sensor;
24 MotionDebugPacket     g_debug;
25 CommandEchoPacket     echo;           // used for debugging
26
27 std::mutex            g_cmd_mtx;      // for thread safety
28 ↪ access
29 CommandPacket         g_cmd;
30
31 std::vector<LidarPoint> g_scan;      // holds the latest LiDAR scan
32 ↪ data
33
34 // EKF instance for localization (tuned noise parameters)
35 SensorFusion g_ekf(
36     RobotUtils::WHEEL_RADIUS,
37     RobotUtils::WHEEL_BASE,
38     0.01,           // dt
39     1e-2,           // processNoisePos
40     1e-3,           // processNoiseAng
41     1e-2,           // processNoiseVel
42     1e-3,           // gyroNoise
43     1e-2,           // imuNoise
44     1e-1            // accelNoise
45 );
46 //
47 ↪
48 ↪
49
50 /* ----- Global Access Functions ----- */
51 ↪ ----- */
52 const SensorPacket &getCurrentSensorData() { return g_sensor; }
53 const MotionDebugPacket &getCurrentDebugData() { return g_debug; }
54 const CommandPacket &getCurrentCommand() {
55     std::lock_guard<std::mutex> lk(g_cmd_mtx);
56     return g_cmd;
57 }

```

```

51
52 /* ----- LiDAR utility functions ----- */
53 ↪ -----
54 std::string timestamp() {
55     using clock = std::chrono::system_clock;
56     auto now = clock::now();
57     auto t    = clock::to_time_t(now);
58     auto us   = std::chrono::duration_cast<std::chrono::microseconds>(
59         now.time_since_epoch()).count() % 1'000'000;
60     std::ostringstream ss;
61     ss << std::put_time(std::localtime(&t), "%Y%m%d_%H%M%S")
62     << '_' << std::setw(6) << std::setfill('0') << us;
63     return ss.str();
64 }
65
66 void scanToImage(const std::vector<LidarPoint> &scan,
67                 cv::Mat &img,
68                 float rangeMax,
69                 int canvas = 600)
70 {
71     img = cv::Mat::zeros(canvas, canvas, CV_8UC3);
72     const cv::Point c(canvas/2, canvas/2);
73     const float ppm = (canvas/2)/rangeMax;
74     for (auto &p : scan) {
75         if (p.distance <= 0.01f || p.distance > rangeMax) continue;
76         float rad = p.azimuth * CV_PI/180.f;
77         cv::Point pt(c.x + std::cos(rad)*p.distance*ppm,
78                     c.y - std::sin(rad)*p.distance*ppm);
79         cv::circle(img, pt, 0.5, cv::Scalar(0,255,0), cv::FILLED);
80     }
81 }
82
83 void saveCSV(const std::string &fname, const std::vector<LidarPoint> &
84 ↪ scan) {
85     std::ofstream ofs(fname);
86     ofs << "azimuth_deg,distance_m,rssi\n";
87     for (auto &p : scan)
88         ofs << p.azimuth << ',' << p.distance << ',' << unsigned(p.rssi
89 ↪ ) << '\n';
90 }
91
92 /* ----- Main Function ----- */
93 ↪ -----
94 int main()
95 {
96     fs::create_directories("scans");
97
98     std::unique_ptr<Serial_Com> serial;
99     //LidarHandler          lidar;
100     UDPCom                 udp(LOCAL_UDP_PORT, REMOTE_IP,
101 ↪ REMOTE_UDP_PORT);
102     RobotLocalization      localize; // default-construct with
103 ↪ logging on
104
105     // ÃÃÃÃ NEW: record chosen serial port for RobotLocalization
106     std::string serialPortString;
107
108     auto ports = Serial_Com::getAvailablePorts();

```

```

103     udp.start();
104     //cv::namedWindow("Lidar Viewer");
105
106     if (ports.empty()) {
107         std::cout << "No serial ports found." << std::endl;
108         return 1;
109     }
110
111     for (const auto &port : ports) {
112         try {
113             std::cout << "Trying to open serial port: " << port << std
114                 ↪ ::endl;
115             serial = std::make_unique<Serial_Com>(port,
116                 ↪ DEFAULT_BAUD_RATE);
117
118             // âĀĀĀ NEW: save the successful port name
119             serialPortString = port;
120
121             // Set system state for debug + echo
122             SystemState sys;
123             sys.controlMode = AUTONOMOUS;
124             sys.debugMode   = MD_AND_ECHO;
125             sys.updateExpectations();
126             serial->setSystemState(sys);
127
128             std::cout << "Successfully opened serial port: " << port <<
129                 ↪ std::endl;
130             break;
131         }
132         catch (const std::exception &e) {
133             std::cout << "Failed to open " << port << ": " << e.what()
134                 ↪ << std::endl;
135         }
136     }
137
138     if (!serial) {
139         std::cerr << "Unable to connect to any serial port." << std::
140             ↪ endl;
141         return 1;
142     }
143
144     auto next      = std::chrono::steady_clock::now();
145     int loop_count = 0;
146     // define your loop period explicitly
147     constexpr auto period = std::chrono::milliseconds(10);
148
149     while (true)
150     {
151         auto now = std::chrono::steady_clock::now();
152         if (now >= next)
153         {
154             next += period;
155             ++loop_count;
156
157             // if (loop_count % 10 == 0)
158             // {
159             //     // Process LiDAR data
160             //     g_scan = lidar.getLatestScan();

```

```

156         //      lidar.dumpNextScan("scans/Range.txt", g_scan);
157
158         //      if (!g_scan.empty()) {
159         //          cv::Mat img;
160         //          scanToImage(g_scan, img, rangeMax, canvas);
161         //          cv::imshow("LakiBeam Viewer", img);
162         //          cv::waitKey(1);
163         //          std::string ts = timestamp();
164         //          //cv::imwrite("scans/" + ts + ".jpg", img);
165         //          //saveCSV("scans/" + ts + ".csv", g_scan);
166         //      }
167         // }
168
169         // Handle remote commands from UDP
170         if (g_cmd.cmdStatus == CMD_TOBE_WRITTEN)
171         {
172             serial->sendCommand(g_cmd);
173             g_cmd.cmdStatus = CMD_JUST_WROTE; // mark as sent
174             std::cout << "[SRL] Command sent: mode=" << int(g_cmd.
175                 ↪ mode);
176
177         }
178
179         // Read & parse serial
180         serial->spinOnce(g_cmd);
181
182         // Update globals
183         g_sensor = serial->getSensor();
184         g_debug = serial->getDebug();
185         auto echoPkt = serial->getCommandEcho();
186
187         // Send telemetry back over UDP
188         if (g_sensor.valid)
189             udp.sendSensor(g_sensor);
190         if (g_debug.valid)
191             udp.sendDebug(g_debug);
192
193         // Run our localization/EKF, then status & logging
194         localize.updateEKF(g_sensor);
195
196         if (echoPkt.valid) {
197             serial->printCommandEcho(echoPkt);
198             std::cout << std::endl;
199         }
200
201         localize.printStatus(g_sensor);
202         localize.logData(g_sensor);
203     }
204
205     // Small delay to prevent excessive CPU usage
206     std::this_thread::sleep_for(std::chrono::milliseconds(1));
207 }
208
209 // never reached, but clean up if we ever break out
210 udp.stop();
211 return 0;

```

Listing 39: Application Entry (main.cpp)

4 Remote Monitoring and Control Interface

The Remote Monitoring and Control Interface provides both a rich graphical dashboard and a lightweight command-line tool for real-time observation and teleoperation of the AMR over UDP. These two components: `interface.py` and `udp.py` cater to different user needs, whether you require detailed plots and parameter tuning or a minimal terminal interface.

4.0.1 Graphical User Interface

The GUI, built with PyQt5 and pyqtgraph, offers a comprehensive window where telemetry is visualized, controls are adjusted on the fly, and system logs are monitored.

Overview

A standalone Python application that:

- Receives UDP packets containing sensor readings, debug info, and velocity data.
- Displays live numeric values (speed, angular rate, wheel speeds) in labeled fields.
- Plots time series of key variables (SPD, ROT_SPD, LFT_FDSPD, etc.) with checkboxes to toggle series visibility.
- Provides input fields and "Send" buttons to adjust PID gains (FWD_KP, FWD_KI, ...) and transmit them over UDP.
- Includes Connect/Stop buttons to manage the background UDP listener thread.
- Shows raw and parsed messages, error notices, and acknowledgements in a scrolling log box.

Key Features

- **Threaded UDP Reception:** A dedicated `UDPReaderThread` listens in blocking mode and emits Qt signals for thread-safe UI updates.
- **Real-Time Plotting:** Uses `pyqtgraph` to update plots at high frame rates, automatically trimming buffers to a fixed length.
- **Parameter Validation:** Ensures invalid or empty inputs generate clear log messages rather than silent failures.
- **Graceful Shutdown:** Cleanly stops the UDP thread and closes the application on exit or error.

```
1 import sys
2 from PyQt5.QtWidgets import (
3     QApplication, QMainWindow, QVBoxLayout, QWidget, QTextEdit,
4     QPushButton,
5     QComboBox, QLabel, QHBoxLayout, QCheckBox, QLineEdit, QSplitter
6 )
7 from PyQt5.QtCore import QThread, pyqtSignal, Qt
8 import pyqtgraph as pg
9 import socket
10
11 class UDPReaderThread(QThread):
```



```

12     data_received = pyqtSignal(str)
13
14     def __init__(self, ip, port):
15         super().__init__()
16         self.ip = ip
17         self.port = port
18         self.running = True
19
20     def run(self):
21         try:
22             sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
23             sock.bind((self.ip, self.port))
24             while self.running:
25                 data, _ = sock.recvfrom(2048)
26                 try:
27                     message = data.decode('utf-8').strip()
28                 except UnicodeDecodeError:
29                     message = f"Received non-UTF-8 data: {data}"
30                 self.data_received.emit(message)
31         except Exception as e:
32             self.data_received.emit(f"UDP Error: {e}")
33         finally:
34             sock.close()
35
36     def stop(self):
37         self.running = False
38         self.wait()
39
40
41 class SerialMonitor(QMainWindow):
42     def __init__(self):
43         super().__init__()
44         self.setWindowTitle("Serial/UDP Monitor")
45         self.setGeometry(100, 100, 1200, 800)
46
47         self.udp_thread = None
48         self.plot_items = {}
49         self.header_checkboxes = {}
50
51         # Data series for graph
52         self.data_series = {
53             "SPD": [], "ROT_SPD": [], "EXP_SPD": [], "EXP_ROT": [],
54             "LFT_FDSPD": [], "RT_FDSPD": [], "LR_SPD": [], "RR_SPD":
55             ↪ [],
56         }
57
58         # Current numeric displays
59         self.current_values = {
60             "Current Speed": 0.0,
61             "Omega": 0.0,
62             "Expected Speed": 0.0,
63             "Expected Omega": 0.0,
64             "Left Front Speed": 0.0,
65             "Right Front Speed": 0.0,
66             "Left Rear Speed": 0.0,
67             "Right Rear Speed": 0.0,
68         }

```

```

69     self.plot_items = {}
70     self.initUI()
71
72     def initUI(self):
73         main_widget = QWidget(self)
74         self.setCentralWidget(main_widget)
75         main_layout = QHBoxLayout()
76
77         # Left panel: controls
78         left_panel = QWidget()
79         left_layout = QVBoxLayout()
80         left_panel.setLayout(left_layout)
81         left_panel.setMaximumWidth(400)
82
83         # UDP controls
84         udp_group = QWidget()
85         udp_layout = QVBoxLayout()
86
87         read_udp = QHBoxLayout()
88         self.ip_label = QLabel("Read IP:")
89         self.ip_input = QComboBox()
90         self.ip_input.addItem("0.0.0.0", "192.168.1.68")
91         self.port_label = QLabel("Port:")
92         self.port_input = QComboBox()
93         self.port_input.addItem("9001", "5005", "9000")
94         read_udp.addWidget(self.ip_label)
95         read_udp.addWidget(self.ip_input)
96         read_udp.addWidget(self.port_label)
97         read_udp.addWidget(self.port_input)
98         udp_layout.addLayout(read_udp)
99
100        send_udp = QHBoxLayout()
101        self.send_ip_label = QLabel("Send IP:")
102        self.send_ip_input = QLineEdit("192.168.1.84")
103        self.send_port_label = QLabel("Port:")
104        self.send_port_input = QLineEdit("9000")
105        send_udp.addWidget(self.send_ip_label)
106        send_udp.addWidget(self.send_ip_input)
107        send_udp.addWidget(self.send_port_label)
108        send_udp.addWidget(self.send_port_input)
109        udp_layout.addLayout(send_udp)
110
111        buttons = QHBoxLayout()
112        self.connect_button = QPushButton("Connect")
113        self.connect_button.clicked.connect(self.connect_to_udp)
114        self.stop_button = QPushButton("Stop")
115        self.stop_button.setEnabled(False)
116        self.stop_button.clicked.connect(self.stop_reading)
117        buttons.addWidget(self.connect_button)
118        buttons.addWidget(self.stop_button)
119        udp_layout.addLayout(buttons)
120
121        udp_group.setLayout(udp_layout)
122        left_layout.addWidget(udp_group)
123
124        # Real-time values
125        values_group = QWidget()
126        value_layout = QVBoxLayout()

```

```

127     self.value_labels = {}
128     for key in self.current_values:
129         label = QLabel(f"{key}: 0.0")
130         self.value_labels[key] = label
131         value_layout.addWidget(label)
132     values_group.setLayout(value_layout)
133     left_layout.addWidget(values_group)
134
135     # Parameter inputs
136     params_group = QWidget()
137     param_layout = QVBoxLayout()
138     self.params = {}
139     for param in ["FWD_KP", "FWD_KD", "FWD_KI", "ROT_KP", "ROT_KD",
140                  ↪ "ROT_KI"]:
141         row_layout = QHBoxLayout()
142         label = QLabel(f"{param}:")
143         input_field = QLineEdit()
144         input_field.setPlaceholderText("Enter value")
145         button = QPushButton("Send")
146         button.clicked.connect(lambda _, p=param, i=input_field:
147                               ↪ self.send_param_udp(p, i))
148         row_layout.addWidget(label)
149         row_layout.addWidget(input_field)
150         row_layout.addWidget(button)
151         param_layout.addLayout(row_layout)
152         self.params[param] = input_field
153     params_group.setLayout(param_layout)
154     left_layout.addWidget(params_group)
155
156     # Other messages
157     self.other_output = QTextEdit()
158     self.other_output.setReadOnly(True)
159     self.other_output.setPlaceholderText("Other Messages")
160     self.other_output.setMaximumHeight(100)
161     left_layout.addWidget(QLabel("Messages:"))
162     left_layout.addWidget(self.other_output)
163     left_layout.addStretch()
164
165     # Right panel: graph
166     right_panel = QWidget()
167     right_layout = QVBoxLayout()
168     right_panel.setLayout(right_layout)
169
170     self.plot_widget = pg.PlotWidget()
171     self.plot_widget.setBackground('w')
172     self.plot_widget.setTitle("Real-Time Data Plot", color='b',
173                               ↪ size="12pt")
174     self.plot_widget.setLabel("left", "Value", color='blue')
175     self.plot_widget.setLabel("bottom", "Sample", color='blue')
176     self.plot_widget.showGrid(x=True, y=True)
177     right_layout.addWidget(self.plot_widget)
178
179     plot_controls = QHBoxLayout()
180     colors = {
181         "SPD": "r", "ROT_SPD": "g", "EXP_SPD": "b", "EXP_ROT": "m",
182         "LFT_FDSPD": "y", "RT_FDSPD": "c", "LR_SPD": (255,128,0), "
183         ↪ RR_SPD": (128,0,128)
184     }

```

```

181     for header, color in colors.items():
182         cb = QCheckBox(header)
183         cb.setChecked(True)
184         cb.stateChanged.connect(self.update_plot_visibility)
185         self.header_checkboxes[header] = cb
186         self.plot_items[header] = self.plot_widget.plot(pen=pg.
            ↪ mkPen(color=color, width=2), name=header)
187         plot_controls.addWidget(cb)
188     right_layout.addLayout(plot_controls)
189
190     main_layout.addWidget(left_panel)
191     main_layout.addWidget(right_panel, stretch=2)
192     main_widget.setLayout(main_layout)
193
194     def update_output(self, data):
195         # ---- handle SENSOR messages ----
196         if data.startswith("SENSOR,"):
197             parts = data.split(",")[1:]
198             try:
199                 yaw, roll, pitch = map(float, parts[0:3])
200                 # you can display or log these if desired...
201                 self.other_output.append(f"Yaw={yaw:.1f}, Roll={roll:.1
            ↪ f}, Pitch={pitch:.1f}")
202             except:
203                 self.other_output.append(f"Malformed SENSOR: {data}")
204
205         # ---- handle DEBUG messages ----
206         elif data.startswith("DEBUG,"):
207             parts = data.split(",")[1:]
208             try:
209                 spdL, spdR, vel, omg, dist, ang, dt = map(float, parts)
210                 # update your graph series:
211                 self.data_series["SPD"].append(vel)
212                 self.data_series["ROT_SPD"].append(omg)
213                 # trim
214                 for key in ["SPD", "ROT_SPD"]:
215                     if len(self.data_series[key]) > 2000:
216                         self.data_series[key].pop(0)
217                 # update labels
218                 self.value_labels["Current Speed"].setText(f"Current
            ↪ Speed: {vel:.2f}")
219                 self.value_labels["Omega"].setText(f"Omega: {omg:.2f}")
220                 # update plots
221                 self.update_plot("SPD")
222                 self.update_plot("ROT_SPD")
223             except:
224                 self.other_output.append(f"Malformed DEBUG: {data}")
225
226         # ---- existing VELOCITY: branch ----
227         elif data.startswith("VELOCITY:"):
228             try:
229                 parts = data.split(":")[1].split(",")
230                 exp_spd, spd, exp_rot, rot_spd, lft_fdspd, rt_fdspd,
            ↪ lr_spd, rr_spd = map(float, parts)
231                 series = {
232                     "EXP_SPD": exp_spd, "SPD": spd,
233                     "EXP_ROT": exp_rot, "ROT_SPD": rot_spd,
234                     "LFT_FDSPD": lft_fdspd, "RT_FDSPD": rt_fdspd,

```

```

235         "LR_SPD": lr_spd, "RR_SPD": rr_spd
236     }
237     for key, val in series.items():
238         self.data_series[key].append(val)
239         if len(self.data_series[key]) > 2000: self.
240             ↳ data_series[key].pop(0)
241         self.value_labels[key.replace("_", " ").title()].
242             ↳ setText(f"{key.replace('_', ' ')}: {val:.2f}")
243         self.update_plot(key)
244     except Exception as e:
245         self.other_output.append(f"Invalid VELOCITY format: {
246             ↳ data}")
247
248     else:
249         self.other_output.append(data)
250
251 def connect_to_udp(self):
252     ip = self.ip_input.currentText()
253     port = int(self.port_input.currentText())
254     try:
255         self.udp_thread = UDPReaderThread(ip, port)
256         self.udp_thread.data_received.connect(self.update_output)
257         self.udp_thread.start()
258         self.other_output.append(f"Listening on {ip}:{port}")
259         self.connect_button.setEnabled(False)
260         self.stop_button.setEnabled(True)
261     except Exception as e:
262         self.other_output.append(f"UDP Connection Error: {e}")
263
264 def stop_reading(self):
265     if self.udp_thread:
266         self.udp_thread.stop()
267         self.udp_thread = None
268     self.other_output.append("Stopped reading.")
269     self.connect_button.setEnabled(True)
270     self.stop_button.setEnabled(False)
271
272 def update_plot(self, header):
273     if self.header_checkboxes[header].isChecked():
274         self.plot_items[header].setData(self.data_series[header])
275     else:
276         self.plot_items[header].clear()
277
278 def update_plot_visibility(self):
279     for header, cb in self.header_checkboxes.items():
280         if not cb.isChecked():
281             self.plot_items[header].clear()
282         elif self.data_series[header]:
283             self.plot_items[header].setData(self.data_series[header
284                 ↳ ])
285
286 def send_param_udp(self, param, input_field):
287     value = input_field.text()
288     if value:
289         try:
290             sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
291             send_ip = self.send_ip_input.text()
292             send_port = int(self.send_port_input.text())

```

```

289         message = f"{param}={value}"
290         sock.sendto(message.encode('utf-8'), (send_ip,
291             ↪ send_port))
292         self.other_output.append(f"Sent: {message} to {send_ip
293             ↪ }:{send_port}")
294     except Exception as e:
295         self.other_output.append(f"Error sending {param}: {e}")
296     else:
297         self.other_output.append(f"No value for {param}")
298
299 def main():
300     app = QApplication(sys.argv)
301     window = SerialMonitor()
302     window.show()
303     sys.exit(app.exec_())
304
305 if __name__ == "__main__":
306     main()

```

Listing 40: GUI Dashboard implementation (`interface.py`)

4.0.2 Command-Line Interface

For scenarios where a GUI is impractical, the CLI tool provides direct keyboard teleoperation and real-time telemetry display in the terminal.

Overview A Python script that:

- Binds a UDP socket to receive and print all incoming robot messages with timestamps.
- Switches the terminal to raw mode to capture single-key presses (no Enter required).
- Maps W/A/S/D keys to forward, left, backward, and right commands, sending TELEOP, f, b, l, r UDP packets.
- Monitors the socket and stdin via `select()`, ensuring responsive handling of both telemetry and keypresses.
- Restores original terminal settings on exit (keypress Q or Ctrl-C).

Key Features

- **Non-Blocking I/O:** Uses `select()` to multiplex UDP and keyboard without threading.
- **Terminal Management:** Employs `termios` and `tty` for raw mode and registers an `atexit` handler for safe cleanup.
- **Immediate Feedback:** Prefixes sent commands with [SEND] and prints incoming packets with a timestamp and sender address.

```

1  #!/usr/bin/env python3
2  import socket
3  import sys
4  import select
5  import termios
6  import tty
7  import atexit
8
9  # UDP listen & send configuration
10 LOCAL_IP      = "192.168.1.68"          # listen on all interfaces
11 LOCAL_PORT    = 9001                   # where we receive robot telemetry
12 ROBOT_IP      = "192.168.1.84"         # robot's UDP listener
13 ROBOT_PORT    = 9000                   # robot listens for TELEOP
14
15 # Set up sockets
16 sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
17 sock.bind((LOCAL_IP, LOCAL_PORT))
18 print(f"Listening for UDP on {LOCAL_IP}:{LOCAL_PORT}\n")
19
20 # Save terminal settings so we can restore later
21 orig_settings = termios.tcgetattr(sys.stdin)
22 def restore_terminal():
23     termios.tcsetattr(sys.stdin, termios.TCSADRAIN, orig_settings)
24 atexit.register(restore_terminal)
25
26 # Put stdin into raw mode for single character reads
27 tty.setcbreak(sys.stdin.fileno())
28
29 def send_teleop(f, b, l, r):
30     msg = f"TELEOP,{int(f)},{int(b)},{int(l)},{int(r)}"
31     sock.sendto(msg.encode('utf-8'), (ROBOT_IP, ROBOT_PORT))
32     print(f"[SEND] {msg}")
33
34 print("Press W/A/S/D for forward/left/back/right, Q to quit.")
35
36 try:
37     while True:
38         # Use select to wait for either UDP or keyboard (timeout 0.1s)
39         rlist, _, _ = select.select([sock, sys.stdin], [], [], 0.1)
40
41         # Handle UDP telemetry
42         if sock in rlist:
43             data, addr = sock.recvfrom(4096)
44             text = data.decode('utf-8', errors='replace').strip()
45             print(f"[{addr[0]}:{addr[1]}] {text}")
46
47         # Handle keypress
48         if sys.stdin in rlist:
49             ch = sys.stdin.read(1).lower()
50             if ch == 'q':
51                 print("Quitting.")
52                 break
53             # Map keys to teleop bit flags
54             f = b = l = r = 0
55             if ch == 'w':
56                 f = 1
57             elif ch == 's':

```

```

58         b = 1
59     elif ch == 'a':
60         l = 1
61     elif ch == 'd':
62         r = 1
63     else:
64         continue
65     send_teleop(f, b, l, r)
66
67 except KeyboardInterrupt:
68     print("\nInterrupted, exiting.")
69 finally:
70     restore_terminal()
71     sock.close()

```

Listing 41: CLI Teleoperation and Telemetry tool (`udp.py`)

5 Conclusion

The AMR software framework detailed herein achieves a robust balance between hard-real-time performance and high-fidelity environment modeling. At the MCU level, the ATmega32U4 firmware provides deterministic control loops for precise sensor reading, odometry computation, motion profiling, and stepper-motor actuation. Meanwhile, the Jetson Nano’s software stack fuses sensor data via an EKF, maintains both online and offline SLAM pipelines for real-time mapping and batch global optimization, and generates collision-free paths using A*, PRM, and hybrid algorithms.

On the user side, the dual-interface approach combining a PyQt5 GUI with a command-line teleoperation tool, caters to varied deployment scenarios, from rich visual debugging to headless operation in constrained environments. This versatility ensures that end users can not only steer and tune the robot in real time but also review detailed telemetry logs and high-quality maps after each mission.

Looking ahead, this modular and well-documented architecture lays the groundwork for advanced capabilities such as dynamic obstacle avoidance, vision-based perception, and coordinated multi-robot systems. By building atop a solid foundation of deterministic control, flexible mapping, and seamless human-robot interaction, future enhancements can be integrated smoothly to further elevate the AMR’s autonomy and practical utility.